



UNIVERSITY OF TRENTO

DOCTORAL SCHOOL IN
INFORMATION AND COMMUNICATION TECHNOLOGY

EMBEDDED REAL-TIME MUSICAL PATTERN DETECTION FOR SMART MUSICAL INSTRUMENTS

Advisor
Prof. Luca Turchet

PhD candidate
Nishal Stanislaus Silva
228991

January 2025



*"When I'm working, I like to think of the process as a cosmic dance. You let the
music guide you, you follow the rhythm of the universe,
and before you know it, you're creating something truly unique.
It's like being in a dream where the only rule is to let go and embrace the melody."*

- Hendrix

Abstract

This thesis explores the development of embedded real-time musical pattern detection (RTPD) systems for smart musical instruments (SMI), a class of musical instruments which integrate sensors, embedded intelligence, and wireless connectivity. We focused on enabling SMIs to detect pre-defined musical patterns in real-time, which can then be used as triggers to control external devices such as stage lights, smoke machines, haptic devices and mixed reality headsets. This work is situated within the broader context of the Internet of Musical Things (IoMusT), a networked ecosystem of interconnected musical devices and systems.

Most existing pattern detection methods are designed for offline contexts - i.e., to detect all occurrences of a pattern in an entire composition. Before the beginning of this thesis work, real-time implementations of such systems did not exist. This thesis fills this research gap by introducing a real-time algorithm optimized for embedded systems with constrained computational resources.

This thesis begins by discussing the challenges of RTPD, especially in live performance where low latency and minimal memory consumption are paramount. Then three datasets tailored for RTPD research are introduced. The Dataset of Musical Patterns, the Dataset of Drum Patterns, and the Dataset of Polyphonic Patterns were developed with contributions from professional musicians across diverse backgrounds. They capture expressive and artistic variations a performer might do during a live performance, and they address gaps in existing resources by focusing on intentional variations in pattern repetitions.

These datasets allowed for the development of *Hot Licks*, an open-source RTPD algorithm that can be integrated into SMIs. Hot Licks was deployed on different SMIs prototypes, and it demonstrated high accuracy in detection as well as a minimal latency. Hot Licks was recognized as a finalist in the prestigious MIDI awards 2023.

User studies involving professional musicians evaluated the SMIs prototypes' usability and effectiveness. Results indicated positive reception, highlighting the potential of RTPD-enabled SMIs to enhance creativity and audience engagement. Furthermore, multi-sensory performances were conducted using these instruments, showcasing their ability to synchronize music with visual and tactile stimuli in real-time. The findings underscore the potential of RTPD in live music contexts, paving the way for new artistic possibilities and interactive experiences.

This thesis concludes by reflecting on its contributions to the fields of music technology and IoMusT while outlining future research directions, including advancements in polyphonic detection algorithms, integration with virtual reality environments, and broader applications in education and therapy.

Keywords

Music Information Retrieval, Real-time Pattern Detection, Deep Learning, Embedded Systems, Smart Musical Instruments.

Acknowledgments

My first thanks go to my parents. Thank you for raising me to be the man I am today. Thank you for the guidance, the support, and everything you've given me. I hope I've made you proud!

To my sister, thank you for the continuous support from the other side of the world. Thank you for the guidance, and the support, and the encouragement. My thanks also goes to Gayan ayya and Amanda for being a consistent source of support.

My thanks go to my extended family. Chooti, Dilli, Sashini, Rashane, and Mots; thank you for the support and the encouragement.

I must give my humble thanks to my supervisor, Luca Turchet. Thank you for having faith in me and for taking me under your wing.

My thanks also go to Chathuranga Weeraddana, who supervised my Master's research, and who introduced me to this wonderful journey of Doctoral research.

Thank you to all my colleagues turned friends. Thank you Dome, for being my fellow geek in the office. Luca V, thanks for being my friend during tough times. Thanks for the crazy jam sessions, and for geeking out over gear. Thanks to Tomson for the laughs, both in Italy and In Montreal. Alberto, thanks for the support and the guidance, and lending an ear when I needed to vent. Thanks Greg, for the laughs. Red Mike, I do love the random jam sessions we have in the office. White Mike, thanks for the jokes and good times. Filippo, thanks for your support, thanks for calling people on behalf of me and for being my translator.

Moving to a foreign country is hard. Moving to a completely new culture and Language is even harder. But there are a few people who made me feel like I never left home. Alice, you've become one of my closest friends. And you are the best flatmate I've had. Thank you for the good times, the good food, and everything in between. Thank you for welcoming me into your family. The Savini's, thank you for opening your doors and giving me a place at your table for the holidays. It will be one of my most cherished memories. The Bartolini's, thank you for welcoming into your house and feeding me truffles from your backyard. Thank you for adopting me as the weird caramel cousin in your family.

My darling Gambling Bambi, thank you for all the good times. Thank you for being there and listening when I needed it the most. Sindik, thank you for checking up on me when I was feeling down, and for being the friend who rings my doorbell at random times.

Sergio, you were my first friend in Trento. Thank you for including me in the group of people who I now call friends. Bea, thanks for being the responsible one. And thank you for the wonderful lunches and dinners.

Alice, Lore, and Signor Arches; you came to my home in Sri Lanka. We had an amazing roadtrip. Thank you for the good times. Thank you for infiltrating into my family, and bringing a bit of my culture with you to Italy.

Pippo, thank you for being there for me, and for never saying no to another beer. Rocche, thanks for the laughs and the drunken philosophical chats. Madda, you are one of the very few people who could handle spice. I hope you use put my chilli oil recipe to good use. Adelaiade, you once said I'm like your Nonna who feeds you - that is one of the highest compliments anyone could ever get. Lore, thank you for the good times, the laughs, and thank you for introducing my people's food to your family. Gabriella, thanks for the wonderful Belgian beer, and for listening to my drunken rants, and for being an amazing friend. Giacomo, thanks for the good times, the good talks, the weird DIY projects, and the food. Ashkan, thanks for the wonderful Persian food from you and from your ma'. Irma, you barged into my house and stole my sarong that one day - only a good friend would do that. Thank you for being that. Rugge, thanks for the good times, and for being my spotter in the gym. Samuele and Illaria, thanks for the good times, the drunk times, and the crazy times. To you, and anyone I may have missed, thank you for welcoming me with open arms.

To the people who made my stay in Montreal just a little bit better. Dr.G, Paloma, Joao, Aline, thank you for everything. Thank you for the good times, the drunken karaokes, the lunch table gossip, and for being there when I needed to vent.

To my boys back home, Iranga and Bilal. We started off at the last row in school, and 15 years later, you are probably my oldest and most consistent friends, and you will continue to be so. Thank you for being by my side in thick and in thin. Thank you for all the good times.

Keshan, you were a mentor, a friend, and a brother. Thank you for the continued support. Rushane, we don't talk as much, but I do appreciate all the jams and the sing songs we had. Thank you for those. To Arjuna, I remember the smile on your face when I told you I got into the PhD. Thank you for being there when I needed someone. You're not here anymore, but when I see you on the other side, I hope you'll have a drink ready for me. Ransith and Dinka, I will never forget your support, in work, and in life. My boys from the garage: Waney, Malaka, Shameera, Kitty, Chanuka, Ishara; thanks for the support, the encouragement, and for being there.

To all my guys from uni: Shihaan, Gal Podda, Abi, Mahesh; thanks for keeping in touch despite being scattered around the world. Thanks for the good times, the food, the songs, and the many many failed science experiments. Those were some of my fondest memories.

There are so many I should thank, but I can only write so many names in this thesis. To everyone I've met, to everyone who shared a drink with me, to everyone who brightened by day with a smile, to everyone who invited me into their homes, to everyone who encouraged me, and lastly to everyone who stabbed me in the back. I am who I am because of you, and for that I give you my thanks.

Contents

List of Tables	ix
List of Figures	xi
Acronyms	xv
1 Prelude	1
1.1 Background	1
1.1.1 Smart Musical Instruments (SMIs)	1
1.1.2 The Internet of Musical Things	3
1.1.3 Real-Time Pattern Detection	3
1.2 Motivation and Aims	5
1.3 Outcomes	6
1.3.1 Publications	6
1.3.2 Submitted Articles	6
1.3.3 Software	7
1.3.4 Datasets	7
1.3.5 Demos	7
1.3.6 Awards	8
1.3.7 Videos	8
1.4 Thesis structure	9
2 Game of Tones: A Song of Patterns and Detection	11
2.1 Symbolic Representation-Based Methods	11
2.2 Polyphonic Audio Classification and Offline Applications	12
2.3 Machine Learning Approaches	15
2.4 Real-time Challenges	16

2.5	Applications of Multi-sensory Performance and Immersive Technologies	16
2.6	Summary	18
3	The Sound of Data	19
3.1	Composing the Monophonic Dataset	22
3.1.1	MIDI Magic: Why Go Symbolic?	22
3.1.2	Dataset Recording	23
3.2	Uncovering Musical DNA	28
3.3	Rhythm in Repetition	30
3.3.1	Groove Analytics: Decoding Patterns and Crunching the Stats	33
3.4	The Sound of Many Voices	34
3.4.1	Harmony in Numbers: Unlocking the Secrets of Polyphonic Patterns . . .	36
3.5	Grooving Through the Data: Patterns, Beats, and Musical Feats!	37
4	The Music Matrix: A Framework for Real-Time Pattern Recognition	39
4.1	Music in Translation: From Notes to Numbers	40
4.1.1	MIDI Reloaded: Building the Matrix	40
4.1.2	Sampling the Sound: Capturing Music in Time	43
4.1.3	Cepstral Tales: Decoding Timbre, One Coefficient at a Time	43
4.2	Algorithmic Jam Session: Comparing Approaches	46
4.2.1	The Structural Similarity Matrix: Mapping Musical Connections	47
4.2.2	Dynamic Time Warping: Aligning Patterns Across the Beat	50
4.2.3	Recurrent Neural Network: Learning Loops and Layers in Music	54
4.2.4	ONNX	59
5	Hitting the Right Notes: Engineering Real-Time Detection Systems	61
5.1	Overture to Intelligence	61
5.2	The Digital Heartbeat: Elk Audio OS	62
5.2.1	The Quantum Pulse: real-time kernel	62
5.2.2	Headless DAW: The Ghost in the Machine	64
5.3	Hot Licks	65
5.3.1	JUCE	66
5.3.2	The Matrix of Sound: Hot Licks Architecture	66
5.4	The Cloud Nexus: Training, Model Updates, and Server Integration	70
5.5	Hot Licks Controller: The Digital Conductor	71
5.6	Summary	75
6	The User Perspective: Musicians' Interaction with Real-Time Detection	77
6.1	Musicians and Machines: The Human Factor in Pattern Detection	77
6.1.1	Procedure	78
6.1.2	Questionnaire quantitative Analysis	84

6.1.3	Questionnaire qualitative analysis	84
7	Symphony of the Senses: Crafting the multi-sensory Experience	89
7.1	Ecosystem description	90
7.1.1	Smart Musical Instruments	90
7.1.2	Musical Things for the audience	91
7.1.3	Router, central server and tablet	92
7.1.4	Hot Licks Mapper	92
7.2	Evaluation	96
7.3	Discussion	98
8	Grand Finale: The Second Performance and Audience Impact	101
8.1	Introduction	101
8.2	Experiment Design	102
8.2.1	Musical Repertoire and Performance Structure	102
8.2.2	Smart Musical Instruments and Pattern Detection	103
8.2.3	Musical Things	103
8.3	Audience Setup	107
8.4	Multi-sensory Performances	110
8.5	Results and Discussion	112
8.5.1	Quantitative Analysis - Audience	112
8.5.2	Quantitative Analysis - Musicians	114
8.5.3	Qualitative Analysis - Audience	114
8.5.4	Qualitative Analysis - Musicians	116
8.5.5	Qualitative Analysis - Composers	117
8.6	Conclusion	118
9	Echoes of Progress: Reflections and Future Directions	119
9.1	Research Contributions	119
9.2	Performance Evaluation and User Experience	120
9.3	Future Implications and Research Directions	121
	Bibliography	123
A		137
A.1	MIDI velocity and Musical dynamics	137
A.2	General MIDI Percussion Key Map	138
A.3	List of musicians Involved in User Study I	139
A.4	User Study I - results	141

B	143
B.1 User Study I - Images and Plots	143
B.2 Match Measure (MM) computation	148
B.3 Logic Flow of Hot Licks Controller	150

List of Tables

3.1	Comparison of Music Datasets.	21
3.2	The primary message types used by the Musical Instrument Digital Interface (MIDI) protocol.	23
3.3	Average length, amplitude variation, and temporal deviation of the patterns in Dataset of Monophonic Patterns (DoMP).	29
6.1	The average latency values for the pattern detection and to emit a control message upon detecting a pattern, along with the average CPU and RAM usage.	81
6.2	Average Precision, Recall, and F1 score across all improvisations.	82
7.1	The initial mappings between musical patterns and multi-sensory stimuli used during the first part of the performance.	97
8.1	The mappings between musical patterns and multi-sensory stimuli used during the performance.	104
A.1	<i>MIDI velocity and dynamics mapping in Logic Pro 9 (retrieved from [1]).</i>	137
A.2	MIDI Drum Mapping	138
A.3	Details of the musicians who participated in the user study.	139
A.4	Results of the user study I	141

List of Figures

1.1	The Sensus smart guitar - augmented with sensors, onboard processing and wireless communication [2].	2
1.2	Examples for repetition in music.	4
1.3	MIDI innovation awards 2023 - Finalist.	8
3.1	An example to demonstrate monophony vs. polyphony	23
3.2	The first four notes of the C major scale in standard music notation.	24
3.3	The first four notes of the C major scale in piano roll notation.	24
3.4	The MIDI messages required to generate the first four notes of the C major scale.	24
3.5	Fishman TriplePlay Connect	25
3.6	The file tree structure of the DoMP.	27
3.7	The Yamaha DTX-432k electronic drum kit used to record Dataset of Drum Patterns (DoDP).	31
3.8	The file tree structure of the DoDP.	32
3.9	The recording pipeline for all 3 datasets.	36
3.10	The file tree structure of the Dataset of Polyphonic Patterns (DoPP).	37
4.1	Pitch bend values of an example musical note.	42
4.2	The first four notes of the C major scale - different representations	44
4.3	Block diagram comparing the different methods used.	46
4.4	Flow chart of the proposed algorithm.	48
4.5	The first four notes of the C major scale (<i>top</i>), and the corresponding matrix-based representation with pitch and duration (<i>middle</i>), and sampled sequence with 30ms sampling frequency (<i>bottom</i>).	51
4.6	C, D, E, F notes with a short pause (top). The same notes, but with a significantly larger pause (bottom). Due to the longer pause, the two patterns will not seem perceptually equivalent despite the pitches being identical.	53

List of Figures

4.7	A block diagram of the DTW algorithm.	54
4.8	A Comparison between different RNN layer configurations	56
4.9	A Block diagram of evaluation of the RNN algorithm.	59
5.1	A block diagram of Elk Audio OS.	63
5.2	HifiBerry ADC+DAC pro audio interface.	63
5.3	Architecture of Sushi: the headless DAW.	65
5.4	Multi threaded approach of Elk Audio OS.	67
5.5	A sample application of Hot Licks.	70
5.6	Model training pipeline involving cloud computing.	71
5.7	Hot Licks Controller: initial page.	73
5.8	Hot Licks Controller: session configuration page.	73
5.9	Hot Licks Controller: recording page.	74
5.10	Hot Licks Controller: training page.	74
5.11	Hot Licks Controller: session load page.	75
6.1	A block diagram of the user study procedure.	79
6.2	Average Precision across the 3 improvisations for each musician.	82
6.3	Average Recall across the 3 improvisations for each musician.	82
6.4	Average F1 Score across the 3 improvisations for each musician.	83
6.5	Average accuracy, precision, and f1 score across the 3 improvisations for each musician.	83
6.6	Average accuracy, precision, and f1 score across the 3 improvisations, presented separately for the guitarists and the keyboard players.	84
6.7	Results of the questionnaire after improvisation 1.	85
6.8	Results of the questionnaire after improvisation 2.	85
6.9	Results of the questionnaire after improvisation 3.	85
6.10	Results of the questionnaire after all improvisations.	86
6.11	Results of the questionnaire after each improvisation.	86
6.12	Results of the final questionnaire after completing all 3 improvisations.	87
7.1	The musicians playing the Smart Musical Instruments (SMIs). The lights and the smoke machine were triggered by the SMIs.	90
7.2	Block diagram of the smart electric guitar, the smart keyboard, and the smart drum.	91
7.3	The developed smart electric guitar	91
7.4	Musical things controlled by the SMIs.	92
7.5	A block diagram of the operation of the mapping system.	93
7.6	A diagram of the functionality of the proposed mapping system.	94

7.7	A screenshot of the initial mapping between the patterns and musical things, as described in Table 7.1.	95
8.1	How each instrument's patterns can be spread within the timeline of the song. .	103
8.2	The smart keyboard and the smart drum kit, with the indication of the respective embedded systems.	104
8.3	The connections between SMIs and Musical Things.	106
8.4	The audience of the performance, wearing the Mixed Reality (MR) headsets and holding the mobile phones.	108
8.5	A diagram of the experiment setting. The performers are shown by the green circles, the audience members are shown by the orange circles. The purple boxes indicate stage lights, the red box indicates the smoke machine, and the blue boxes indicate the position of the loudspeakers.	108
8.6	Screenshot of the MR headset view.	109
8.7	Mean and standard error for the two performances.	113
8.8	Participants' preference between the RTPD and random triggered performances. .	113
8.9	3D Graphics coherence measured against participants experience with VR headsets.	114
8.10	Results of the questionnaire given to the musicians.	115
B.1	A pattern played by one of the keyboard players during the study.	144
B.2	A pattern played by one of the guitar players during the study.	145
B.3	The improvisations by one of the keyboard players	146
B.4	The improvisations by one of the guitar players	147
B.5	Logic Flow of Hot Licks Controller	150

Acronyms

AI Artificial Intelligence. 80, 84

AR Augmented Reality. 17

DAW Digital Audio Workstation. 62, 64

DCT Discrete Cosine Transform. 45

DMI Digital Musical Instrument. 61, 111

DoDP Dataset of Drum Patterns. xi, 7, 19, 30–34, 37, 38

DoMP Dataset of Monophonic Patterns. ix, xi, 7, 19, 22, 23, 25, 27–38

DoPP Dataset of Polyphonic Patterns. xi, 7, 19, 34, 36–38

DSP Digital Signal Processing. 1, 39, 46

DSPaaS Digital Signal Processing as a Service. 70, 75

DTW Dynamic Time Warping. 39, 46, 47, 50, 52

FFT Fast Fourier Transform. 44, 45

GUI Graphical User Interface. 62, 64, 66, 71, 72, 75

IoMusT Internet of Musical Things. 3, 5, 6, 101, 121

IoT Internet of Things. 3, 61

JUCE Jules’ Utility Class Extensions. 65, 66, 69, 72

MFCC Mel Frequency Cepstral Coefficient. 43–46, 65–67

MIDI Musical Instrument Digital Interface. ix, xi, 19, 20, 22–26, 28, 30, 32, 33, 35, 40, 61, 64–67, 75

MIR Music Information Retrieval. 3, 19

ML Machine Learning. 1, 7, 46, 61, 62, 70

MM Match Measure. viii, 49, 50, 148, 149

MR Mixed Reality. xiii, 92, 96–99, 101, 103, 107–109, 112

ms milli-seconds. 43

OSC Open Sound Control. 64–66, 69, 71–73, 91, 93, 103, 107, 110

RNN Recurrent Neural Network. vi, 39, 43, 46, 47, 54, 65–67, 69, 71, 103, 119

RTPD Real-time Pattern Detection. 1, 2, 5–8, 11, 39, 40, 77, 92, 101, 102, 110–112, 114, 116–121

SIMD Single Instruction/Multiple Data. 69

SMI Smart Musical Instrument. v, xii, xiii, 1–3, 5, 6, 16, 45, 61, 62, 64, 65, 75, 77, 89–94, 96, 98, 99, 101, 104, 106, 119–121

SSIM Structural Similarity Matrix. vi, 39, 40, 46–50, 149

VR Virtual Reality. 16, 93, 107

VST Virtual Studio Technology. 62, 65, 71, 75, 119

Chapter 1

Prelude

This thesis aims to document the doctoral research done on Embedded Real-time Pattern Detection (RTPD) for Smart Musical Instrument (SMI). The ultimate aim of this research was to develop a class of SMIs with built-in pattern detection capabilities. These SMIs are able to detect when a previously defined pattern is played on it, and then use it as a trigger to control external peripherals (i.e., stage lights, smoke machines) which could provide musicians with another output that complement their music. This chapter highlights the motivations and aim of this research (Section 1.2). The main outcomes of this work is presented in Section 1.3. The publications associated with this thesis (Section 1.3.1), submitted articles (Section 1.3.2), published software and datasets (Section 1.3.3 and Section 1.3.4), demos and talks relative to this thesis work (Section 1.3.5), and open source software and datasets delivered (Section 1.3.3) are also presented. This chapter concludes by introducing the structure of this thesis.

1.1 Background

1.1.1 Smart Musical Instruments (SMIs)

SMIs represent a fusion of traditional craftsmanship and modern technological innovation which incorporate embedded electronics, sensors, and software to enhance the capabilities beyond the conventional limits of their acoustic counterparts. The integration of these capabilities, facilitated by Digital Signal Processing (DSP) and Machine Learning (ML) algorithms allow for the real-time interaction, feedback, and customization of sound, along with networked interconnections between instruments and other peripherals that allow communication and remote control, which can provide a dynamic and adaptive experience for both the player and the audience.

SMIs allow musicians to bypass the limitations posed by conventional acoustic, electric or digital instruments. These limitations include the lack of embedded intelligence and the ability to connect to other devices and being limited to physical interactions only. Traditional instruments



Figure 1.1: The Sensus smart guitar - augmented with sensors, onboard processing and wireless communication [2].

depend on external equipment for enhanced functionality, where SMIs incorporate features such as effects processing or onboard computers. Moreover, traditional instruments have no integration with digital ecosystems or cloud services, which has been overcome by SMIs due to their onboard connection capabilities [3].

SMIs are characterized by several key features as shown below;

- **Embedded Intelligence:** A primary feature of SMIs is their onboard data processing capabilities. This will allow the instrument itself to analyze musical input and respond dynamically. The onboard intelligence allows for many applications within the musical instrument - RTPD being one of them.
- **Wireless Connectivity:** SMIs can communicate with external devices using communication protocols such as WiFi and Bluetooth without the need of physical cables. The wireless connectivity is enabled through embedded computing devices that facilitate their intelligence.
- **Feedback Systems:** SMIs usually include mechanisms for providing real-time feedback to players. This can take the form of visual displays or auditory cues that enhance the learning and performance experience.

The *Sensus Smart Guitar* developed by Elk Audio is an example of a SMI (Figure 1.1). This guitar is equipped with multiple sensors embedded throughout its body, ergonomically placed to not disrupt the natural interaction between the musician and the instrument. These sensors allow for multiple gestures to be tracked, i.e., finger pressure and positions in various areas of the instrument (e.g., the neck), the distance of the hand from a specific part of the soundboard, and the position of the instrument and its linear acceleration along the three axes [2]. A demonstration of the smart guitar is available at <https://youtu.be/fqzEQnsSIoY>.

1.1.2 The Internet of Musical Things

The concept of SMIs falls under the umbrella of the Internet of Musical Things (IoMusT), which refers to a network of interconnected intelligent musical devices and systems capable of communicating and interacting with each other, and also with human users [4]. The IoMusT represents an ecosystem of interconnected **musical things**, which include SMI, wirelessly connected controllers and stage equipment, and wearable devices for performers and audience members. This ecosystem allows for multi-directional real-time exchange of musical data, creating new possibilities for collective musical experiences and networked performances [5].

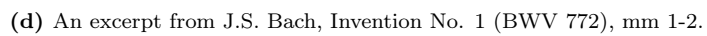
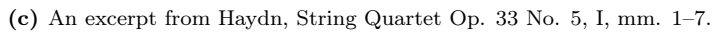
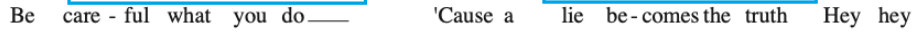
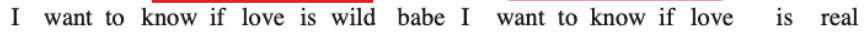
The IoMusT is a subfield of the Internet of Things (IoT) focused on music-specific applications. The IoT is a broader paradigm involving the connection of physical devices across a network to enable communication between devices without human intervention. While the IoT is primarily concentrated in areas such as home automation and industry, the IoMusT extends its principles to the realm of music by incorporating musical instruments, sound devices, and musical stakeholders into an interconnected system.

1.1.3 Real-Time Pattern Detection

A musical pattern is a group of notes and pauses that repeat within a musical composition. The repetitions of the pattern may be identical duplicates, be transposed or time shifted, or they may contain embellishments. Repetition can occur as pitched or unpitched sounds (i.e., a phrase played by a guitar player is pitched, and a phrase played by a percussion player will be unpitched). Patterns may also be monophonic or polyphonic. Figure 1.2a and Figure 1.2c shows excerpts from a popular rock song and a classical piece, with repeating patterns highlighted. Figure 1.2b shows a repeating pattern, where the repetition has been transposed [6]. Similarly, Figure 1.2d shows the concept of repetition in polyphony using an excerpt from a classical composition.

Repetitive patterns are fundamental structures of music that underpin its creation, perception, and analysis. These patterns can occur in rhythmic, melodic, and harmonic contexts, facilitating a listener’s ability to recognize and predict musical sequences [7–9]. Research indicates that human cognition is inherently geared towards pattern recognition, which is crucial not only for music appreciation, but also for broader cognitive processes [10, 11]. The study of musical patterns involves exploring their representation, discovery, and interpretation; with implications for music theory, performance practices, and the development of intelligent musical systems [12].

The field of Music Information Retrieval (MIR) has extensively explored the detection of musical patterns, leading to various applications such as audio fingerprinting, indexing, plagiarism detection, music classification, and recommendation systems. A number of studies which produce a list, visualization, summary of repeated patterns from an input piece of music have been published in the literature [9, 13–17]. Although widely studied in an offline context, i.e., when the entire composition is available for analysis, limited attention has been given to real-time



implementations: where analysis must be done as soon as the musical pattern is produced. This case is particularly challenging as it is not possible to rely on pre-processing or post-processing techniques typically employed in offline contexts.

RTPD may play an important role in advancing IoMusT applications. This significance is especially evident in applications focused on SMIs [3]. However, research in SMIs remains limited, particularly concerning their usage in IoMusT scenarios, largely due to the scarcity of suitable real-time tools that are computationally efficient enough to function on embedded systems with restricted capabilities. These limitations include reduced computing power, memory constraints, limited I/O ports, and the necessity for minimal power consumption.

1.2 Motivation and Aims

The motivation for this doctoral research stems from the challenges faced by RTPD systems, and the research gap present in the state of the art. Primarily, existing pattern detection methods are focused on offline analysis, where the entire musical composition is available for processing. Such approaches allow for a more comprehensive data analysis, but they are not suited for real-time applications where immediate feedback is required [18, 19]. The limited availability of real-time tools efficient enough to run on resource constrained embedded systems is another challenge faced by RTPD systems. A similar challenge is the need to balance accuracy and latency. Real-time systems must deliver results instantly, which may lead to a reduced accuracy, especially in resource-constrained environments [18, 20].

The intended aim of this research is to develop methods for SMI-based RTPD in music, specifically live performances. The methods presented by this thesis must be able to operate efficiently within the limitations of embedded systems, and ensure SMIs can utilize these methods without excessive resource consumption. The methods must also be capable of handling expressive variations a musician might do in a live performance setting. Moreover, they must allow SMIs to trigger external devices wirelessly based on detected patterns, to enhance performances with synchronized multi-sensory effects. In reaching these goals, this research will address the following research questions:

1. How can RTPD systems effectively manage expressive variations a musician might do when playing a musical pattern live?
2. What techniques facilitate a balance between accuracy and computational efficiency?
3. How can RTPD be designed to work optimally within embedded hardware, thereby facilitating SMIs with RTPD capabilities?
4. How can RTPD-based SMIs be used to enable multi-sensory performances where visual and tactile stimuli can complement the auditory elements?

The importance of addressing these research questions stems from the applications envisioned in the IoMusT paradigm. RTPD-based SMIs has the potential to transform traditional performances into immersive, multi-sensory experiences by enabling musical patterns to control stage elements and other peripheral devices. This will allow musicians to explore novel creative uses for their instruments, and composers to design compositions with auditory, visual, and haptic elements. Such technologies has the potential to deliver enhanced experiences for the audience through multi-sensory components complementing the music.

One of the primary challenges of RTPD systems is temporal constraints. RTPD systems cannot rely on future data, and has to use the past and/or current input and no pre/post processing can be performed on entire compositions, as commonly done with offline detection methods [18,20]. The *real-time* element in RTPD is another constraint, as results must be delivered instantaneously, as soon as the musician ends the pattern. The requirement to run RTPD methods on embedded devices is another challenge, where real time results must be obtained in computational resource constrained environments. The remainder of this document will discuss our approaches to answer the stated research questions, while overcoming the challenges listed above.

1.3 Outcomes

1.3.1 Publications

The doctoral research presented by this thesis led to the publication of the manuscripts listed below:

- Nishal Stanislaus Silva and Luca Turchet, *Real-Time Pattern Recognition of Symbolic Monophonic Music*, in Proceedings of the 19th International Audio Mostly Conference: Explorations in Sonic Cultures, pages 308–317, Sep 2024.
- Nishal Stanislaus Silva and Luca Turchet, *Demo of a smart musical instrument-based real time pattern detection system*, in Proceedings of the 19th International Audio Mostly Conference: Explorations in Sonic Cultures, pages 552–554, Sep 2024.
- Nishal Stanislaus Silva and Luca Turchet, *A Structural Similarity Index-based Method to Detect Symbolic Monophonic Patterns in Real-time*, in Proceedings of the 25th International Conference on Digital Audio Effects, pages 161-168, Oct 2022.

1.3.2 Submitted Articles

The following manuscripts are currently under review for publication,

- Nishal Stanislaus Silva and Luca Turchet, *Real-Time Polyphonic Audio Pattern Detection on Smart Musical Instruments*, submitted to IEEE Transactions on Human Machine Systems.
- Nishal Stanislaus Silva and Luca Turchet, *Performer-Audience Multi-sensory Interactions: An IoMusT Performance Ecosystem Based on Real-Time Musical Pattern Recognition*, submitted to the International Journal of Human-Computer Studies.

1.3.3 Software

The release of the developed algorithms under an open-source license was part of the doctoral work. The list below presents the source code of the developed algorithms:

- *Hot Licks*: The RTPD algorithms presented by this work, including the ML models and the synthetic data creation model has been published as source-code.
https://github.com/CIMIL/hot_licks

1.3.4 Datasets

The following datasets recorded during this research has been published:

- Dataset of Monophonic Patterns (DoMP) is a dataset containing 4000 symbolic recordings of musical patterns and repetitions containing expressive variations. The dataset was recorded with the contributions from 20 guitar players and 20 keyboard players: <https://doi.org/10.5281/zenodo.10818616>
- Dataset of Drum Patterns (DoDP) is a dataset containing 1000 symbolic recordings of musical patterns and repetitions containing expressive variation. The dataset was recorded with contributions from 10 drummers: <https://doi.org/10.5281/zenodo.14497974>
- Dataset of Polyphonic Patterns (DoPP) is a dataset containing 2000 symbolic recordings of musical patterns and repetitions containing expressive variations. The dataset was recorded with the contributions from 10 guitar players and 10 keyboard players: <https://doi.org/10.5281/zenodo.14497998>

1.3.5 Demos

This research was presented in the form of a prototype smart guitar at the following academic and industrial conferences:

- The 19th International Audio Mostly Conference: Explorations in Sonic Cultures, Milan, Italy, September 17-20, 2024
- The 24th International Society for Music Information Retrieval Conference, Milan, Italy, November 5-9, 2023.

1.3. Outcomes

- SoundMIT International Synth Expo, Turin, Italy, November 11-12, 2023.
- The 4th International Symposium on the Internet of Sounds, Pisa, Italy, 26-27 October 2023

1.3.6 Awards

The pattern detection algorithm *Hot Licks*, and the prototype smart musical instrument was recognized as a finalist in the prestigious MIDI innovation awards 2023.¹². The recorded livestream of the award ceremony is available at <https://www.youtube.com/watch?v=bL698UF-1-U>.



Figure 1.3: MIDI innovation awards 2023 - Finalist.

1.3.7 Videos

- A demonstration of the multi-sensory performance presented in Chapter 8 can be viewed at <https://youtu.be/axEHkdnxFB8>
- A demonstration of the multi-sensory performance presented in Chapter 7 can be viewed at <https://youtu.be/jfm5L3DYIcc>
- A demonstration of the system, and a discussion on RTPD is available at <https://youtu.be/gk0qf0T8zXM>

¹<https://midi.org/announcing-the-finalists-in-the-2023-midi-innovation-awards>

²<https://hotlicksvst.github.io/>

1.4 Thesis structure

This thesis is structured as follows: In Chapter 2, we discuss the state of the art in real-time pattern detection. Chapter 3 introduces the novel datasets created for this research. The methods evaluated for real-time pattern detection are presented in Chapter 4. The packaging of the algorithm as a VST plugin, and its integration into a musical instrument is discussed in chapter Chapter 5. Chapter 6 states the user study conducted to identify the potential usage and the experience of professional musicians. Chapter 7 and Chapter 8 discusses the SMIs-based multi-sensory performances conducted. Chapter 9 provides concluding remarks and proposes future research directions.

Chapter 2

Game of Tones: A Song of Patterns and Detection

There has been a multitude of research published on the detection of repeated patterns in music. Most of these studies are aimed towards the offline detection of repeated patterns in recorded music and entire compositions where the detection system operates with prior knowledge of the composition in its entirety. However, we are aiming towards a system to be used in a live environment with no prior knowledge of the incoming notes beforehand. Hence, a direct comparison cannot be computed between the proposed methods and the state of the art.

In recent years, the MIREX¹ task “Discovery of repeated themes and sections” has served as a platform for researchers to submit their work on pattern detection. Manifold methods have been proposed throughout the years, which primarily cater to the discovery of previously unknown patterns and establishing a hierarchy of their significance. This chapter serves as an introduction on the state of the art methods in musical pattern detection. We have grouped these methods into pattern detection methods for symbolic music, polyphonic and offline classifications, and machine learning methods. Moreover, we introduce the concept of multisensory performances, of which a RTPD based application will be presented later in this thesis.

2.1 Symbolic Representation-Based Methods

The authors of [21] present a geometric approach to detect patterns in both monophonic and polyphonic music. The symbolic representations of music are used, and the method inspects all displacements between note pairs, i.e., if there is a pattern occurring twice in the piece, A and B—the distances from all notes in A to their counterparts in B should be the same. The

¹<https://www.music-ir.org/mirex>

chroma vector is used to cluster the repeated patterns. To tackle transposed patterns, the authors introduce a transposition of all chroma vectors to a single chroma vector.

The algorithm presented in [22] employs a self-similarity matrix to identify repetitive musical patterns. This method is able to work with audio or symbolic representations. A chromagram is obtained for the windowed signal, and the key-transposition invariant self-similarity matrix is computed. A scoring and threshold are performed followed by a grouping by occurrence.

The authors of [23] propose a generalized skipgram model which allows arbitrary cost functions, filtering, recursive and memory-efficient applications. Skipgrams may be used for the discovery of repeated patterns of close, non-simultaneous events or notes due to the generalizations and optimizations made. The algorithm has been evaluated on symbolic, polyphonic music from a dataset containing several piano sonatas of Wolfgang Amadeus Mozart.

The system proposed in [24] introduces a compositional hierarchical model to provide a hierarchical representation of the audio signal. The input to the system is a symbolic representation of the music signal consisting of a set of pitches, each defined by an onset and an offset. The model first produces compositions of two pitches, with the number of pitches incrementing. Following this, a statistical approach is employed to retain the compositions which cover the most information in the input layer.

The authors of [25] present a point-set comparison algorithm where the music is presented in the form of a multidimensional point set. The algorithm uses two-dimensional points, (t, p) , where each point represents a single note or sequence of tied notes whose onset time is t in tatums and whose morphetic pitch is p . The authors also introduce the maximal translatable patterns vector, which is the set of points in the dataset that can be translated by a vector to give other points. This greedy compression algorithm is able to find the best maximal translatable patterns, append them to a new vector, and remove these points from the dataset.

The algorithm described in [26] finds repetitions of sequential patterns from monophonic sequences represented in a symbolic format. The algorithm progressively analyzes the musical sequence through one single pass. The analysis is carried out for each successive note of the sequence. For each new note, it is checked 1) if pattern occurrence(s) ending at the previous note can be extended with the new note; 2) if the new note initiates the start of a new pattern occurrence.

2.2 Polyphonic Audio Classification and Offline Applications

Existing research on polyphonic audio classification has explored various signal processing methods, as well as machine learning techniques, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs). These techniques have been used to handle the complexity of multiple simultaneous notes involved with polyphonic music. Most studies are focused on

offline applications (i.e., finding all occurrences of given musical patterns in a musical score or recording), but real-time processing remains a challenge scarcely investigated.

A comprehensive review and analysis of pattern recognition techniques applied to polyphonic music transcription is provided in [27]. The authors discussed various approaches, including signal processing methods, machine learning algorithms, and hybrid systems. Their work provided a foundation for subsequent research in automatic music transcription.

An extensive overview of the field of pattern discovery in music is reported in [28]. The authors reviewed existing techniques, discussed challenges, and proposed future research directions.

The authors of [26] introduced a method for extracting motivic patterns in polyphonic music based on adaptive redundancy filtering. Their approach could identify significant musical motifs in complex polyphonic textures, providing valuable tools for music analysis and composition.

The authors of [29] discussed various evaluation metrics for polyphonic sound event detection systems. They analyzed the strengths and weaknesses of different metrics, proposing new measures that better capture the performance of detection algorithms in realistic environments. Their work contributed to standardizing evaluation practices in the field.

The authors of [30] addressed the challenge of discovering repeated note content directly from polyphonic music audio. They developed algorithms that could identify repetitive patterns without relying on symbolic representations, significantly advancing the field of audio-based music analysis.

A method for detecting repetitions in complex polyphonic textures, across multiple musical dimensions including pitch, rhythm, and timbre, is reported in [31]. Such a work contributed to the understanding of musical structure in polyphonic compositions.

The authors of [32] introduced novel algorithms for matching patterns in symbolically encoded polyphonic music, allowing for pattern matching under transposition invariance. Their approach demonstrated improved accuracy and efficiency compared to previous methods, particularly in handling complex polyphonic structures.

The study described in [33] focused on identifying repetitions under transposition and time-warped invariances in polyphonic symbolic music. The researchers developed algorithms that could detect patterns even when they were distorted through transposition or temporal variations. This work significantly advanced the field of music pattern recognition by addressing the complexities of real-world musical variations.

The 2010 study on “A Methodology for Detection of Melody in Polyphonic Musical Signals” introduced a novel approach to melody extraction based on the human auditory system model [34]. The researchers developed algorithms that could identify the main melodic line within complex polyphonic textures. Their method showed improved performance compared to existing techniques, particularly in handling diverse musical genres.

The perception of syncopation in polyphonic musical contexts was explored in [35]. The authors conducted experiments to investigate how different instrumental combinations and textural complexities affect listeners’ perception of syncopation. Their findings provided insights into the

2.2. Polyphonic Audio Classification and Offline Applications

cognitive processes involved in rhythm perception in polyphonic music.

The authors of [36] conducted an empirical evaluation of end-to-end polyphonic optical music recognition (OMR) systems. They compared various neural architectures for transcribing polyphonic music scores into symbolic representations. Their findings highlighted the strengths and limitations of different deep learning approaches in OMR, providing valuable insights for future research in this area.

The work reported in [37] explores the different features humans would use to categorize songs into groups. The authors have identified that the contour, rhythm, and motifs of a melody are important in establishing a similarity between melodies. In particular, the re-occurrence of short characteristic motifs is an important factor in establishing the similarity of songs that belong to the same tune family. The authors present an annotated dataset of 260 folk songs categorized into 26 tune families. In computational approaches to the study of variation among folk song melodies from oral culture, both global and local features of melodies have been used. From a computational point of view, the representation of a melody as a vector of global feature values, each summarizing an aspect of the entire melody, is attractive. However, from an annotation study on perceived melodic similarity and human categorization in music presented by the same work [37], it followed that local features of melodies are most important to classify and recognize melodies.

The study presented by [38] compares between the global and local features in the classification of folk songs. The study, through an annotation study, investigates the importance of global features which summarize a melody as a vector from a computational point of view, and the importance of local features in the perception of similarity and human categorization. The categorization uses a nearest neighbor classification as well as a Euclidean distance. The study concludes that local features carry more melodic information than global features, and hence locality is a crucial factor in establishing similarity among folk song melodies.

A method to detect repeating patterns in audio versions of Indian music is presented in [39], which utilizes an entire structure, as opposed to a note based repetitions. Features such as mel-frequency cepstral coefficients, modulation spectral features, and jitter are calculated to reduce the computational time observed in signal level comparison. The audio is split into frames, and a dynamic time warping is used to measure the similarity between frames. The study states that songs with high complexity contain longer patterns that repeat less often, and songs with low complexity contain shorter patterns that repeat more often.

A method to automatically detect repeating patterns in polyphonic music in the JKUPDD database, in audio or symbolic representations is presented by [40], using a variable Markov oracle. The study uses chroma features which are processed based on musical heuristics such as modulation, beat-aggregation, etc, and fed into the variable Markov oracle based framework.

The authors of [41] propose a correlation-based method to determine the similarity between songs. A statistical operation, named the correlation coefficient is introduced and evaluated on a dataset of Indian classical music-based songs. The computation is based on the occurrence of

16 fundamental frequencies between two songs, and is aimed toward the development of a music recommendation system for music therapy applications.

2.3 Machine Learning Approaches

The study reported in [42] applied deep neural networks to polyphonic sound event detection. A multi-label classification approach was developed using convolutional (CNN) and recurrent (RNN) neural networks. The method demonstrated improved performance in detecting overlapping sound events compared to traditional approaches.

An approach to detecting overlapping sound events in polyphonic audio was proposed in [43]. The authors developed a method based on non-negative matrix factorization to learn dictionaries from annotated polyphonic recordings. Their approach showed promising results in separating and identifying concurrent sound events.

In our earlier study, prior to this doctoral research [19], we presented a comparison between a deterministic method and several machine learning methods designed to detect patterns in symbolic real-time music. The deterministic method was a straightforward boundary-checking system of pitch, amplitude, and time. We presented a single neural network-based method, a multiple neural network-based method: using a dedicated neural network for each pattern, a recurrent neural network-based method, and a convolutional neural network-based method. Of all methods presented, the deterministic method had the best results, identifying 100% of the patterns with zero false detections from a synthetically created dataset. However, some significant limitations of this study were the fixed window size (i.e., each pattern repetition must be identical in length to the ground truth for successful detection) and the lack of realistic variations in the dataset (i.e., repetitions with added notes, removed notes, or pitch modulations).

The study reported in [44] presents a method based on large pre-trained audio neural networks. The neural networks are trained on the large-scale AudioSet dataset and are capable of being transferred to other tasks. The authors have transferred the pretrained audio neural networks to several pattern recognition tasks using several different datasets.

The authors of [45] present a method that models human judgment of what constitutes a significant pattern by incorporating annotations of repeated patterns, avoiding the need to design heuristics. The system works on symbolic representations of monophonic music and has been evaluated on the Meertens Tune Collection Annotated Corpus dataset². The algorithm uses a neural network trained to detect a low-dimensional embedding of the feature space, mapping passages of music close together when they are occurrences of the same ground-truth pattern, based on human annotations.

The work described in [46] presents a hypothesis that fusing the output from various musical pattern discovery algorithms will improve the pattern discovery results. The study presents a

²<https://www.liederenbank.nl/mtc/>

method to combine the output from ten state-of-the-art algorithms using the MIREX dataset and the annotated corpus of the Dutch song database, which are human-annotated, and presents a meta-analysis of the (dis)similarities among pattern discovery algorithms' output.

2.4 Real-time Challenges

Real-time audio processing techniques have been developed to minimize latency and ensure seamless integration with live performances [47, 48]. These techniques often involve optimizing algorithms for speed and efficiency, as well as leveraging specialized hardware and embedded real-time operating systems [49, 50]. Embedded real-time audio processing is critical to the development of SMIs and is crucial for applications such as live performances, interactive installations, and real-time effects processing.

While there has been significant progress in both polyphonic audio classification and SMI technologies, the integration of real-time polyphonic pattern detection within an SMI remains underexplored. This research aims to fill this gap by developing and validating a system that combines these elements.

Methods of unknown pattern discovery are usually designed to determine if repeating patterns are present in entire compositions and require prior knowledge of the composition in its entirety to operate efficiently. However, as we are implementing a system to be used in a live environment with no knowledge of the incoming notes beforehand (but with the knowledge of which patterns to detect), solutions presented in existing literature cannot be used to accomplish our desired task.

2.5 Applications of Multi-sensory Performance and Immersive Technologies

Multi-sensory performances represent a rapidly evolving area of research and artistic exploration, integrating various sensory stimuli to create immersive and engaging experiences for the audience [51–54]. These performances are not limited to traditional auditory elements, but incorporate additional senses such as vision, touch, and smell, aiming to enhance the emotional engagement of the audience. Most research in this field has been focused on enhancing the participation and immersion of the audience in a Virtual Reality (VR) setting [55], especially involving audio and visual or audio and tactile modalities [56–59].

The study presented in [60] explores the integration of haptic wearable devices into live musical performances to enhance audience engagement through tactile feedback. Such haptic devices allow audiences to experience music through touch, augmenting the traditional auditory experience.

The authors of [61] explore the interplay between different senses in Augmented Reality (AR) performances. The authors presented two systems designed to enhance music creation and playback through AR interfaces in 3D environments. The systems allow users to engage in collaborative music-making experiences that incorporate visual, auditory, and tactile modalities, creating a rich multi-sensory environment. Users are able to interact with virtual bubbles containing musical segments which can be captured and arranged on a playback timeline, and users can manipulate virtual musical instruments through hand movements and speech commands. The authors also presented a user study which showed the systems' ability to improve user interactions significantly, compared to traditional interfaces.

Three public multi-sensory performances which mapped expressive gestures and voice to real-time audiovisual outputs is presented by [62]. Through questionnaires across different audience engagement levels (performers, participants, and audience), they found that direct participation and understanding of the technology significantly enhanced audience appreciation. Their findings suggest that successful multi-sensory performances require both robust technical implementation and clear narrative elements, with active engagement leading to better reception than passive viewing.

Some researchers have explored the concept of multi-sensory interactions and connected musical instruments. The study reported in [63] describes a smart guitar prototype delivering, in real time over Wi-Fi, messages to virtual reality headsets used by audience members.

The study presented by [64] introduces a novel wearable garment designed to transfer the somatic sensations of an expert singer's vocal techniques to the audience, creating a *fourth voice* through haptic gestures that mimic the singer's upper-body movements. An evaluation was conducted through immersive opera performances with audience members wearing the garment. The findings showed that such technologies can provide experiences that surpass traditional boundaries between performer and audience.

The authors of [65] introduce an ontology designed to facilitate interoperability within IoMusT ecosystems. The authors discuss the enhancement of musical experiences through haptic stimulation's and providing new capabilities for creative participation with embedded sensor interfaces. The study discusses the potential for novel interactions within IoMusT ecosystems involving performers, composers, and audience members.

The authors of the study reported in [66] have explored the integration of an interactive screen-score system within a musical performance ecology, involving two musicians and a composer. The system presented by this study facilitates the real-time creation and manipulation of musical notation to enhance the inter-connectivity between musicians and the musical score. The paper examines the interactions between the composer, instrumentalists, and the technology through a detailed case study and highlights how the screen-score acts as a mediator in the performance process. The findings suggest that such systems can exert a strong influence on musical interactions.

Although not directly related to music performance ecosystems, the study conducted by [67]

provides a comprehensive review of the role and the impact of multi-modality in VR environments. The authors discuss how integrating sensory modalities—such as visual, auditory, haptic, olfactory, and proprioceptive inputs—enhances user experiences by creating more immersive and realistic interactions within VR. The study highlights various applications of multi-modality, and its potential to improve performance, increase presence, and facilitate more effective learning and training scenarios.

2.6 Summary

Taken together, the studies above show that existing research on musical pattern classification has explored various types of signal processing and machine learning techniques. These techniques have been used to handle the complexity of multiple simultaneous notes involved with polyphonic music. Most studies have focused on offline applications (i.e., finding all occurrences of a given musical pattern in a musical score or recording). However, real-time detection remains a challenge scarcely investigated. Moreover, the reviewed literature shows that only a handful of studies has focused on the creation of IoMusT ecosystems using SMIs to control devices delivering immersive and multi-sensory content to the audience.

This research addresses these gaps by developing systems that integrate real-time polyphonic pattern detection into live performance contexts, enabling enhanced musical interaction and immersive experiences.

Chapter 3

The Sound of Data: Crafting a Musical Patterns Dataset

A fundamental step in understanding the structural and expressive aspects of music is the detection and subsequent analysis of musical patterns [12, 68]. Although computational approaches in pattern detection have advanced significantly, existing datasets primarily focus on complete musical pieces or isolated musical elements. Moreover, the existing approaches lack attention to how musicians incorporate expressive variations while maintaining pattern identity. This limitation becomes particularly evident when studying musical patterns played by a musician during a live performance, which may include the incorporation of expressive and artistic variations on certain musical phrases that they might repeat during the performance [69].

To address these limitations, we present three complementary datasets for the analysis of repeating musical patterns which include artistic variations. The DoMP forms the foundation with 4000 recordings of musical patterns in MIDI format. DoMP features 400 distinct monophonic patterns along with 3600 variations and was constructed with the contribution of 40 musicians (20 guitar players and 20 keyboard players). Building upon this framework, the DoDP captures rhythmic patterns from 10 drummers, following the same methodological approach. Finally, the DoPP extends into the acoustic domain, comprising of high quality audio recordings of polyphonic patterns and variations. DoPP was also created through audio recordings obtained from 10 piano players and 10 guitar players, and it enables the analysis of timbral and harmonic variations alongside rhythmic and melodic elements.

Currently available datasets for MIR, while extensive, present several limitations for research on pattern repetitions resembling a live performance setting. They are not ideally suited for analyzing musical patterns with expressive variations due to their focus on complete performances as opposed to pattern repetitions and expressive nuances. The main limitation is the reduced availability of repetitions for each pattern. Large-scale collections such as DISCO-10M [70] (10 million audio-MIDI pairs) and MetaMIDI Dataset (MMD) [71] (436,631 MIDI files), despite their

extensive collections, lack explicit annotations for pattern repetitions. Pattern-focused datasets such as the BPS-Motif [72] (32 pieces with 263 patterns) and JKU-PDD [73] (5 pieces with 32 patterns) emphasize the discovery of repeated patterns in complete musical compositions, and only offer a limited number of pattern repetitions. Similarly, instrument-specific collections like MAESTRO [74] (200+ hours of piano performances) and DadaGP [75] (26,181 guitar pieces) also focus on complete performances without annotations on pattern repetitions, thus limiting their usability when studying pattern repetitions with expressive variations. In the percussion domain, IDMT-SMT-Drums [76] and ENST-Drums [77] datasets primarily address drum transcription and classification tasks, and do not focus on repetitive patterns and expressive variations. Presented above were some examples for datasets with MIDI and audio data, both in monophony and polyphony, and they include piano, guitar and drum recordings. table 3.1 provides a comprehensive comparison of these datasets, and shows their unsuitability for analyzing pattern repetitions with expressive variations. This factor was the main motivation behind the three datasets, specifically created for analyzing pattern repetition with expressive variations.

This chapter presents our three datasets, detailing their design principles, recording methodologies, and technical specifications. We discuss how these complementary collections address existing gaps in music datasets, particularly in capturing intentional variations in pattern repetitions across different musical contexts.

The following sections provide detailed documentation of the datasets’ structure, characteristics, and validation procedures, establishing them as valuable resources for researchers studying musical pattern analysis, performance variation, and cross-instrumental comparison. Our approach of involving professional musicians and focusing on explicit pattern variations provides a unique foundation for advancing our understanding of how musicians naturally incorporate expressivity in pattern repetition across different musical contexts and instrumentation.

Dataset Name	Purpose	Size	Format	Content Type	Unique Features
DISCO-10M	Advancing research in machine learning for music	Large-scale	Audio and meta-data	Music recordings and metadata	Multi-stage filtering process; precomputed CLAP embeddings
MetaMIDI Dataset	Music information retrieval	436,631 MIDI files	MIDI	MIDI files	Matched against 32M Spotify clips; linked with MusicBrainz
BPS-Motif	Repeated pattern discovery	263 motives, 4944 occurrences	MIDI, symbolic	Note-level annotations	Peer review process; focuses on contextual relationships
JKU-PDD	Intra-opus pattern discovery	5 pieces	Symbolic	Musical sequences with annotations	Hierarchical pattern annotations
MAESTRO	Piano performance learning	200 hours	MIDI and audio	Audio recordings and MIDI files	Fine alignment (3 ms); includes key velocities and pedal data
DadaGP	Generative modeling	26,181 song scores	GuitarPro, tokenized	Song scores	Custom tokenization format; GuitarPro encoder/decoder
IDMT-SMT-Drums	Drum transcription and separation	608 WAV files (2:10 hours)	WAV	Drum recordings	104 polyphonic recordings; manual annotations
ENST-Drums	Drum transcription and processing	225 minutes	Audio	Audio recordings	Three professional drummers; 8 audio and 2 video channels

Table 3.1: Comparison of Music Datasets.

3.1 Composing the Monophonic Dataset: The Guitar and Keyboard Sessions

One of the first fundamental steps in our research was the creation of DoMP - focusing on monophonic musical patterns. Monophony in music implies single line melodic patterns, where only a single musical note is present at any given instant. In the context of analysis, monophonic music can be described as much *simpler* than polyphonic music where multiple simultaneous musical notes may occur. Figure 3.1 shows a graphical illustration of a monophonic and polyphonic musical section. The monophonic and symbolic DoMP serve as the primary resource in understanding fundamental aspects of expressive variations between musical patterns before extending this research into more complex musical contexts.

The choice of monophonic music as the initial focus offers several methodological advantages. Monophonic music, characterized by a single melodic line without harmonic accompaniment, provides a controlled environment for studying any artistic and expressive variations a musician might perform to a pattern. This simplification allows for unobstructed analysis of fundamental musical parameters such as pitch, timing, and dynamics without the complexity introduced by harmonic interactions and multiple concurrent voices. However most instrument soloing tend to monophonic for the greater part, and monophonic pattern detection too has a very usable use case in a real-world scenario.

3.1.1 MIDI Magic: Why Go Symbolic?

MIDI¹ is the protocol that allows communication between electronic musical instruments, computers, and other connected devices [78]. MIDI operates by transmitting digital messages in place of audio signals. As opposed to audio formats like mp3 or WAV that reproduce exact sounds, MIDI transmits instructions that are used by devices to produce sound using their internal sound generation mechanisms [79]. The MIDI protocol operates at 31.25 kbps using 8-bit serial transmission, with a single start bit and a single stop bit.

The MIDI communication relies on several message types as described in Table 3.2. The primary message types in music generation are *note on* and *note off*. Each message encapsulates the starting and ending time of the note, as well as how loud the note must be. Consider the first four notes of a C major scale as shown by Figure 3.2 and Figure 3.3 in musical and piano roll notations respectively. The MIDI sequence messages required to represent these musical notes are shown by Figure 3.4.

The decision to use MIDI as the primary symbolic data format during the first stage of this research was taken as it can provide a musically meaningful representation which captures the performance details without the complexity of audio signals. Moreover, MIDI provides direct

¹<https://midi.org/>



Figure 3.1: An excerpt from the popular song “Bohemian Rhapsody”. The single line monophonic vocal section from the first two bars of the song (top) and the accompanying polyphonic choral section (bottom) illustrates the difference between monophony and polyphony.

Message Type	Function	Parameters
Note On/Off	Controls note playback	Note number, Velocity
Control Change	Modifies parameters	Parameter ID, Value
Program Change	Switches instruments/patches	Program number
System Messages	Controls system functions	Clock, Transport

Table 3.2: The primary message types used by the MIDI protocol.

access to note events and properties and a clear representation of all parameters of each note (i.e., velocity, duration, modulations etc.).

3.1.2 Dataset Recording

The recording process for DoMP was contented with the contribution of 40 musicians (20 guitar players, and 20 keyboard players). The guitar and the keyboard were selected as they are arguably the two most popular musical instruments with onboard, or easily attachable mechanisms to obtain MIDI data.. The musicians were from different ethnic groups (Italian, British, Scottish, Filipino, Spanish, Ugandan, and Sri Lankan), male, and aged between 18-43 (mean age: 28.5, standard deviation: 6.244).

Moreover, the musicians were from diverse musical backgrounds, some of them including classically trained musicians, church organists, contemporary studio/ session musicians, jazz musicians, rock and pop musicians, solo musicians, musicians involved in bands and groups, blues musicians, and ethnic musicians. This diversity in experience and backgrounds of the musicians involved in the dataset contributed a high variance in data suitable for a generalization.

In recording the dataset, most keyboard players preferred to use their own instrument. Some keyboard players used workstations or digital pianos, while some keyboard players used MIDI controllers. The MIDI signal was obtained directly form the instrument for all keyboard players.

However, recording process for the eclectic guitar segment was somewhat restricted due to

3.1. Composing the Monophonic Dataset

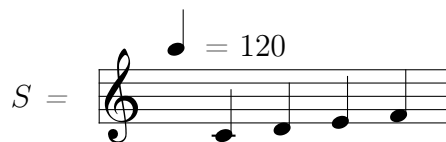


Figure 3.2: The first four notes of the C major scale in standard music notation.

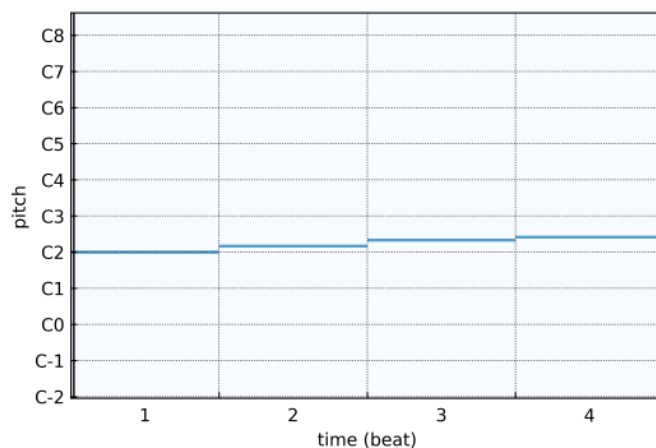


Figure 3.3: The first four notes of the C major scale in piano roll notation.

```
1  note_on channel=0 note=48 velocity=72 time=0
2  note_off channel=0 note=48 velocity=64 time=480
3  note_on channel=0 note=50 velocity=72 time=0
4  note_off channel=0 note=50 velocity=64 time=480
5  note_on channel=0 note=52 velocity=72 time=0
6  note_off channel=0 note=52 velocity=64 time=480
7  note_on channel=0 note=53 velocity=72 time=0
8  note_off channel=0 note=53 velocity=64 time=480
```

Figure 3.4: The MIDI messages required to generate the first four notes of the C major scale.



Figure 3.5: Fishman TriplePlay Connect: the MIDI tracker used to record the dataset. Image from product webpage².

the need of a MIDI tracker for the electric guitar. The MIDI tracker converts the notes played on the guitar into symbolic representation, thereby enabling them to be used with our algorithm. Figure 3.5 shows an electric guitar that has been outfitted with the Fishman TriplePlay² MIDI tracker which was chosen due to its ease of operation. The guitarists used an Ibanez S470 (6 strings, 25.5" scale length, HSH pickup configuration, double locking tremolo), and an Epiphone SG standard (6 strings, 24" scale length, HH pickup configuration, tune-o-matic bridge) guitars to record the dataset.

For the dataset recording, each musician was asked to compose and record 10 distinct monophonic patterns that could potentially be repeated in a live performance or improvisation. The patterns reflected each musician's personal playing style and background. The 10 patterns acted as the ground truth data.

The musician was then asked to repeat each pattern 9 times, and the musician was allowed to use any artistic and expressive variations in the subsequent recordings up to a limit. The only condition presented to the musician was that each *version* of the pattern should be perceptually equivalent to the original ground truth, according to their opinion. This decision was taken to understand the perceptual variance between a musical pattern and a repetition with expressive variation, as the level of expressive variations varied with each musician. Hence, each musician recorded 10 distinct patterns with 9 variations of each to yield a total of 400 musical patterns, which resulted in creation of DoMP with 4000 monophonic musical patterns and variations.

The keyboard players used different expressive techniques in their playing. It was evident that some musicians prefer certain techniques over others. Most keyboard players involved in live bands used the pitch-bend controller, while its presence was very rare among piano players and church organists. Some musicians prefer hard keystrokes, while some prefer to play very soft.

²<https://www.fishman.com/tripleplay/>

3.1. Composing the Monophonic Dataset

Adding passing chromatic notes in sequences, using the pitch-bend to reach subsequent notes, and slight pitch modulations on notes are some examples of the expressive techniques exhibited by the keyboard players in recording the dataset.

Similarly to the keyboard players, the guitar players also used various expressive techniques that are present in the dataset. The most common variation is deviations in time. String bending was also a very common technique used by many guitar players: either to reach the next note or to add slight vibrato effects. Hammer-ons and pull-offs were also some common techniques used.

However, several restrictions had to be applied to overcome the limitations of the MIDI tracker. The following list describes some of these limitations:

- Very fast playing was tracked as wrong notes or a single note with very large pitch bends.
- Fast legato sequences were sometimes tracked as a single note with large pitch bends.
- Multi-finger tapping was often tracked as single notes with large and frequent pitch modulations
- Palm muted notes and ghost notes were often tracked as random notes.
- Harmonics (artificial/ natural/ pinch) were tracked as false notes.

Moreover, the MIDI tracker operates on the MIDI 1.0 standard³, and it lacks polyphonic pitch bending, which is a technique most guitar players use frequently. This was not a concern as the dataset is monophonic. The guitarists were asked to refrain from using techniques listed above to overcome the hardware limitations. Despite the restrictions imposed, the dataset contains a large number of artistic and expressive variations.

The dataset contains 4000 patterns in the form of MIDI files. Of these patterns, 400 are distinct ground-truths and 3600 are their repetitions. The folder structure of the dataset is shown by Figure 3.6. Figure 3.6 shows the file tree of the dataset. The filenames are named in the format `artistID_patternID_versionID.mid`, where $\mathbb{Z}$ represents a set of integers and;

- `artistID` $\in \mathbb{Z} : 0 \leq \text{artistID} \leq 39$;
- `artistID` $\in \mathbb{Z} : 0 \leq \text{artistID} \leq 9$;
- `versionID` $\in \mathbb{Z} : 0 \leq \text{versionID} \leq 9$.

The expressive variations performed on the patterns in the dataset range from very subtle to noticeably obvious, such that perception of the original pattern is retained. However, these levels of variation may be deemed too high for a live performance, where performers are likely to bring about minimal variations to a predefined pattern. As a result, we can consider our dataset as a *worst case scenario* as it represents the most challenging situations that a pattern detection algorithm may encounter. These characteristics, the inclusion of different musical instruments, along with the involvement of diverse musicians in recording the dataset make it an ideal candidate for evaluating our algorithms, as well as a benchmark for future studies.

³<https://midi.org/midi-1-0>

```

└─ dataset/
  └─ guitar/
    └─ 0/
      └─ 0_0/
        └─ 0_0_0.mid.
        └─ ....
        └─ 0_0_9.mid.
      └─ 0_1/
        └─ 0_1_0.mid.
        └─ ....
        └─ 0_1_9.mid.
      └─ ....
      └─ 0_9/
        └─ 0_9_0.mid.
        └─ ....
        └─ 0_9_9.mid.
    └─ 1/
    └─ ....
    └─ 9/
      └─ 19_0/
        └─ 19_0_0.mid.
        └─ ....
        └─ 19_0_9.mid.
      └─ 19_1/
        └─ 19_1_0.mid.
        └─ ....
        └─ 19_1_9.mid.
      └─ ....
      └─ 19_9/
        └─ 19_9_0.mid.
        └─ ....
        └─ 19_9_9.mid.
  └─ keyboard/
    └─ 20/
      └─ 20_0/
        └─ .5 20_0_0.mid.
        └─ .5 ....
        └─ .5 20_0_9.mid.
      └─ .4 20_1/
      └─ .4 ....
    └─ 21/
    └─ ....

```

Figure 3.6: The file tree structure of the DoMP.

3.2 Uncovering Musical DNA: Analyzing Expressive Variations

The primary purpose of creating the dataset was to define an objective and suitable definition for a musical pattern. In the literature, the commonly used definition of a musical pattern is a set of ontime pitch pairs that are repeated at least once within a composition [80, 81]. However, most research in MIR struggle to reach a general consensus on what is a musical pattern [82]. Some definitions present patterns as a function of multiple attributes (duration, contour, harmony, etc.) [26]. Some researchers have presented drastically different definitions where patterns do not need to be repeated but must have musical *meaning* to create musical statements and to construct a musical syntax [83].

The research presented by [82] suggests that patterns can be defined as variations on an initial musical element, where variations consist of repetitions with modifications to one or more of its attributes (e.g., pitch, duration). A recognition system for music must incorporate perceptual features for successful operation [84].

The traditional definition of a musical pattern has evolved beyond simple repetition to incorporate various musical attributes [85]. This evolution reflects the understanding that patterns are subject to artistic interpretation as opposed to static structures. Hence, the work presented by this thesis, influenced by the above definitions as well as those given by various musicians, considers a more perceptual definition of a musical pattern: *every ordered sequence of notes and pauses that occur at least twice in a composition, either identically or with variations, is considered as a pattern. Two sequence occurrences that present artistic variations are considered to be repetitions of the same pattern when untrained human listeners can perceptually agree on their common origin/ equivalence/ similarity.*

This research supports a perception-centered definition in alignment with cognitive music processing, where the human brain structures the environment through discriminating between similar, and different elements [11]. With this hypothesis, the similarity between musical patterns can be defined as a function dependent on temporal, pitch, and dynamic similarity as shown by Equation (3.1), where a musical pattern P is defined as a function of its temporal, pitch, and dynamic characteristics. This relationship is a crucial element in evaluating the performance of the pattern detection methods discussed in the proceedings sections.

$$P_{(pattern)} = f(temporal, pitch, dynamic) \quad (3.1)$$

A comprehensive analysis of DoMP reveals several key characteristics in musical patterns and repetitions. Some of these are shown by Table 3.3. The average length of a pattern is 20.45 MIDI notes, the average variation in amplitude is 9.24 MIDI velocity, and the average deviation in time is 77.64 MIDI tick times.

Several characteristics of elements being added to, or being removed from a pattern are listed below. Thus, the rate of modification can be defined as 28.117% using the formula presented by

Parameter	Mean	Standard Deviation
Pattern Length	20.45	5.32
Amplitude Variation	9.24	2.87
Temporal Deviation	77.64	12.45

Table 3.3: Average length, amplitude variation, and temporal deviation of the patterns in DoMP.

Equation (3.2).

- The average number of notes added to a musical pattern is 4.84;
- The average number of notes removed from a musical pattern is 0.69;
- The average number of extra pauses added to a pattern is 1.93;
- The average number of pauses removed from a pattern is 0.33;

$$\text{Modification Rate} = \frac{\text{Elements Added} - \text{Elements Removed}}{\text{Total Elements}} \times 100\% \quad (3.2)$$

A notable finding from the dataset was pitch modifications. Analysis of DoMP showed that approximately 9% of the notes in a pattern contain pitch bends. Moreover, an average number of 3.65 pitch bends were added as extra to a pattern. The dataset also showed that 956 of the pattern variations had added variations when the ground-truth did not. These findings revealed that there is a 26.5% probability that a pitch bend may be included in a pattern. The dataset also revealed that chromatic passing notes was a common embellishment technique. An average number of 1.58 notes per pattern may be inserted between two notes separated by a whole tone.

This comprehensive analysis of DoMP provides extensive insight into how musicians would employ expressive variations when repeating a musical pattern during a live performance. These insights can be summed down into three critical considerations for the detection of pattern detection algorithms:

1. Algorithms should be more permissive of additions than deletions, as the data suggests a higher probability of notes and pauses being added as opposed to removed;
2. Algorithms must be flexible temporally, and allow both note timing variations and strategic insertion of pauses. A somewhat stricter criteria may be maintained for the removal of pauses;
3. Algorithms must incorporate pitch bends, particularly in transition regions between notes.

3.3 Rhythm in Repetition: Adding Drum Patterns

Building upon the foundation of the DoMP, this research expanded into the rhythmic and percussive patterns by introducing the Dataset of Drum Patterns (DoDP). While DoMP focused primarily on expressive variation dependent on melody, the DoDP aims to capture the rhythmic expressions and variations commonly employed by drummers during live performances. Similar to the DoMP, DoDP also contains 10 distinct drum patterns by each musician, with 9 accompanying variations of each pattern. A total of 10 drummers were involved in creation of the dataset. Similar to DoMP, the drummers had different musical backgrounds and experience levels.

The complete dataset was recorded using a Yamaha DTX-432k electronic drum kit⁴. The choice of an electronic drum kit was due to the ease of obtaining MIDI data. Its silent operation was also a crucial factor when conducting experiments in laboratory conditions, as well as its small footprint and ease of transportation. Figure 3.7 shows the electronic drum kit used to record DoDP. The drum kit configurations included:

- Yamaha DTX drum module
- Bass drum
- Snare drum
- 3 × toms
- Cymbals (hi-hat, crash, and ride)
- Hi-hat control pedal

A total of 10 drummers participated in the recording sessions to create DoDP. All musicians were Italian, aged between 18-36 (mean age: 26.0, standard deviation: 5.07), they had different experience levels, and represented diverse musical backgrounds including:

- Professional session drummers
- Jazz drummers
- Rock and metal band members
- Orchestral percussionists
- Traditional/ethnic percussion players
- Contemporary and fusion musicians

The recording procedure was identical to the DoMP, where each drummer was asked to compose 10 distinct drum patterns that they would typically use during a live performance. The patterns varied in complexity, time signatures, and style. Upon recording the ground-truths, the musician was asked to play each pattern 9 more times, employing various expressive techniques that they preferred. Subsequently, the DoDP resulted in:

⁴<https://shop.usa.yamaha.com/en/p/instruments/drums-percussion/electronic-drums/dtx432k-electronic-drum-set>



Figure 3.7: The Yamaha DTX-432k electronic drum kit used to record DoDP.

- 100 distinct ground truth patterns
- 900 repetitions with expressive variations
- 1000 total recorded patterns

Similar to DoMP, the variations remain perceptually equivalent to the original ground truths, while incorporating common drumming techniques and expressions. The dataset contains a comprehensive range of drumming techniques and variations, including:

- **Dynamic Variations:**
 - Ghost notes and accents
 - Crescendos and diminuendos within a pattern
 - Sudden velocity changes
 - Cross-stick and rim-shot variations
 - Switching between left and right hand
- **Temporal variations:**
 - micro-timing adjustments
 - puch-pull techniques
 - Subdivision variations
 - Tempo changes
 - single, double, and half time techniques

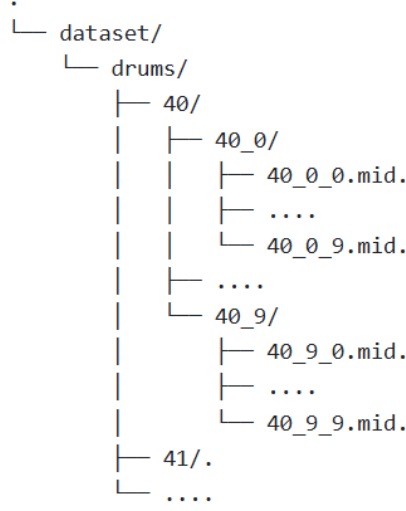


Figure 3.8: The file tree structure of the DoDP.

- **Embellishments:**
 - Variations in drum fills
 - Grace notes
 - Interchanging toms within a pattern
 - Addition or removal of notes
 - Inclusion of drum rolls
- **Stylistic variations:**
 - Genre-specific groove changes
 - Ethnic and cultural rhythm adaptations
 - Fusion of different styles

The dataset follows an identical organization to DoMP. Figure 3.8 shows file structure of the dataset. The files are named in the format `artistID_patternID_versionID.mid`, where $\mathbb{Z}$ represents a set of integers and;

- $\text{artistID} \in \mathbb{Z} : 40 \leq \text{artistID} \leq 59$;
- $\text{artistID} \in \mathbb{Z} : 0 \leq \text{artistID} \leq 9$;
- $\text{versionID} \in \mathbb{Z} : 0 \leq \text{versionID} \leq 9$.

The drum kit was set up to follow the General MIDI percussion mapping, which is one of the standard methods for symbolic percussion. The General MIDI percussion mappings are presented in Appendix Table A.2. In contrast to the patterns present in DoMP, the drum patterns need several modulation and controller data for a successful sound reproduction. To fulfil this requirement, each MIDI file contains,

- MIDI note numbers of each hit
- Velocity data for each hit. MIDI velocity is in the range 0-127.
- Precise timing information in MIDI ticks
- Controller data for hi-hat pedal positions

3.3.1 Groove Analytics: Decoding Patterns and Crunching the Stats

A comprehensive analysis into the percussive patterns and expressive variations present in DoDP revealed similarities, as well as differences between DoMP and DoDP. Below are several key characteristics of the patterns present in DoDP:

Pattern Statistics

- Average pattern length: 16.8 MIDI events (standard deviation: 4.2)
- Mean velocity variation: 12.6 MIDI velocity (standard deviation: 3.8)
- Average deviation in time: 42.3 MIDI ticks (standard deviation: 8.9)
- Average tempo: 98.6 BPM (standard deviation: 18.4)

Expressive Variations

- Average addition of drum hits: 3.2 MIDI events per pattern
- Average removal of drum hits: 0.8 MIDI events per pattern
- Average addition of ghost notes: 2.4 MIDI events per pattern

Time Signature Distribution

- 4/4: 75% of patterns
- 6/8: 15% of patterns
- 7/8 and odd meters: 2% of patterns
- Other meters: 8% of patterns

The DoDP exhibits several distinct characteristics when compared with DoMP. The average deviations in time is smaller (42.3 vs. 77.62 in MIDI ticks), and the pattern lengths are more consistent. Moreover, a lower rate of both addition and removal of elements was observed. These findings highlight several attributes that must be considered when designing a pattern detection system for a percussive context as shown below:

1. Algorithms must be more sensitive to timing variations to accommodate for micro, double, and half time variations.

2. Systems must be able to handle a greater range of velocity variations, such as the usage of ghost notes and accents.
3. Algorithms should ideally be able to detect patterns within larger rhythmic sections.
4. Systems should be flexible to handle different grooves, along with changes in groove.

3.4 The Sound of Many Voices: Creating the Polyphonic Dataset

The research presented by this thesis started from the exploration of symbolic monophonic musical patterns, then led to the analysis of rhythmic and percussive patterns to end at the study of expressive variation within polyphony. Upon establishing robust datasets for monophonic patterns and rhythmic drum patterns, we present the dataset designed to capture the nuances in polyphonic musical patterns and their repetitions with expressive variations. This progression reflects the increasing complexity of analysis, where multiple voices interact with each other to create harmonic phrases. This advancement in data collection provides insight into how a musician would handle pattern variations in a more complex musical context, where timbral characteristics must also be analyzed in parallel with the musical characteristics.

The DoPP is consistent with the data collection framework used in DoMP and DoDP. A primary change is the adaptation into the challenges of audio recording. The recording setup included Universal Audio RME Fireface UFX+⁵ and Zoom U-24⁶ audio interfaces. All recordings were done with a 44.1 kHz sampling rate and 24-bit resolution.

A total of 20 musicians (10 guitar players and 10 keyboard players) participated in the recording sessions to maintain parity with DoMP and DoDP. The musicians were from different ethnicities (Italian and Sri Lankan), aged between 26 and 55 years (mean age = 38.571, standard deviation = 9.234), and had different musical backgrounds including classically trained musicians, contemporary studio/ session musicians, funk musicians, jazz musicians, and rock musicians.

Similarly to the methodology adopted to the construction of DoMP and DoDP, each musician was asked to compose 10 distinct polyphonic musical patterns, along with 9 more repetitions of each of them. Each musician was given the artistic freedom to add any variations or expressive techniques they preferred on each subsequent version so long as the original pattern remained perceptually equivalent: preserving the original idea of perception, where an untrained human listener should identify each variation to be the same as the original pattern.

The dataset contains a total of 2000 polyphonic audio patterns and variations, of which 1000 are played on a keyboard and 1000 are played on an electric guitar. 200 total patterns are distinct (100 guitar patterns and 100 keyboard patterns), and 1800 patterns are repetitions of the ground-truths with expressive variations.

⁵<https://rme-audio.de/fireface-ufx.html>

⁶<https://zoomcorp.com/en/us/audio-interface/audio-interfaces/u-24/>

Most musicians preferred to use their own instrument in recordings the dataset. Some of instruments used in recording the dataset were:

- **Electric Guitars:**

- Ibanez S470
- Ibanez RGA7
- Fender Stratoaster
- Gibson Les Paul
- Fender Telecaster
- Epiphone SG standard

- **Electric Keyboards:**

- Korg Krome
- Dexibell Vivo S7
- Roli Seaboard
- Studiomagic Numa X
- Korm KRONOS

Thus, the dataset contains an excellent representation of different spectral and timbral characteristics of multiple instruments. The guitarists also experimented with different pickup selections which can also be attributed as timbral variations of the pattern. The unprocessed direct input was obtained from the electric guitars, and the keyboard players were instructed to refrain from using time-based effects such as delay or reverb.

The keyboard players used different techniques such as using the pitch bend and using modulation/ sustain pedals. Adding extra notes, removing notes and changing the durations of notes were some common variations present. Similarly, the guitar players used techniques such as string bending, double stops, shifting between staccato and legato playing, muted notes, ghost notes and harmonics. The audio recording process eliminated the limitations encountered when recording guitar for DoMP, which were due to hardware restrictions of the MIDI tracker.

Thee polyphonic musical patterns exhibit several distinct features, some of them including:

- **Harmonic Elements:**

- Combination of monophony and polyphony
- Chord voicing between 2-4 notes
- Extended harmony
- Double stops
- Voice leading

- **Textural Elements:**

- Contrapuntal motion
- Rhythmic accompaniment
- Dissonance

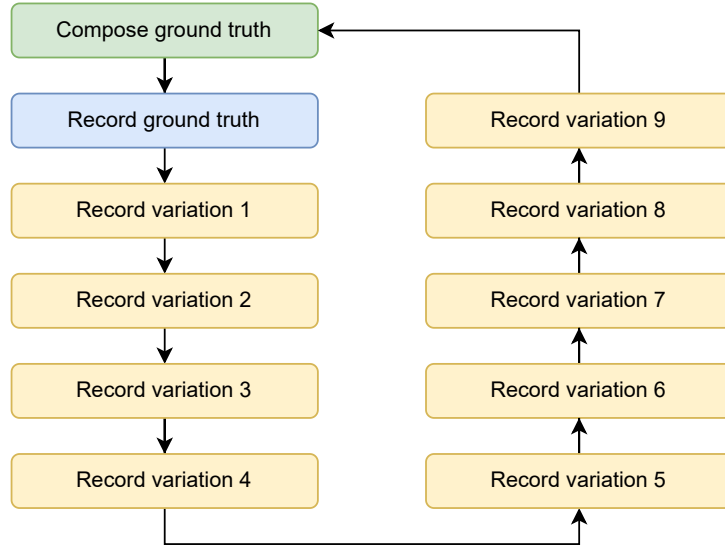


Figure 3.9: The recording pipeline for all 3 datasets.

Moreover, the dataset includes various polyphonic instrument-specific techniques. Some of the guitar-specific techniques captured are fingerstyle playing, hybrid picking, string grouping variations, polyphonic string bending, and multi finger tapping. Some of the keyboard-specific techniques present in the dataset are split hand movements, pedal variations, usage of the pitch bend and modulation controller, usage of aftertouch, and advanced harmony changes. The presence of a wide variety of expressive techniques, as well as a wide variety of tonal and timbral characteristics makes DoPP an ideal candidate to evaluate pattern detection systems.

3.4.1 Harmony in Numbers: Unlocking the Secrets of Polyphonic Patterns

A comprehensive analysis was done on the DoPP, which revealed several key characteristics of polyphonic musical patterns.

- Average pattern duration: 8.4 seconds (standard deviation: 5.54);
- Tempo range: 60-180 BPM;
- Dynamic range: -24dB to 0dB.

The file naming convention was kept identical to DoMP and DoPP. The file structure of DoPP is shown by Figure 3.8.

The file naming format is `artistID_patternID_versionID.mid`, where $\mathbb{Z}$ represents a set of integers and;

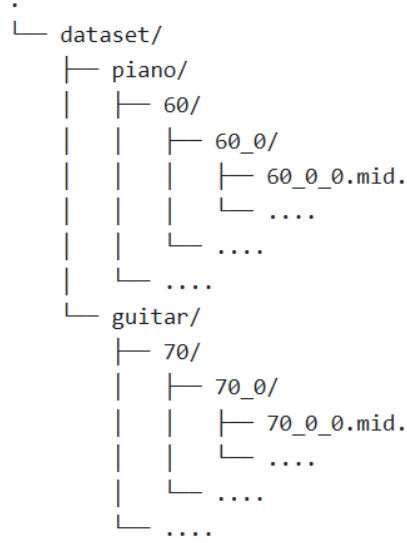


Figure 3.10: The file tree structure of the DoPP.

- $\text{artistID} \in \mathbb{Z} : 60 \leq \text{artistID} \leq 89$;
- $\text{artistID} \in \mathbb{Z} : 0 \leq \text{artistID} \leq 9$;
- $\text{versionID} \in \mathbb{Z} : 0 \leq \text{versionID} \leq 9$.

This comprehensive polyphonic dataset completes our three-tiered approach to musical pattern analysis, providing researchers with a robust foundation for studying the full spectrum of musical expression from simple melodies to complex polyphonic arrangements. The progression from MIDI-based monophonic and rhythmic patterns to audio-based polyphonic recordings reflects the natural hierarchy of musical complexity, offering invaluable insights into how musicians conceive and vary musical patterns across different contexts and complexities.

3.5 Grooving Through the Data: Patterns, Beats, and Musical Feats!

This chapter has outlined the design, creation, and significance of three complementary datasets—DoMP, DoDP, and DoPP—each uniquely addressing specific challenges in the analysis of musical patterns and their expressive variations. These datasets were developed with the aim of bridging gaps in existing resources for musical information retrieval (MIR), particularly with respect to live performance settings. They stand apart in their ability to capture nuanced artistic expressions that musicians naturally incorporate when repeating patterns across monophonic, rhythmic, and polyphonic contexts.

3.5. *Grooving Through the Data: Patterns, Beats, and Musical Feats!*

The DoMP dataset, focusing on symbolic monophonic patterns, serves as a foundation for understanding how musicians introduce expressive variations such as timing shifts, dynamic modulation, and note embellishments. The inclusion of diverse musicians with varied stylistic backgrounds ensured a rich representation of playing techniques. These insights into monophonic expressivity provide valuable baseline data for computational models seeking to understand or emulate musical performance.

Building on DoMP, the DoDP dataset expanded the scope to rhythmic and percussive patterns, emphasizing temporal flexibility and dynamic variation in drum performances. Captured from drummers across a range of genres, the dataset highlights intricate micro-timing adjustments, ghost notes, and stylistic embellishments. By doing so, it not only provides a benchmark for rhythm-specific pattern detection but also reflects the diversity of rhythmic expressions encountered in live performance scenarios.

The DoPP dataset marks a significant progression, delving into the complex interplay of harmony, melody, and timbre in polyphonic musical contexts. By incorporating audio recordings alongside symbolic representations, it offers researchers the opportunity to analyze expressive techniques in richer detail, including chordal textures, contrapuntal motion, and instrumental timbre. The dataset also overcomes limitations of symbolic-only formats, particularly for guitar performances by capturing expressive nuances like string bending and harmonic variations.

Together, these datasets form a cohesive resource that advances the study of musical pattern analysis, enabling cross-instrumental and cross-contextual comparisons. Their systematic organization, diverse participant pool, and focus on expressive variations establish a comprehensive benchmark for evaluating pattern detection algorithms. They also set a new standard for capturing musical expressivity in ways that are both perceptually meaningful and computationally accessible.

In summary, the datasets not only provide a foundation for understanding how musicians intuitively create and manipulate patterns but also contribute to the broader fields of MIR, cognitive musicology, and performance analysis. Future researchers can leverage these datasets to explore topics such as machine learning-based pattern recognition, expressive performance modeling, and cross-cultural musical analysis. By addressing existing limitations and incorporating diverse expressive contexts, these datasets pave the way for a deeper understanding of the “sound of data” in music.

Chapter 4

The Music Matrix: A Framework for Real-Time Pattern Recognition

The detection of musical patterns involves a complex intersection between signal processing, data analysis, and as of recently, machine learning techniques. This chapter presents a comprehensive examination of various methods evaluated within the course of this doctoral research in the context of identifying musical patterns in real-time. Several computational approaches including DSP methods, traditional machine learning methods, and modern deep learning methods were evaluated to identify a suitable candidate delivering the perfect balance between accuracy and performance. DSP methods such as the Structural Similarity Matrix (SSIM) and Dynamic Time Warping (DTW), traditional neural networks, and Recurrent Neural Networks (RNNs) were among the methods assessed.

Every computational method has its own advantages as well as drawbacks. The selection of the methods described above was driven by their strengths in the context of RTPD. Traditional neural networks demonstrate capabilities in learning hierarchical patterns between data points. RNNs on the other hand, offer advantages in processing musical patterns due to their ability to capture temporal dependencies. The SSIM offers a novel approach to pattern detection with its focus on structural relationships between musical sequences.

These detection methods were evaluated on the basis of performance, which include accuracy, computational load, as well as real-time processing capability and latency. A critical assessment is crucial, as real-time implementations of a pattern detection system is drastically different from an offline implementation due to the need of processing to be in real-time. Moreover, the need to implement the system on an embedded device with limited computational power requires a system with a low memory footprint. This section will discuss the implementation of each approach, including preprocessing steps, feature extraction techniques, and model architectures.

The experimental setup was designed to provide a fair comparison between all methods, while exploiting the strengths of each method. The datasets introduced in Chapter 3, comprising of diverse range of musical genres and styles were used in the evaluation. Several unique

data representation methods are used, including matrix-based representations, discrete time series representations, and spectrogram-based representations. Each method employs a different analysis technique, and the data representation methods were designed to obtain the fullest potential. Through this systematic evaluation, this thesis presents the advantages and drawbacks of each method in the context of musical pattern detection, the experimental results, along with the most suited candidate for a real-time usage. These findings not only advance the state of the art, but also highlight promising directions for future research.

4.1 Music in Translation: From Notes to Numbers

The raw music, either in symbolic or audio format, must first be transformed into meaningful information that can be analyzed by computational processes. This transformation of data is crucial in creating well performing algorithms for musical pattern detection.

This section illustrates the data representation mechanisms introduced to facilitate computational analysis. Each detection method that was evaluated in this research works differently, and data must be fed in a format that aligns with the specific functioning of each algorithm. A matrix-based data representation, a sampled discrete time series representation, and a spectrogram representation that was introduced are discussed hereinafter. Each format provides unique insights into musical data. from the precision of matrix-based analysis to the temporal clarity of discrete time series, or the expressive timbral characteristics presented by spectrograms, these representations serve as essential building blocks for advanced algorithms in RTPD.

4.1.1 MIDI Reloaded: Building the Matrix

A matrix-based data representation is particularly well suited for the computation of SSIM as it operates on the principle of comparing structural similarities within 2-dimensional data. The SSIM is primarily an image processing technique, and by organizing MIDI information into a matrix format resembling that of a single channel image, the inherent structures of the music can be preserved. This research uses a modified version of the matrix-based representation introduced by our pre-doctoral work reported in [19].

The original representation uses three attributes to denote each single musical note, namely the *pitch*, *amplitude*, and *duration*. This research further enhances this representation by introducing an additional attribute: *Pitch Bend* - used to denote the graceful increase or decrease of the pitch of a musical note. A column matrix is used to encapsulate the four attributes and denote a single musical note. The four attributes can be described as follows:

1. **Pitch:** The pitch of a note is a perceptual quality which allows for it to be deemed *higher* or *lower* than other notes, and enables a note to be ordered on a frequency-related

scale [86, 87]. The MIDI standard uses the 12 tone equal temperament (12-TET) tuning system and the pitch is obtained from the MIDI number.

2. **Pitch-bend:** Pitch-bend is a musical technique used to obtain vocal-like expressive characteristics and articulations from musical instruments by increasing or decreasing the pitch of a note. String instruments achieve this effect by bending the desired string, and keyboard style instruments achieve this effect by the use of a pitch controller. Pitch-bend is similar to the music concept of *glissando*.

The MIDI pitchwheel parameter is used to denote the pitch bend. The assumption that the commonly used interval of ± 2 semitones for the pitchwheel range is maintained in this research [88].

To denote a smooth and graceful pitch bend, MIDI instruments encode the pitch bends as a series of MIDI messages with fine increments or decrements. Using all pitchwheel values would render extremely long sequences with arbitrary lengths. Hence, only the local minima or maxima are used to denote the pitch bend to retain its shape with a minimal amount of data. Figure 4.1 shows an illustration of a pitch bend done to a single musical note. For this particular note, the pitch bend changes are represented by 158 MIDI messages. The representation standard uses values corresponding to the 6 peaks and 5 troughs instead of all 158 values. Thus, the note shown by Figure 4.1 will have pitch bend values of $[0, 8191, 800, 8191, 4454, 8191, 2114, 6096, 266, 7451, 61, 7492, 0]$.

3. **Amplitude:** The amplitude corresponds to the loudness of the note. As shown by Equation (3.1), the relative loudness in each note of a sequence as an important attribute in determining the match with a pattern. MIDI velocity is used to represent the amplitude.

The velocity value corresponds to the rate at which a keyboard key is pressed [88, 89]. This action on an acoustic instrument will render a louder, or a softer sound. Although MIDI velocity may be used to control multiple parameters in a synthesizer [89], the evaluations operate under the assumption that MIDI velocity is used to control only the amplitude.

In music, dynamics refer to the variation in loudness between notes and phrases, and are represented using dynamic markers, which may range from *pianississimo*, which refers to *very very quiet* playing, to *fortississimo*, which refers to *very very loud* playing [90]. Appendix A.1 shows a complete list of dynamics markers, along with their symbols and corresponding MIDI velocity values as mapped by the popular music production software Logic Pro [1].

4. **Duration:** The duration of a note is the time duration where the note undergoes its gradual decay. The duration is presented in seconds.

Equation (4.1) provides an example of a single musical note denoted using the representation mechanism introduced above. For any i^{th} note in a sequence N_i , the pitch is denoted by P_i , the

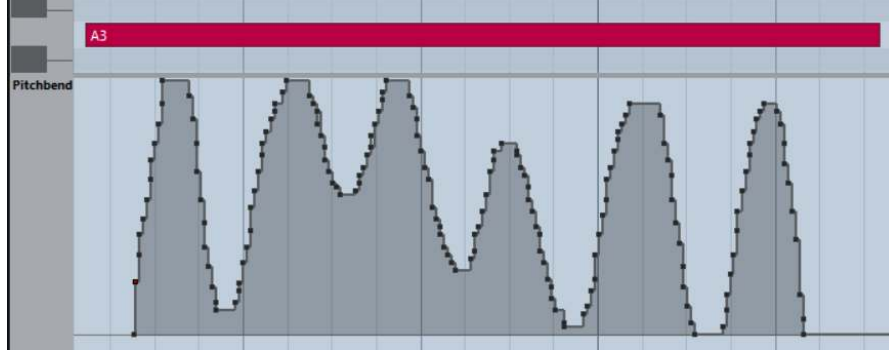


Figure 4.1: Pitch bend values of an example musical note.

current value of pitch-bend is denoted by B_i , the amplitude is denoted by A_i , and the duration is denoted by D_i .

$$N_i = \begin{bmatrix} P_i \\ B_i \\ A_i \\ D_i \end{bmatrix}. \quad (4.1)$$

Accordingly, Equation (4.2) shows a sequence of notes N , which are represented using the matrix-based format:

$$N = \begin{bmatrix} P_i & P_{i+1} & P_{i+2} \\ B_i & B_{i+1} & B_{i+2} \\ A_i & A_{i+1} & A_{i+2} \\ D_i & D_{i+1} & D_{i+2} \end{bmatrix}, \dots \quad (4.2)$$

It should be noted that the MIDI note number 0 is used to represent the C_{-1} note, which is an octave below the $C_0 = 16.35Hz$ note [91]; just below the lower limit of human hearing, and is well outside the frequency range of most contemporary musical instruments. Therefore, the occurrence of a C_{-1} note can be safely ruled out, and its MIDI number used to represent a pause.

The first four notes of the C major scale presented by Figure 3.2, with the assumption that all notes have an equal moderately loud amplitude (refer Appendix A.1), no pitch bends, and a tempo of $\text{♩} = 120 \text{ bpm}$, can be denoted using the matrix-based representation as follows:

$$S = \begin{bmatrix} 60 & 62 & 64 & 65 \\ 0 & 0 & 0 & 0 \\ 80 & 80 & 80 & 80 \\ 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix}. \quad (4.3)$$

4.1.2 Sampling the Sound: Capturing Music in Time

RNNs are designed to process sequential data, through carefully maintaining a memory of previous inputs. This characteristic allows RNNs to learn temporal dependencies and patterns over time. To achieve the fullest potential of any RNN, the input data must be organized into a discrete time series, where each element in the sequence represents a distinct and equally spaced moment in time.

Hence, the RNN algorithm employed in this research relies on a discrete time series representation of the input MIDI data. This data restructuring is done by sampling the MIDI input at regular intervals of 30 milli-seconds (ms). The choice of sampling frequency is based on psychoacoustic principles, as it is the approximate temporal resolution at which the human auditory system distinguishes between two sequential sound events [92]. The MIDI data is sampled every 30 ms to capture pitch and other expressive parameters at discrete time steps. The MIDI *note number* and *pitchwheel* parameters are used to construct the discrete time series.

MIDI pitch bend is a 14-bit control message that enables continuous pitch modulation of musical notes. The pitch bend message contains a *pitchwheel* parameter in the range 0-16,383, with 8192 representing the neutral no bend position. The actual deviation in pitch is determined by the sound producing device settings. The standard practise is a range of ± 2 semitones for the MIDI pitchwheel range [88, 93]. At each time step t , a combination of the MIDI pitch and pitchwheel values are used to yield the pitch value P , as shown by Equation (4.4). Figure 4.2 shows the first four notes of the C major scale with a pause, its matrix representation denoting pitch and duration, and the time series constructed by sampling its notes.

$$P_t = \text{pitch}_t \pm (\text{pitchwheel}_t - 8192) \times \frac{1}{4096} \quad (4.4)$$

4.1.3 Cepstral Tales: Decoding Timbre, One Coefficient at a Time

While the matrix and time series analyses are suited for monophonic music, polyphony requires a different approach to accommodate the multiple concurrent notes. A spectrogram is a visual representation of the frequency spectrum of a signal as it changes over time. A spectrogram is calculated by the division of a signal into frames, and then computing the amplitudes of each frequency component [94].

Spectrograms are particularly suited for audio analysis, as they are able to display multiple simultaneous frequencies to capture the harmonic structure of polyphonic audio [95, 96]. This allows the analysis of overlapping tones and timbral variations, allowing spectrograms to be used for tasks such as source separation, transcription, and pattern detection [97, 98].

Mel Frequency Cepstral Coefficients (MFCCs) are a widely used feature extraction technique in audio signal processing. MFCCs are able to represent the spectral properties of sound in a

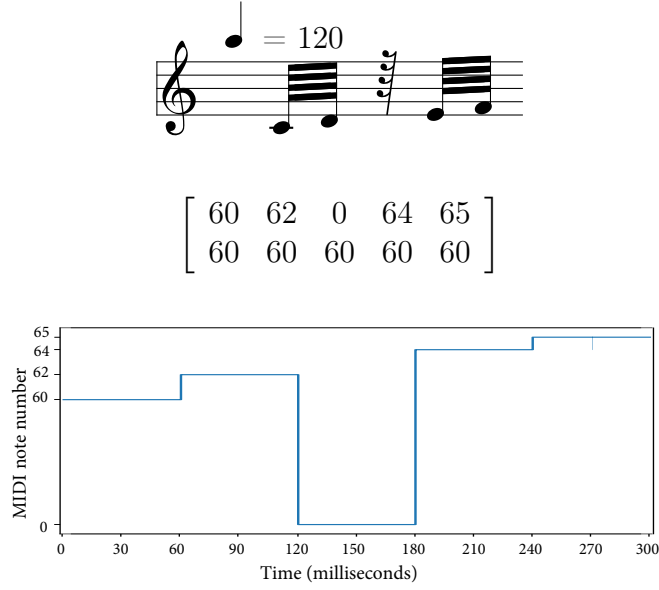


Figure 4.2: The first four notes of the C major scale (*top*), and the corresponding matrix-based representation with pitch and duration (*middle*), and sampled sequence (*bottom*).

compact manner suitable for computational analysis. The main advantage of MFCC is its ability to model the human auditory system's non-linear perception of frequency, which is achieved by mapping the signal's frequency content into the Mel scale - a perceptually motivated scale with increasing resolution with frequency [99]. Following is a detailed description of the MFCC computation.

1. The algorithms presented by this research computes the MFCC of the windowed audio signal every 30ms. The windowed audio signal $y[n]$ of length N , is then multiplied by a Hamming window $w[n]$ as illustrated in Equation (4.5) and Equation (4.6). The window function is used to reduce spectra leakage.

$$y[n] = x[n] \cdot w[n], 0 \leq n \leq N \quad (4.5)$$

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right) \quad (4.6)$$

2. Following the window function, the magnitude spectrum of the windowed audio segment is computed using the Fast Fourier Transform (FFT). The FFT is used to transform the signal from the time domain to the frequency domain as shown by Equation (4.7).

$$X[k] = \sum_{n=0}^{N-1} y[n] e^{-j \frac{2\pi}{N} kn}, \quad 0 \leq k < N \quad (4.7)$$

3. $X[k]$ represents the windowed audio segment in the frequency. The magnitude spectrum is then obtained as shown by Equation (4.8)

$$|X[k]| = \sqrt{\text{Re}(X[k])^2 + \text{Im}(X[k])^2} \quad (4.8)$$

4. The magnitude spectrum is then passed through a bank of filters set according to the Mel Scale. The Mel scale can be defined by Equation (4.9) for frequency f in hz, where $m(f)$ is the corresponding Mel frequency.

$$m(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right) \quad (4.9)$$

The filter bank consists of M filters $H_m[k]$ as shown by Equation (4.10). The filters are designed to emphasize lower frequencies and have a low resolution at higher frequencies.

$$H_m[k] = \begin{cases} \frac{f[k]-f(m-1)}{f(m)-f(m-1)}, & f(m-1) \leq f[k] \leq f(m) \\ \frac{f(m+1)-f[k]}{f(m+1)-f(m)}, & f(m) \leq f[k] \leq f(m+1) \\ 0, & \text{else} \end{cases} \quad (4.10)$$

The filtered energy for each filter m is then computed as shown by Equation (4.11).

$$E_m = \sum_k |X[k]|^2 \cdot H_m[k] \quad (4.11)$$

5. Upon computing the energy values, the Discrete Cosine Transform (DCT) is computed to compress the information into a smaller set of coefficients. Equation (4.12) shows the computing of the n -th MFCC, where $0 \leq n < N_{\text{coeff}}$.

$$c_n = \sum_{m=1}^M \log E_m \cdot \cos \left(\frac{\pi n}{M} \left(m - \frac{1}{2} \right) \right) \quad (4.12)$$

The experimental setup presented by this thesis is primarily concerned about SMIs. As all calculations must be done in real-time within a low powered embedded device, the FFT size was set to $N = 1024$ (Equation (4.7)), with a hop size of 512. The number of Mel bins was set to $M = 10$ (Equation (4.10)). These values were experimentally identified to be the best compromise between accuracy and speed to be computed on the utilized embedded device.

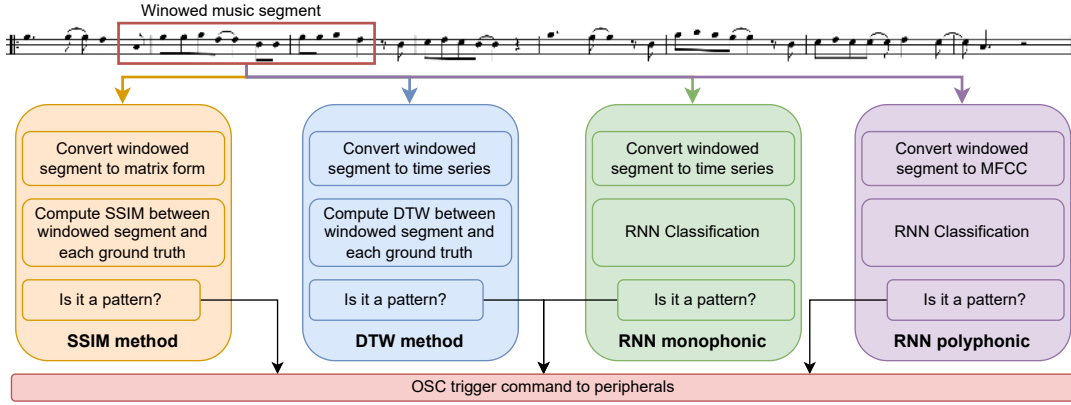


Figure 4.3: Block diagram comparing the different methods used.

4.2 Algorithmic Jam Session: Comparing Approaches

The comparison of musical pattern detection algorithms described in this thesis presents an intersection of DSP, computer vision, and ML approaches. Each method offers its own capabilities in capturing the complex relationships within musical patterns. This section highlights the methodology of the algorithms designed around the three methods: SSIM, DTW, and RNN.

For a fair evaluation, each method has an identical experimental setup. The common steps for evaluating each algorithm are:

1. Extraction of a windowed section of music;
2. Feature extraction;
3. Comparison with the ground truth patterns;
4. Evaluation of whether the windowed music section is a match to any ground truth.

Figure 4.3 shows a graphical illustration of the evaluation framework. A square window is used to preserve the characteristics of the music. The size of the window is dependent on the specific detection method. Each detection method has a different feature extraction method:

- The SSIM uses the MIDI data transformed into a matrix-based representation.
- The DTW and the monophonic version of the RNN uses the MIDI data, sampled and restructured into a discrete time series.
- The polyphonic version of the RNN uses a windowed audio segment, in the form of a MFCC spectrogram.

Upon feature extraction, the SSIM, and DTW methods will compare the features against those of the list of ground-truths. The RNN is trained to classify the windowed segment. If the probability is high enough as discussed in Equation (3.1), the segment is confirmed as a pattern.

4.2.1 The Structural Similarity Matrix: Mapping Musical Connections

The SSIM is a Computer Vision technique to measure the *similarity* between two images. SSIM provides advantages over standard comparison methods such as a Mean Squared Error, and it derives inspiration from the human visual perception to identify structural information from a scene [100]. The SSIM index for two image signals x and y , is presented as follows: [100];

$$SSIM_{(x,y)} = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}. \quad (4.13)$$

where C_1 and C_2 are constants to ensure stability when the denominator reaches 0 for each measure. μ_x , and μ_y is measured by averaging over all the pixel values. σ_x^2 and σ_y^2 is measured by computing the variance of all the pixel values, and the covariance between x and y are denoted by σ_{xy} , as shown in Equation (4.15). They are defined for each pixel i of an arbitrary variable x as below.

$$\mu_x = \frac{1}{N} \sum_{i=1}^N x_i. \quad (4.14)$$

$$\sigma_x = \left(\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu_x)^2 \right)^{\frac{1}{2}}. \quad (4.15)$$

The SSIM outputs a single value between -1 and $+1$. A value of $+1$ indicates that the given images are identical, and a value of -1 indicates that the given images are very different.

Bicubic Interpolation

Interpolation is a common technique used in Computer Vision to resize images. There are several interpolation methods available for image resizing, and bicubic interpolation is a common technique used by several popular image processing software [101].

In the context of computer vision, bicubic interpolation calculates a weighted average of the surrounding 4x4 pixel square to calculate the interpolated value for an unknown pixel u . Each surrounding pixel is assigned a weight based on its distance to the destination pixel [102]. The kernel for the bicubic interpolation is illustrated in Equation (4.16), where d is the distance between the interpolated point and the grid point [101, 103].

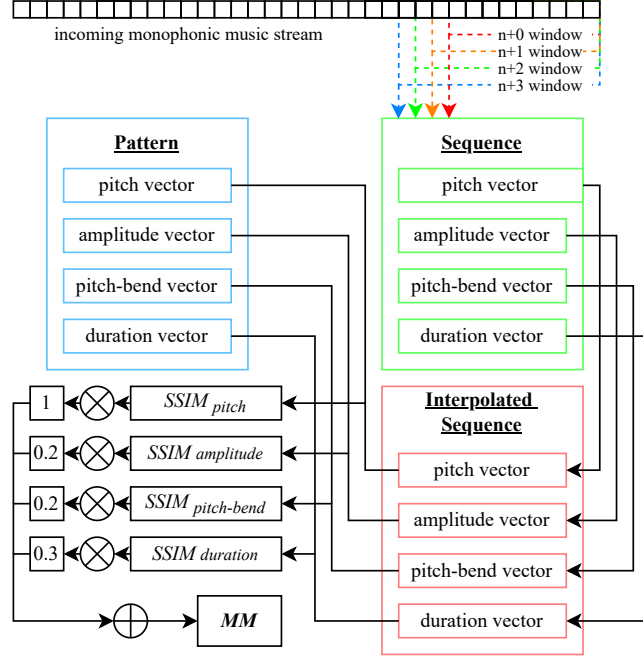


Figure 4.4: Flow chart of the proposed algorithm.

$$u(d) = \begin{cases} 3/2|d|3 - 5/2|d|2 + 1, & 0 \leq |d| < 1, \\ -1/2|d|3 - 5/2|d|2 - 4|d| + 2, & 1 \leq |d| < 2, \\ 0 & 2 < |d|. \end{cases} \quad (4.16)$$

Dynamic Window

Musicians may add extra notes or remove some notes from a pattern based on their expressive needs at a live performance. With the understanding that a sequence can still be perceived as a pattern despite some extra notes being present (Equation (3.2)), a dynamic window is used to obtain sequences to compute the SSIM against each pattern.

The size of the window is used as a control parameter to allow a p number of notes to be added to the pattern. The evaluations presented in this thesis used the value of $0 \leq p \leq 3$, which allows for a maximum of 3 extra notes to be added. As illustrated in Figure 4.4, if the length of a pattern is n , multiple sequences will be extracted from the music stream, each with lengths $n + 0$, $n + 1$, $n + 2$, and $n + 3$. p was chosen arbitrarily, and it may be adjusted to make the system as strict, or as lenient as the user desires.

Resizing using Bicubic Interpolation

One of the primary criterion for the SSIM is that the two matrices being compared has to be identical in size. As the windowed segments may vary in length by a factor of p , a bicubic interpolation is performed to resize the sequences of length $n + p$, to the length of the pattern, n (Equation (4.16)). Following which, the interpolated vectors are used to compute the SSIM values with the pattern attribute vectors as illustrated in Figure 4.4.

SSIM Computation

The interpolated sequence and the pattern is first split into pitch, amplitude, pitch-bend, and duration vectors. This step is taken to ensure that each attributes contribute independently to the final measure. The SSIM between each pair of the pitch, amplitude, pitch-bend and duration vectors of the interpolated sequence and each pattern are then computed as illustrated by Figure 4.4.

For each vector pair x and y , the μ_x , μ_y , σ_x , and σ_y are calculated as presented in Equation (4.13). The calculated μ_x , μ_y , σ_x , and σ_y values are then used to compute the SSIM value for x and y , for all four vector pairs.

The constants C_1 , and C_2 are small constants, included to avoid instability when $\mu_x^2 + \mu_y^2$, and $\sigma_x^2 + \sigma_y^2$, respectively, is very close to zero. The evaluations used arbitrary values $K_1 = 0.01$, and $K_2 = 0.03$, same as [100]. Furthermor the value $L = 127$, was selected as the dynamic range for pitch and amplitude. C_1 and C_2 are defined as [100]:

$$C_1 = (K_1 L)^2, \quad (4.17)$$

$$C_2 = (K_2 L)^2. \quad (4.18)$$

As shown above, four seperate SSIM values for each pair of note attributes are computed. For each sequence x , and pattern y , the SSIM between the two pitch vectors is $SS_{p(x,y)}$, the SSIM between the two amplitude vectors is $SS_{a(x,y)}$, the SSIM between the two pitch-bend vectors is $SS_{pb(x,y)}$, and the SSIM between the two duration vectors is $SS_{d(x,y)}$. The four SSIM values are then used to compute the final metric between each sequence and pattern - the MM.

MM

MM is is obtained through a weighted summation of all four SSIM values as illustrated in Equation (4.19). This step ensures that all attributes in the sequence and the pattern make a contribution based on their importance to the definition of a pattern. A greater difference in one set of attributes will lead to a smaller MM. If all attributes are identical, a value of $MM = 1.7$ is obtained, and for every deviation from the pattern, the MM will decrease. The lowest possible value is $MM = -1.7$

$$MM_{(x,y)} = SS_{p(x,y)} + 0.2 \times SS_{a(x,y)} + 0.2 \times SS_{pb(x,y)} + 0.3 \times SS_{d(x,y)}. \quad (4.19)$$

MM is obtained through a weighted summation of the SSIM values of the four note attribute vector pairs. The weights for each SSIM value, as shown by Equation (4.19), were selected arbitrarily based on the relative contribution each attribute makes to the definition of the pattern.

The on-time pitch pairs are the most important attribute to determine if a sequence and a pattern is a match [26]. It was empirically identified that the amplitude contributes less to the definition of a pattern. It should be noted that the user may change these weights based on the importance they give to each note attribute.

Appendix Appendix B.2 shows the computation of MM for a pattern and one of its occurrences in the JKUPDD dataset.

4.2.2 Dynamic Time Warping: Aligning Patterns Across the Beat

Dynamic Time Warping (DTW) is a time-series analysis method for temporal sequences that may vary in speed and has applications in domains such as speech recognition and stock market analysis. The similarity, or the distance between two sequences is computed by constructing a distance matrix between each point and measuring the shortest path. This property of DTW makes it an ideal candidate to measure similarity between musical sequences of different lengths such as that of a musician playing a pattern with some variation.

For two sequences x and x' , which lie in the same dimensional space with respective lengths n and m , DTW distance can be expressed as:

$$\mathbf{DTW}_q(x, x') = \min_{\pi \in A(x, x')} \left(\sum_{(i,j) \in \pi} d(x_i, x'_j)^q \right)^{\frac{1}{q}}, \quad (4.20)$$

where an alignment path π is a sequence of K index pairs $((i_0, j_0), (i_1, j_1), \dots, (i_{K-1}, j_{K-1}))$, which represents the shortest possible path between x and x' , and $A(x, x')$ is the set of all possible paths. The following criteria must be met for the alignment path π to be considered admissible:

- The beginning and the end of the time series' must be matched: $\pi_0 = (0, 0)$, and $\pi_{K-1} = (n-1, m-1)$;
- The sequence should increment in both i and j , and all indexes should be present: $i_{k-1} \leq i_k \leq i_{k-1} + 1$, and $j_{k-1} \leq j_k \leq j_{k-1} + 1$.

The DTW method first extracts a sub-sequence of notes from the incoming music through a windowing function. The size of the window will be dependent on the lengths of the patterns

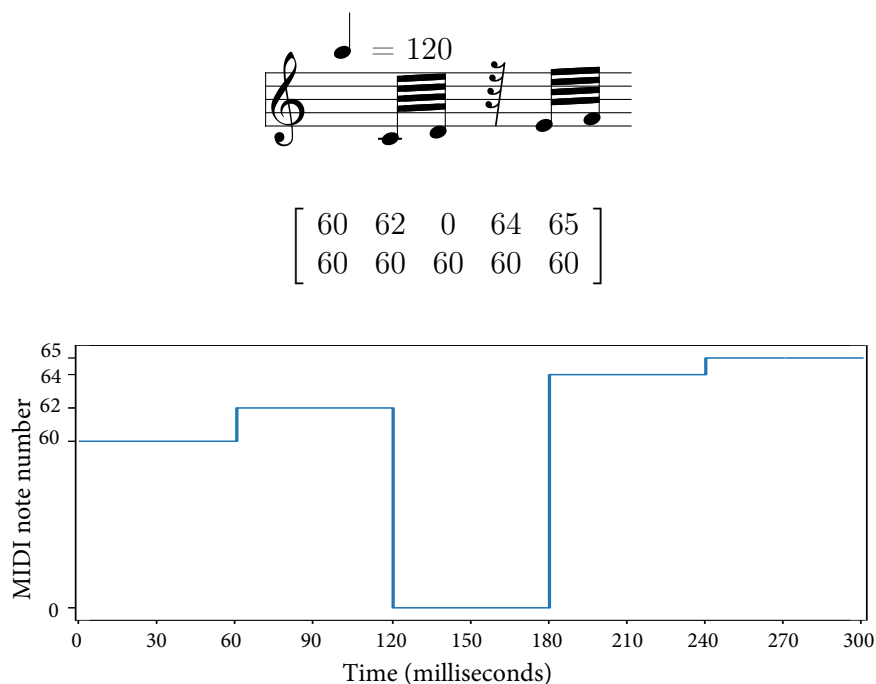


Figure 4.5: The first four notes of the C major scale (*top*), and the corresponding matrix-based representation with pitch and duration (*middle*), and the sampled sequence with 30ms sampling frequency (*bottom*).

that are to be recognized. As explained in Chapter 3, a musician may add an average of 24% of additional notes and pauses. Hence, the size of the window function was set to be approximately 20% larger than the length of the longest pattern.

Representation of Music

As presented in Section 4.1.1 (and reported in [19,104]), the MIDI data is converted to a matrix-based representation, where the pitch and duration of each note is used to represent it. Upon receiving each MIDI event, gradient changes in pitch are scanned in order to determine changes between notes and pauses. Whenever a gradient change is detected i.e., the transition between a *note on* message and a *note off* message, or receiving a *pitch bend* message, it is considered as a new event. Figure 4.5 shows the first four notes of a C-major scale with a pause, and its matrix based representation.

The pitch is an integer in the range 0–127. However, pitch bends are denoted using a separate *pitchwheel* message which lies in the range 0–16383 [88]. Although this range of values could be mapped to a pitch change of any interval in a synthesizer, the common practice is to use a pitch bend of ± 2 semitones [88]. This assumption is maintained the pitch of a note is denoted as a floating point value where a unit pitchwheel increment is considered as a $1/8192$ increment in pitch.

4.2. Algorithmic Jam Session: Comparing Approaches

A sub-sequence is obtained upon receiving each new event - may it be a note, pause, or a pitch-bend. The DTW between each ground-truth and the sub-sequence is then computed to measure similarity. The MIDI notes are converted to the matrix-based representation format consisting of a pitch vector and a duration vector (see Figure 4.5), which requires the use of a multidimensional DTW to obtain a similarity.

Multidimensional DTW

This multidimensional DTW may be achieved by a singular warping path across all dimensions, or by merged warping paths of each individual dimension [105, 106]. The latter was chosen, as it enables a controlled *contribution* of each attribute towards the final similarity measure. The merged warping path for two matrices $A \in \mathbb{R}^{n \times D}$, $B \in \mathbb{R}^{m \times D}$ can be as follows:

1. Split the matrices into vectors:

$$A = \{\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_D\}, \quad \mathbf{a}_d \in \mathbb{R}^n \quad (4.21)$$

$$B = \{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_D\}, \quad \mathbf{b}_d \in \mathbb{R}^m \quad (4.22)$$

2. Compute the DTW distance between each vector pair, where $d \in \mathbb{Z}$, $0 \leq d \leq D$:

$$\mathbf{DTW}_d(a_d, b_d) = \min_{\pi \in A(x, x')} \left(\sum_{(i,j) \in \pi}^d (a_{di}, b_{dj})^q \right)^{\frac{1}{d}}, \quad (4.23)$$

3. Upon computing each DTW distance between vector pairs, the summation can be obtained as:

$$DTW_{\text{merged}}(A, B) = \sum_{d=1}^D DTW_d(A, B) \quad (4.24)$$

It was empirically identified that while the duration of notes do play a part in defining a pattern, it could be given a *lesser* importance. Refer to Figure 4.6 for an example of this scenario, where significant changes in some *less-important* attributes may alter the perceptual definition. Hence, a weighted summation was implemented to obtain the final metric. Equation (4.25) could be modified to include the weights w_d :

$$DTW_{\text{merged}}(A, B) = \sum_{d=1}^D DTW_d(A, B) \times w_d \quad (4.25)$$

The DTW method presented uses a weighted sum to obtain the final metric. The algorithm splits the sequence to a pitch vector s_p and a duration vector s_d . As shown in Figure 4.7, a



Figure 4.6: C, D, E, F notes with a short pause (top). The same notes, but with a significantly larger pause (bottom). Due to the longer pause, the two patterns will not seem perceptually equivalent despite the pitches being identical.

1-dimensional DTW for pitch and duration (DTW_p and DTW_d) is obtained between each pitch and duration vector in the sequence (s_p and s_d), and the corresponding pitch and duration vectors of a pattern i in a list of patterns (p_{pi} and p_{di}), to obtain DTW_p and DTW_d . The final metric is obtained via a weighted summation of DTW_p and DTW_d , where w_p and w_d are the weights for pitch and duration respectively. The weights were empirically identified to obtain the highest accuracy. Algorithm 1 shows the pseudo code of the DTW method.

Algorithm 1 Weighted DTW method.

1. **Input:** Sequence $s = (s_p, s_d)$, Patterns $\{p_1, p_2, \dots, p_n\}$, Weights w_p, w_d
 2. **Output:** Final metrics $\{\text{Metric}_1, \text{Metric}_2, \dots, \text{Metric}_n\}$
 3. Split sequence s into pitch vector s_p and duration vector s_d
 4. For each pattern i in the list of patterns $\{p_1, p_2, \dots, p_n\}$:
 - (a) Extract pitch vector p_{pi} and duration vector p_{di} from pattern i
 - (b) Compute pitch DTW: $DTW_p \leftarrow \text{DTW}(s_p, p_{pi})$
 - (c) Compute duration DTW: $DTW_d \leftarrow \text{DTW}(s_d, p_{di})$
 - (d) Compute weighted sum: $\text{Metric}_i \leftarrow w_p \cdot DTW_p + w_d \cdot DTW_d$
 5. Return the list of metrics: $\{\text{Metric}_1, \text{Metric}_2, \dots, \text{Metric}_n\}$
-

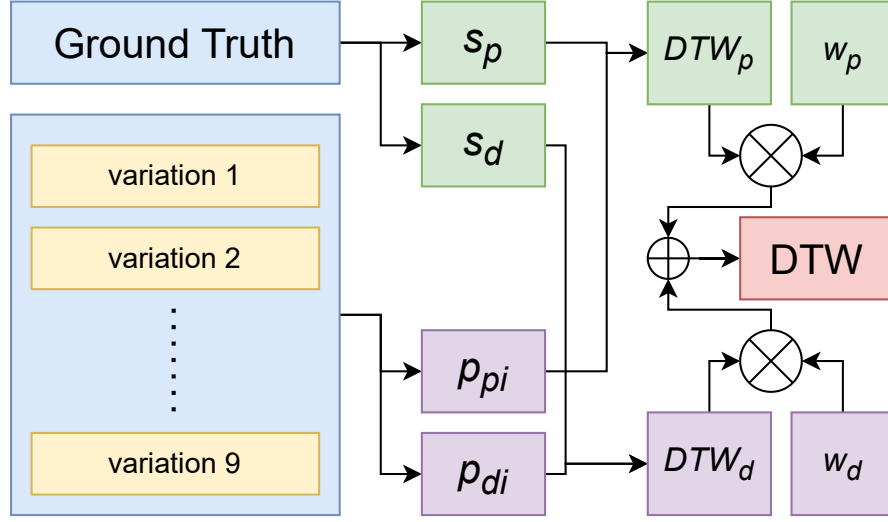


Figure 4.7: A block diagram of the DTW algorithm.

4.2.3 Recurrent Neural Network: Learning Loops and Layers in Music

RNN are a subclass of artificial neural networks specifically designed to process sequential data. This sequentiality is achieved by maintaining a memory of previous inputs. RNNs maintain feedback connections, unlike traditional feedforward networks, which allow them to maintain a hidden state that updates itself over time, effectively giving them a *memory* of past inputs [107]. This property makes RNNs well suited for tasks such as language translation, time series prediction and analysis, and well as speech synthesis where the temporal structure is essential in understanding patterns.

RNN will process an input sequence $x = \{x_1, x_2, \dots, x_T\}$, where x_t represent a feature at time-step t . The RNN updates its hidden state h_t , based on the current input x_t and the previous hidden state h_{t-1} as shown in Equation (4.26):

$$\mathbf{h}_t = f(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b}_h) \quad (4.26)$$

where:

- h_t is the hidden state at time-step t ;
- W_h is the weight matrix of the hidden state;
- W_x is the weight matrix of the input;
- b_h is the bias;
- f is the non-linear activation function (e.g., $\tanh$, ReLU)

The hidden state h_t will store the information on the previous inputs until time t . The output of the RNN y_t is shown by Equation (4.27).

$$\mathbf{y}_t = g(W_y \mathbf{h}_t + \mathbf{b}_y) \quad (4.27)$$

where:

- W_y is the weight matrix of the output;
- b_y is the output bias;
- g is the activation function (e.g., *softmax* for classification, *linear* for regression);

Several different RNN configurations exist, each with their own merits and drawbacks. For our approach, we empirically identified that a stacked configuration, consisting of a fully connected RNN layer, a GRU layer, and a LSTM layer yielded the best accuracy.

A Fully Connected RNN consists of an architecture where each node is connected to every node in the next layer. This comprehensive connectivity allows information to flow between all nodes. Fully connected RNNs can capture intricate relationships present in sequential data more effectively than simpler architectures such as Vanilla RNNs. However, due to their complex structure, fully connected RNNs may require more training data and computational resources.

The long-short term memory (LSTM) networks involve a more complex memory which allows it to retain information over long sequences. LSTMs consist of a hidden state and a cell state, which contain the input at the current time step along with the information learned from previous time steps. The LSTM has three gates as illustrated by Figure 4.8. The input gate regulates the flow of new information into the cell state, the forget gate controls the information to be discarded, and the output gate controls the flow of information from the cell state to the hidden state. This architecture allows the LSTM layer to capture long-term dependencies in sequential data.

The gated recurrent unit (GRU) layer is also able to overcome the vanishing gradient through its update gate and reset gate, which determines the degree of past information forwarded to the future and the amount of past information to discard respectively. The GRU also includes a candidate hidden state as well as a final hidden state, which allows it to selectively update its hidden state at each time step, making it well-suited for tasks with sequential information.

Data representation for monophony

Presenting monophonic data as a discrete time series for analysis by an RNN is essential because it aligns with the sequential nature of music, where each note or sound event follows in a temporal order. This representation allows the RNN to process the data in a way that captures temporal dependencies and relationships between successive notes, which is crucial for understanding musical patterns and structures. By discretizing the data into a time series, each element can be systematically analyzed, enabling the RNN to learn from the sequence of events and make predictions or classifications based on the learned patterns.

The RNN method uses a discrete time series representation of the input data. This is achieved by sampling the MIDI input every 30 milliseconds to obtain a sub-sequence with equal temporal

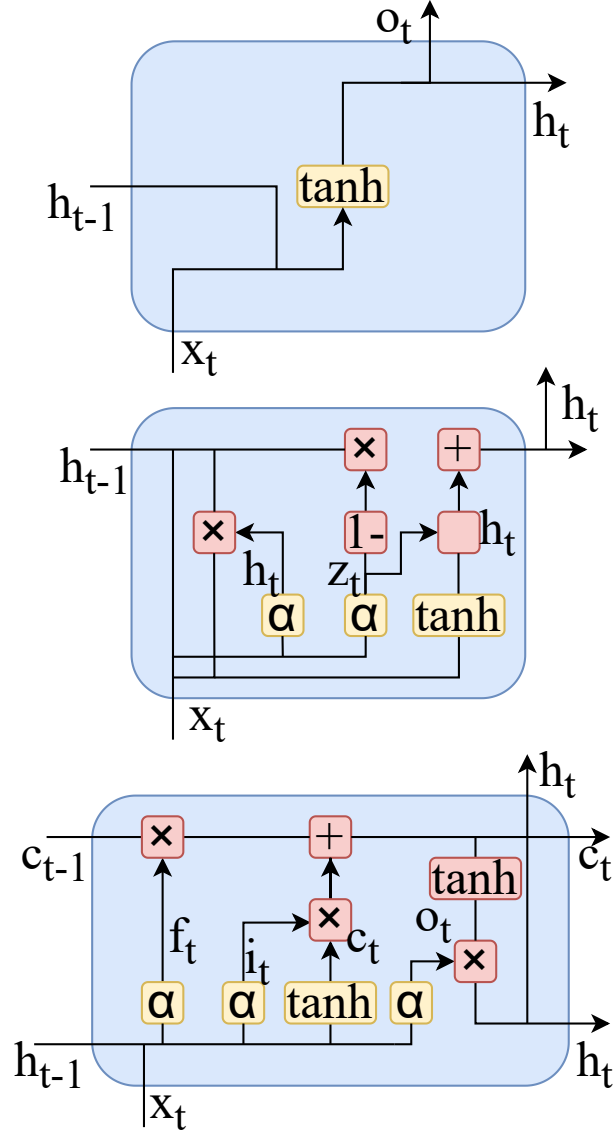


Figure 4.8: A traditional RNN layer (*top*); a GRU layer (*middle*) with a reset gate r_t and an update gate z_t ; and a LSTM layer (*bottom*) with a forget gate f_t , input gate i_t , and output gate o_t . At each timestep t : x_t , o_t , h_t , and c_t are the input, output, hidden state, and intermediate cell state, α and $\tanh$ are the sigmoid and tanh activation functions respectively.

spacing between elements. The interval of 30 ms was chosen as it is approximately the temporal resolution of the human hearing system to distinguish two sequential sound events [92]. Similar to the DTW method, the pitch is denoted as a floating point value where a unit increase in MIDI pitchwheel corresponds to $1/8192$ increment in pitch. Figure 4.5 shows an example for this time-series representation, where the first four notes of the C major scale in standard musical format and the sampled time series.

Data representation for polyphony

The sound produced by the musical instrument is fed directly into the algorithm, which extracts a windowed stream of music every 30ms. The 30ms sampling interval was selected as it is approximately the temporal resolution of the human hearing system to distinguish between two sequential sound events [92]. Upon extracting the windowed audio segment, the Mel Frequency Cepstral Coefficients (MFCC) are calculated to extract the audio features. The MFCC was selected to represent the audio features as it is designed to mimic the human auditory system's response to sound. Moreover, the MFCC enables for a compact representation which allows for the efficient encoding of the most important aspects of the audio, while simultaneously reducing the dimensionality of the data. MFCC's also captures both temporal and spectral characteristics of the audio, which is particularly important for music analysis where both rhythm and pitch information are relevant, like in the case of pattern detection tasks.

We used the MFCC with a FFT size of 1024, and a hop size of 512. The number of Mel bins was set to 10. These values were experimentally identified to be the best compromise between accuracy and speed to be computed on the utilized embedded device. The MFCC of the windowed audio segment was computed and used as the input to the RNN for classification. The RNN was trained using a synthetically generated dataset, which consists of artificially created variations for each pattern that is selected for detection. The dataset and algorithm were designed to account for up to a total of 10 patterns.

Synthesizing Variations: Rule-Based Models for Training Data

A primary requirement for any accurate deep learning model is the availability of sufficient training data that correspond to possible variations and fluctuations of the input. In an approach such as ours, the training set must adequately represent all variations that musicians may perform as described in Section 3.2. It is impractical for a musician to record numerous versions of each single pattern. Hence we analyzed the pattern variations present in the dataset to identify the most commonly applied variations, and apply them to create a synthetic training dataset of expressive and melodic variations given a single ground-truth.

These variations are inspired by those present in the recorded dataset. Upon extensive analysis of the dataset, we identified several expressive variations that were common across most musicians. Some examples for said variations are:

4.2. Algorithmic Jam Session: Comparing Approaches

- Changing the duration of notes and pauses;
- Doubling one, or a number of notes;
- Adding pitch modulations on one, or several notes;
- Using the pitch-bend controller or bending the string to reach a subsequent note;
- Adding chromatic passing notes between notes;
- Removing notes and/or pauses;
- Addition of extra notes and/or pauses;
- Alternating between staccato and legato style playing;
- Changing octaves and/ or transpositions.

Such common expressive variations are used as rules when creating the synthetic dataset. Some examples of the applied rules are:

- Changing the duration of arbitrary notes and pauses by a constant factor, so as the total duration of the melody remains unchanged.
- Doubling notes: It was identified that approximately 23.6% notes may be added as extra. An arbitrary number of notes between 0 and 23.6% are doubled, while halving their duration.
- Removal of notes: It was identified that approximately 3.3% notes may be removed. An arbitrary number of notes between 0 and 3.3% are removed. The durations are left unchanged, and the removed duration is added to other arbitrary notes/pauses.
- Pitch bends: For an arbitrary number of notes between 0 and 9%, where each note and the subsequent note are less than 2 semitones apart, a pitch bend is applied to the first note to gracefully reach the pitch of the subsequent note, which is removed and its duration added to the first note.
- Chromatic notes: If a note and the subsequent note are exactly 2 semitones apart, adding the middle note.

There have been studies that apply rule-based modulation to mimic certain styles of music, where the rules may be strict [108]. However, our approach tries to generalize expressive variations that has the potential to work with most musicians' playing styles. Longer musical patterns have the potential for more expressive variation, and vice versa. It was empirically identified that approximately 10000 such variations per ground-truth were sufficient.

During training a neural network, it is crucial to have all training data in an equal size. As the synthesized dataset contains different sized variations, we pad the training dataset to fit the length of the longest variation present.

As the task is a relatively simple time series classification, and as the model needs to be light enough to run on an embedded system in real time, the RNN layer consists of 64 neurons, the GRU layer consists of 32 neurons, and the LSTM layer consists of 64 neurons. Furthermore, we use a masking layer to disregard any padded inputs in the training dataset. The training dataset is also sampled at 30ms increments to maintain consistency between training data and actual input.

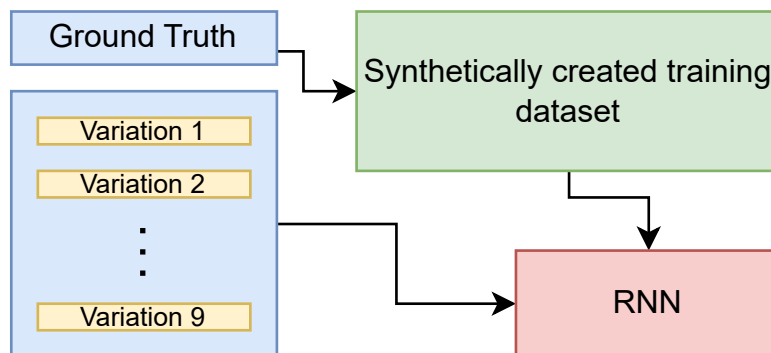


Figure 4.9: A Block diagram of evaluation of the RNN algorithm.

The evaluation of the RNN was kept as close as possible to the evaluation of the DTW to maintain a fair comparison. 10 groups of musical patterns, each with 1 to 10 patterns were used. For each case, the RNN was trained with synthetically created variations as discussed, and evaluated against the *actual* pattern variations recorded by the musician. Figure 4.9 shows a block diagram of the evaluation.

4.2.4 ONNX

The Open Neural Network Exchange (ONNX) is an open-source format designed to enable interoperability between different machine learning (ML) frameworks and tools. Initially developed by Facebook and Microsoft in 2017, ONNX provides a standardized representation for machine learning models, allowing developers to train models in one framework (e.g., PyTorch or TensorFlow) and deploy them seamlessly in another (e.g., ONNX Runtime or custom inference engines) [109,110]. ONNX achieves this by defining a computational graph format, which represents the model's structure, operations, and parameters in a framework-agnostic manner. It supports a wide range of operations and data types, enabling the representation of both traditional ML algorithms and deep learning architectures.

ONNX is particularly valuable in scenarios where performance and deployment flexibility are critical. Its lightweight format allows developers to optimize models for inference on various platforms, including edge devices, servers, and cloud environments. With broad support from major industry players like NVIDIA, AWS, and Intel, ONNX enables the deployment of AI models across GPUs, CPUs, and specialized accelerators. Furthermore, ONNX includes tools like ONNX Runtime, which is optimized for low-latency and high-performance inference. This makes ONNX a powerful solution for developers aiming to streamline the end-to-end AI workflow while ensuring compatibility and efficiency across diverse deployment environments.

Chapter 5

Hitting the Right Notes: Engineering Real-Time Detection Systems

5.1 Overture to Intelligence

SMIs are an innovative class of Digital Musical Instruments (DMIs) that integrates embedded intelligence, bidirectional wireless connections and onboard sound delivery systems. These instruments, positioned at the intersection of music technology and IoT, must be capable of processing data in real-time and communicating with external devices without the use of a central computing unit (i.e., external server or laptop). Several commercial and research prototypes demonstrate the practical implementation of SMIs. The smart mandolin [111] and the smart cajon [112] are some notable examples capable of performing complex computational tasks within the physical boundary of the instrument.

In this chapter, we discuss the design and development of three smart instruments with embedded real-time pattern detection capabilities. These instruments are:

1. A smart electric guitar embedded with a real-time audio, as well as MIDI pattern detection systems.
2. A smart keyboard embedded with real-time audio, as well as MIDI pattern detection systems.
3. A smart drum kit embedded with a real-time MIDI pattern detection system.

These SMIs are equipped with the ability to detect if the musician plays one of the patterns in a previously specified list. Moreover, the SMIs are able to emit control messages wirelessly upon detection of a pattern. Both tasks can be accomplished in real time. The SMIs are also able to communicate with external servers for on-cloud-training for the ML models.

These tasks are accomplished through the use of an embedded computing device mounted on the instrument itself. All real-time algorithms are processed, and all communications are

done on the embedded system. For our evaluations, we chose the Raspberry pi 4 with the Hi-Fi berry audio interface shield as the embedded computing system due to its small footprint on the instrument and powerful performance. As a real-time operation is needed, the chosen operating system for the Raspberry Pi is Elk Audio OS [113]. The algorithms presented in this thesis are hosted within Virtual Studio Technology (VST) plugins that run on Elk Audio OS.

In this chapter, we introduce Elk Audio OS and present *Hot Licks*, the VST plugin that houses the detection algorithm. We will also introduce the wireless controller designed as a Graphical User Interface (GUI) for the plugin. Finally, we introduce the remote training function implemented to enable fast training of complex ML models on the cloud to fulfill scenarios envisioned in the DSP as a Service paradigm [114].

5.2 The Digital Heartbeat: Elk Audio OS

Elk Audio OS is a Linux-based operating system specifically designed for real-time audio applications¹ [113]. Elk Audio OS has a very specific focus on ultra-low latency performance, as well as integrated audio processing within embedded systems. Built around a real-time kernel, it is ideal for applications which require minimal latency such as SMIs and live audio effects. In addition to its real-time operation, Elk Audio OS also features a headless Digital Audio Workstation (DAW), which provides a platform to run audio processing tasks without the use of a GUI. Figure 5.1 shows a high level block diagram of the operating system. With its hardware components highlighted in white, standard Linux components in pink, Elk components in blue/orange, and other custom components in green².

We use the general purpose GPIO ports to connect a HifiBerry ADC+DAC pro audio interface to the Raspberry Pi 4. The audio interface acts as the primary component in capturing audio from instruments, and outputting audio through its balanced pair of TRS connections. The interface supports both input and output sample rates up to 192 kHz/24bit. Figure 5.2 shows the selected audio interface attached to the GPIO pins of a Raspberry Pi [115].

5.2.1 The Quantum Pulse: real-time kernel

The core of Elk Audio OS is its real-time Xenomai Cobalt kernel, optimized to prioritize audio tasks over less critical system processes. This prioritization is a crucial feature for musical instruments where latency must be low enough to allow real-time operation [116]. Standard Linux kernels often fail to meet the latency required by audio applications due to their general purpose nature [117]. Elk Audio OS addresses this limitation by implementing features such as preemptive scheduling, real-time thread priorities, and low-latency interrupt handling [118].

¹www.elk.audio

²https://elk-audio.github.io/elk-docs/html/intro/how_elk_works.html

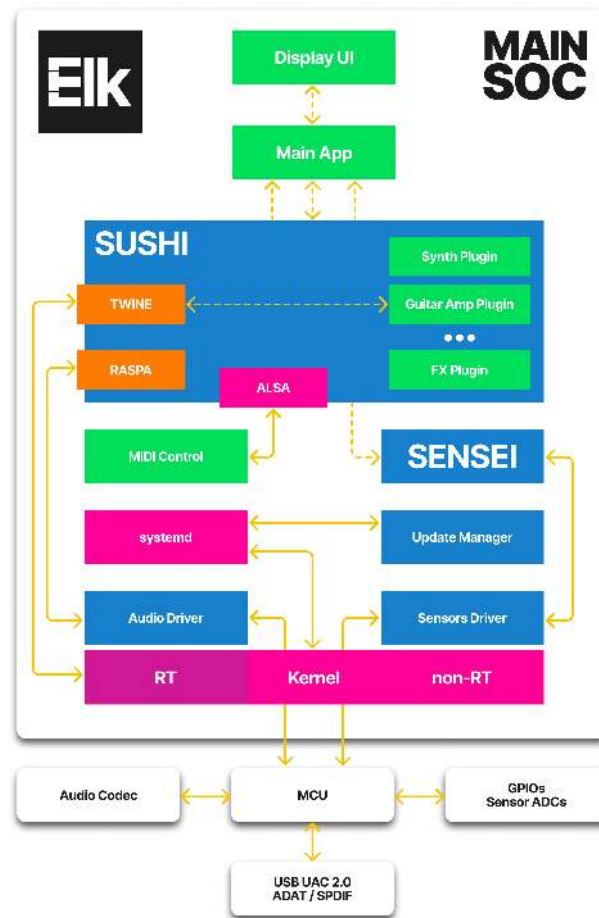


Figure 5.1: A block diagram of Elk Audio OS.



Figure 5.2: HifiBerry ADC+DAC pro audio interface.

With these enhancements, Elk Audio OS achieves round trip latency levels as low as 1ms, making it capable of real-time operation even on resource constrained hardware.

As an example, in an SMI equipped with Elk Audio OS, the real-time kernel allows for instantaneous processing of audio signals, enabling features such as real-time pitch shifting, dynamic effects, and interaction with external devices. This low-latency performance ensures that musicians experience natural and responsive feedback, a crucial aspect in any musical instrument.

5.2.2 Headless DAW: The Ghost in the Machine

Sushi is a track-based, headless DAW that is bundled with Elk Audio OS. Sushi works as a plugin host that supports multiple plugin standards including VST 2.4, VST 3.7 and LV2 plugins. It features advanced audio routing, high performance and stability with low latency, and it can be controlled by external devices through MIDI, Open Sound Control (OSC), or gRPC interfaces. Sushi is highly optimized for audio processing, and it also has multi-threaded support for tracks [119].

Sushi shares the same basic architecture as common DAWs with an unlimited number of parallel channels. Each channel can support as many plugins as the CPU can handle. Sushi also supports the MIDI protocol, along with auxiliary sends and return tracks for effects processing.

It supports multiple audio frontends including RASPA, Jack, and PortAudio. Sushi can be run on many Linux systems as it supports Jack - the most common audio framework on Linux. Sushi also supports many protocols for communication with external sources. Sushi is a headless host (without any GUI) and users can control sushi and the plugins directly using MIDI, Ableton Link, or OSC. Figure 5.3 shows the architecture of the headless DAW. In this research, we exploit the MIDI, audio and OSC functionality of Sushi to run the real-time pattern detection system.

The ability to meet the demands of real-time audio processing, the adaptability to embedded hardware, and the integration with digital workflows make Elk Audio OS an ideal technological enabler for SMIs. The real-time kernel ensures that all processing is done with minimal latency, which enables the real-time operation that we require. Moreover, the ability to control with external devices is crucial for our SMIs as control messages must be sent to devices that are triggered by the pattern detection.

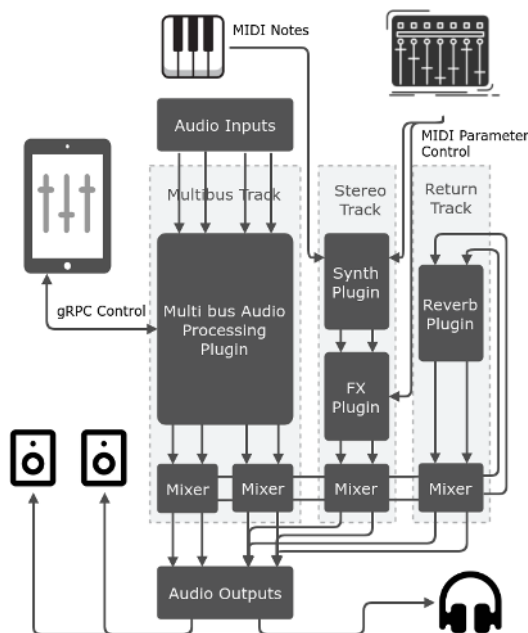


Figure 5.3: Architecture of Sushi: the headless DAW.

5.3 Hot Licks

Each SMI is enabled by packaging the RNN based algorithm presented in Chapter 4 into Hot Licks: a VST plugin that runs on Elk Audio OS. The input to the plugin is obtained from the instrument itself, and the control messages are emitted wirelessly. The plugin has two versions: the symbolic version and the audio version. The core operation of Hot Licks is identical, with the only difference being in the feature extraction as described below,

- The symbolic version obtains the MIDI input from the instrument through one of the USB ports of the Raspberry Pi. The MIDI input is then sampled and used as input to the RNN.
- The audio version of the plugin obtains the audio output of the instrument, which is connected to the HiFiBerry audio input. A stream of audio is obtained and its features are extracted through computing the MFCC. The MFCC spectrogram is then used as the input to the RNN.

The RNN housed within the VST is trained to classify the input audio segment into classes - one for each pattern that is to be detected, along with an extra “not a pattern class”. Upon classification, if a pattern is detected, the OSC message corresponding to that pattern is sent via the local network to trigger the device assigned to it. The plugin is designed using the Jules’

Utility Class Extensions (JUCE)³ framework, and uses the ONNX⁴ ecosystem to port the trained model to the embedded system.

5.3.1 JUCE

JUCE is a cross platform C++ application framework commonly used in audio software development. JUCE simplifies the creation of complex audio applications by providing an extensive set of tools and libraries for real-time audio processing, graphical user interfaces (GUIs), plugin development, and cross-platform deployment. It supports most operating systems including Windows and Linux. JUCE is widely used in prototyping and development, and it has been used in several major commercial software including Ableton Live, Tracktion, and Arturia’s synthesizers.

The extensive feature set offered by JUCE includes support for VST, VST3, Audio Unit, and AAX plugin formats. Its audio processing capabilities are optimized for real time performance. Moreover, it provides a toolkit for GUI development, along with support for protocols such as MIDI and OSC. JUCE has a modular design, which allows developers to only use the components needed. We chose JUCE as the framework to design Hot Licks due to its efficiency, modular structure and comprehensive documentation.

5.3.2 The Matrix of Sound: Hot Licks Architecture

The plugin architecture consists of three main elements apart from the general operating procedures:

1. **Windowing and feature extraction:** the input audio or MIDI stream is used to construct a sub-sequence for analysis, and the corresponding features are extracted.
2. **Pattern Classification:** Upon extracting the sub sequence in the necessary format, the RNN classification is done.
3. **Control Message Transmission:** If the RNN classification detects a pattern, the corresponding OSC message is then sent wirelessly.

Although the first step, windowing and feature extraction differs between the audio and MIDI versions of the plugin, the other two steps are the same between both. The different data extraction techniques for audio and MIDI versions of the plugin are discussed in Section 4.2.3 and Section 4.2.3. The audio version uses a MFCC spectrogram representation, and the MIDI version uses a discrete time series representation of data.

The input is stored in a temporary circular buffer for feature extraction. The buffer continuously processes incoming data in blocks of 512 samples at a standard sampling rate of 44.1 kHz.

³<https://juce.com/>

⁴<https://onnx.ai/>

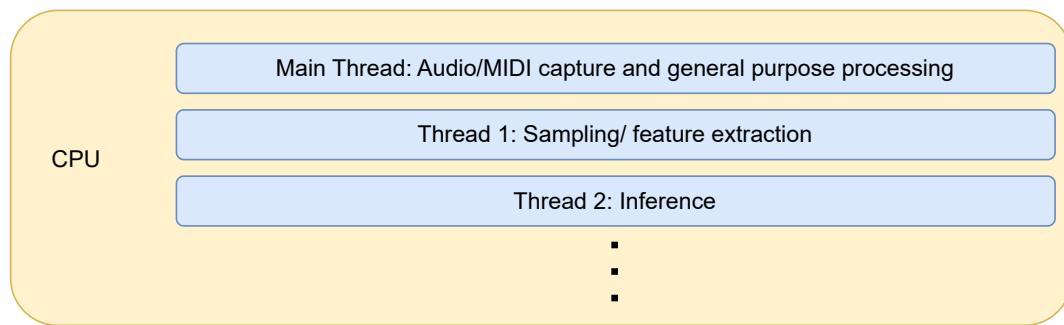


Figure 5.4: Multi threaded approach of Elk Audio OS.

Hot licks uses a multi-threaded approach where the main thread handles real-time audio processing, while background threads continuously perform the computationally intensive tasks of feature extraction and RNN inference. This parallelization ensures minimal latency in the signal path. Figure 5.4 shows an illustration of the multi-threaded approach.

The main processing pipeline is implemented as a `processBlock` method that handles all incoming data in real-time. The system maintains a sequence buffer for audio processing and a temporary buffer for feature extraction. The buffer management is implemented as follows:

```

1 void processBlock(juce::AudioBuffer buffer,
2   juce::MidiBuffer midiMessages) {
3   const int bufferLen = buffer.getNumSamples();
4   const int sequenceBufferLen =
5     ↳ sequenceBuffer.getNumSamples();
6   if (sequenceBufferLen > bufferLen + writePosition) {
7     sequenceBuffer.copyFrom(channel, writePosition,
8       ↳ bufferData, bufferLen);
9   } else {
10     const int bufferRemaining = sequenceBufferLen -
11       ↳ writePosition;
12     sequenceBuffer.copyFrom(channel, writePosition,
13       ↳ bufferData, bufferRemaining);
14     sequenceBuffer.copyFrom(channel, 0, bufferData,
15       ↳ bufferLen - bufferRemaining);
16   }
17 }

```

The feature extraction involves sampling the data for the MIDI version, and constructing the MFCC spectrum for the audio version. The MFCC spectrum is performed using a modified

version of the librosa library adapted for real-time processing.

```
1 // Parameter initialization for MFCC computation
2 int sr = 44100;    // Sample rate
3 int n_fft = 1024;  // FFT window size
4 int n_hop = 1024;  // Hop length
5 std::string window = "hann"; // Window type
6 bool center = false;
7 std::string pad_mode = "constant";
8 float power = 2.f;
9 int n_mel = 20;    // Number of mel bands
10 int fmin = 80;     // Minimum frequency
11 int fmax = 7600;   // Maximum frequency
12 int n_mfcc = 20;   // Number of MFCC coefficients
13 bool norm = true;
14 int type = 2;
15
16 // Get audio data from buffer
17 float* start = tmpBuffer.getWritePointer(0);
18 int size = tmpBuffer.getNumSamples();
19 std::vector my_vector(start, start + size);
20
21 // Compute MFCC features
22 std::vector> mfcc = librosa::Feature::mfcc(
23     my_vector, // Input audio vector
24     sr,        // Sample rate
25     n_fft,     // FFT size
26     n_hop,     // Hop length
27     window,    // Window type
28     center,    // Center flag
29     pad_mode,  // Padding mode
30     power,     // Power for mel spectrograms
31     n_mel,     // Number of mel bands
32     fmin,      // Minimum frequency
33     fmax,      // Maximum frequency
```

```
34     n_mfcc,      // Number of MFCC coefficients
35     norm,        // Normalization flag
36     type         // DCT type
37 );
```

Then the pattern classification is performed using RNN model deployed through ONNX Runtime. The model accepts input features of shape $[1, 215 \times 20, 1]$ and produces output predictions for n distinct pattern classes. The inference is implemented as follows:

```
1 std::array<float, n> onnxModel::inference(
2 const std::vector<float> inputVec) {
3     const std::array<int64_t, n> inputShape = { 1, 215*20, 1 };
4     const std::array<int64_t, 2> outputShape = { 1, n };
5     // ... inference implementation
6 }
```

Hot Licks has demonstrated the ability to identify up to 10 distinct classes of musical patterns. The classification is done using a state machine, where we use a bit to hold the state which represents whether a pattern was detected. The state machine is reset with each new classification. If a pattern is detected, an OSC message is sent to the designated IP address. A simple threshold is used to trigger the state machine as shown below:

```
1 if (results[1] > 0 results[1] > results[2]) {
2     uiChangeBit = 1;
3     _sleep(2000);
4     uiChangeBit = 0;
5 }
```

The system achieved an average latency of approximately 23.2 ms for inference, well within acceptable limits for real-time music applications. The plugin implements several optimizations to maintain real-time performance.

- Lock-free circular buffer implementation for audio data management;
- Background threads for feature extraction and inference;
- Memory management using pre-allocated buffers;
- Single Instruction/Multiple Data (SIMD) optimizations through JUCE's DSP module.

A sample operation of the system is illustrated by Figure 5.5. The instrument output is fed into the system. Upon detecting a pattern, the corresponding OSC message is sent to the controller (in this example, a lighting console), which translates the OSC message into a format that peripheral devices can read (e.g., DMX).

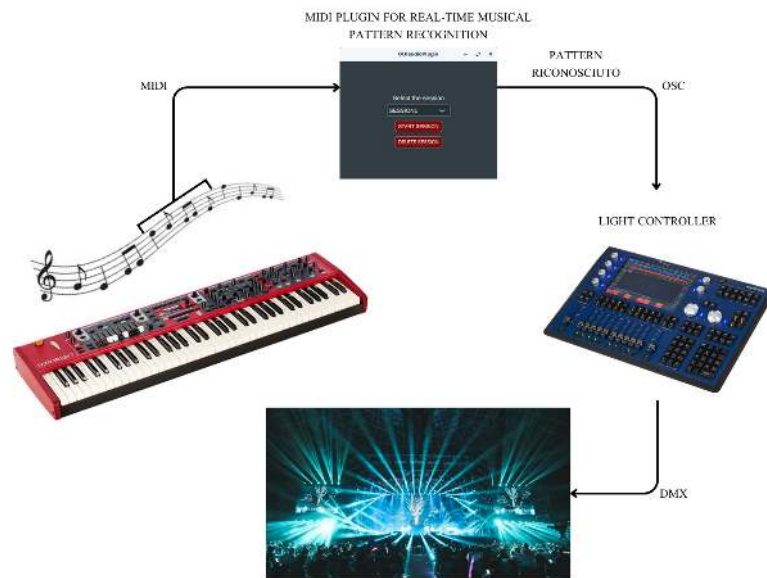


Figure 5.5: A sample application of Hot Licks.

5.4 The Cloud Nexus: Training, Model Updates, and Server Integration

We have developed the system to combine edge computing and cloud services in a hybrid architecture. This approach leverages the local resources for real-time processing, and remote computational resources for labor intensive tasks (i.e., ML model training). The system exemplifies the Digital Signal Processing as a Service (DSPaaS) paradigm, where complex signal processing tasks are distributed between edge devices and cloud resources. The architecture minimizes latency by only performing the inference locally.

The training server is implemented using Flask and TensorFlow, which provides an API endpoint to handle training requests. Figure 5.6 shows a graphical illustration of the model training pipeline. The server receives the ground truth patterns as encoded numerical sequences, which are then decoded and used to create the synthetic training dataset and subsequently train the model. The server employs a Flask based REST API that handles both model training and distribution:

```
1 @app.route("/files")
2 def list_files():
3     files = []
4     for filename in os.listdir(UPLOAD_FOLDER):
```

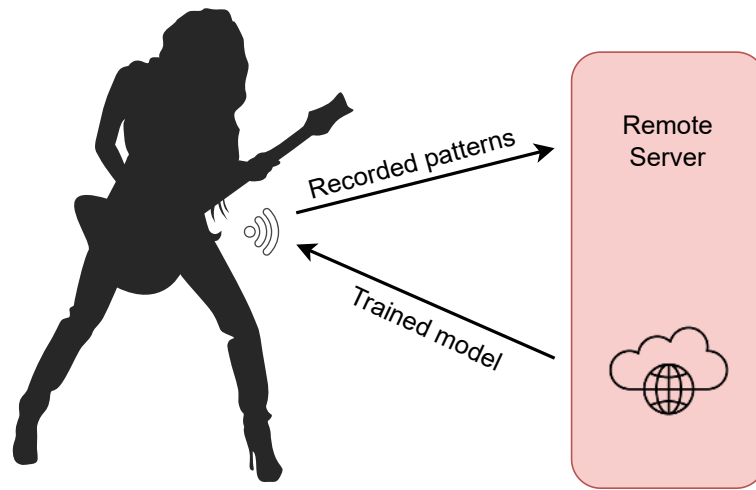


Figure 5.6: Model training pipeline involving cloud computing.

```
5     path = os.path.join(UPLOAD_FOLDER, filename)
6     if os.path.isfile(path):
7         files.append(filename)
8     return json.dumps(files)
9
10 @app.route("/files/<path:path>", methods=['GET'])
11 def get_file(path):
12     return send_from_directory(UPLOAD_FOLDER, path,
        ↪ as_attachment=True)
```

5.5 Hot Licks Controller: The Digital Conductor

The training process of the RNN is described in Chapter 4. The training first involves recording a number of ground truth patterns. As the system runs as a headless plugin, we have implemented an isolated controller which acts as the GUI for Hot Licks. The Hot Licks Controller is also a VST plugin that can be run standalone on a desktop computer.

The recording is done partially on the `processBlock` method, as it is the only method where the input can be accessed. The storing and encoding of the recorded patterns occur in an isolated thread to ensure the multi-threaded operation of the plugin. The communications between Hot Licks and Hot Licks Mapper are done using OSC commands through the local network. Below is an example code snippet for the **UNDO** button on the Hot Licks Controller

5.5. Hot Licks Controller: The Digital Conductor

using the OSC reference class bundled with JUCE:

```
1 void oscMessageReceived(const juce::\ac{osc}Message message) {
2     switch(message[0].getInt32()) {
3         case UNDOBTN: {
4             if(recThread.index>0){
5                 recThread.index--;
6             }
7             juce::String fileName = "/home/mind/hotLicks/"
               ↪ +recThread.sessionName+"/track";
8             fileName+=recThread.index;
9             fileName+="mid";
10            juce::File midiFile(fileName);
11            if(midiFile.deleteFile()){
12                std::cout << fileName << " deleted correctly" <<
               ↪ std::endl;
13            }
14            break;
15        }
16    }
17 }
```

Below is a step by step sequence to the use of Hot Licks Mapper, alongside Hot Licks.

1. When opening the GUI, the user can choose between starting from an existing session or creating a new one from scratch. A session corresponds to a model trained to detect a fixed number of patterns. Each session corresponds to a maximum of 10 patterns that can be recognized simultaneously. A screenshot of initial page is shown by Figure 5.7.



Figure 5.7: Hot Licks Controller: initial page.

2. If creating a new session, the user must enter the session name and the number of patterns they wish to record. A screenshot of the session configuration page is shown by Figure 5.8.

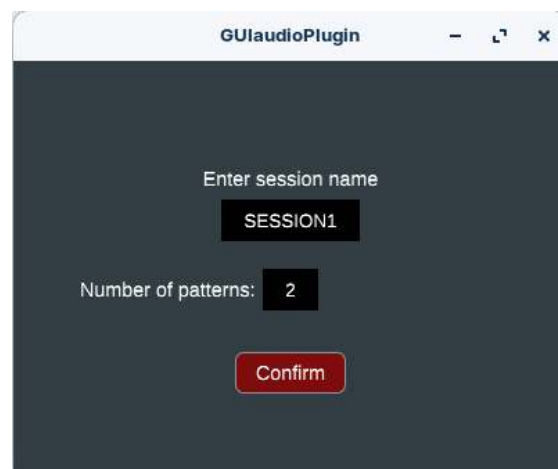


Figure 5.8: Hot Licks Controller: session configuration page.

3. Upon entering the session name and number of patterns, the user can proceed with pattern recording. The corresponding OSC message must be entering alongside each pattern when recording. The OSC messages and the class names will be saved in a local configuration file which will be called at runtime. Figure 5.9 shows a screenshot of the recording page.



Figure 5.9: Hot Licks Controller: recording page.

4. Upon recording all the patterns, the TRAIN MODEL button will send the encoded patterns to the cloud server. Once training is complete, the onnx model is then downloaded to the edge device. Figure 5.10 shows a screenshot of the training page.

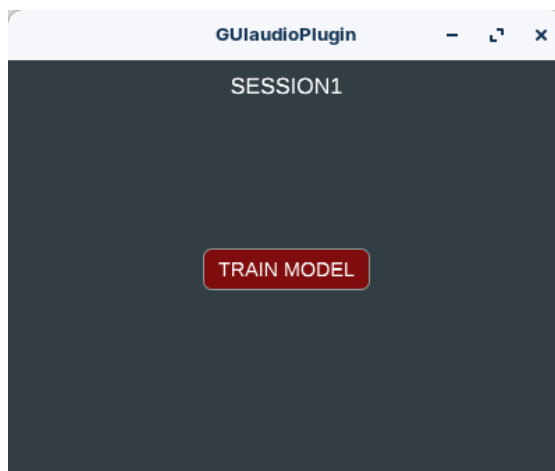


Figure 5.10: Hot Licks Controller: training page.

5. If the user intends to open an existing session, they can browse the saved session and load it to run the pattern detection system. Figure 5.11 shows a screenshot of the session load page.

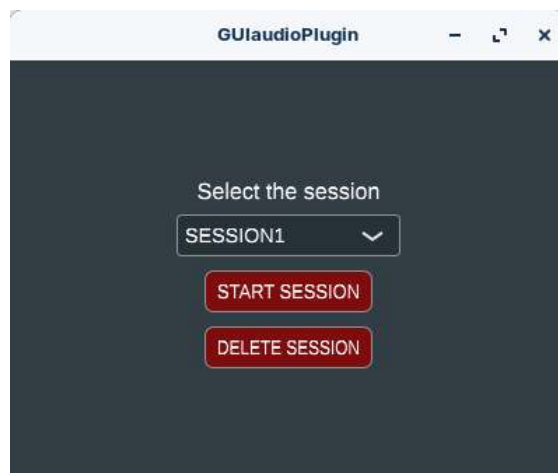


Figure 5.11: Hot Licks Controller: session load page.

5.6 Summary

The development of SMIs represent a significant leap forward in merging traditional musicianship with technology. One of our contributions to this field is Hot Licks: a VST plugin which hosts a real-time pattern detection algorithm. The VST plugin is able to detect the presence of musical patterns, while maintaining an ultra-low latency. The hybrid architecture of the system combines edge computing for real-time processing and cloud computing for resource intensive tasks, demonstrating a practical application of the DSPaaS paradigm.

The system has the ability to work with both audio or MIDI inputs. We also introduce a user friendly GUI, which makes the system accessible to musicians with little to no expertise in using commandline API calls. We have evaluated the pattern detection system for up to 10 patterns. Moreover, the efficient multi threaded approach demonstrate SMIs potential to become tools in live musical performances and interactive creations.

Chapter 6

The User Perspective: Musicians' Interaction with Real-Time Detection

6.1 Musicians and Machines: The Human Factor in Pattern Detection

In this chapter, we present a comprehensive user study conducted to evaluate musicians' perception of SMIs with embedded RTPD. The study was designed to assess the performance as well as the practical applicability of the system in a realistic musical context.

The evaluation framework consisted of three primary components: a pattern composition task, an improvisation task, and a detailed questionnaire. A set of 22 musicians, not involved in the recording phase of the dataset (Chapter 3), were selected to represent a diverse range of musical styles and skill levels, ensuring that the results were generalizable. Specifically, the user study involved 11 keyboard players and 11 guitar players (all Sri Lankan; 9 F, 11 M; aged between 19 and 36 years, mean = 27.28, SD = 4.86). Some musicians were recruited from the University of Visual and Performing Arts in Colombo, Sri Lanka, and some were professional musicians involved in live bands, music production, and music composition. They had varying musical backgrounds which include western classical music, eastern classical music, ethnic music, contemporary rock and pop, jazz, and electronic music. Details of the musicians, including their musical background, instrument, age, and gender are presented in Appendix A.3.

The experiments took place at the University of Visual and Performing Arts Colombo premises, private recording studios, and at homes of the musicians.

6.1.1 Procedure

The musicians were given three simple backing tracks to improvise over. The backing tracks were obtained from royalty free online sources. The tracks were selected such that both keyboard and guitar players could improvise over them. All backing tracks were approximately 2 minutes in length.

- Track 1: A pop style track using the **I-vi-IV-V** chord progression in the key of **F major**. The track consists of drums, bass guitar, rhythm guitar, and a synth pad.
- Track 2: A groovy track using the **vi-II** chord progression in the key of **D major**. The track consists of drums, bass guitar, soft strings, and a rhythm guitar.
- Track 3: A rock style backing track using the Andalusian Cadence in the key of **C major**. The track consists of drums, bass guitar, an acoustic rhythm guitar, and a slightly overdrive rhythm guitar in some sections.

Each musician was asked to compose two musical patterns for each backing track. The musicians were given the artistic freedom to use expressive techniques and note choices that they desired. The composed patterns served as the ground-truth for training the system. Upon composing the patterns for each track, the pattern detection model was trained on a remote server, and the musician was asked to improvise over the track. The musician was instructed to incorporate the two recorded patterns in their improvisation to test the pattern detection system. The accuracy of the system was measured using the number of patterns detected during the improvisation, and each musician was asked to fill a questionnaire to understand their perception of the system. A block diagram outlining the procedure of the user study is presented by Figure 6.1

Pattern Composition Task

Each musician was given the freedom to compose two distinct patterns for each backing track. The two composed patterns reflected the musical background of each musician, as well as their personal playing styles. Guitar players incorporated techniques such as string bending and slides, while keyboard players used techniques such as pitch bending and arpeggio sequences. The duration of patterns ranged from 2 seconds to 5 seconds. It should be noted that all musicians opted to compose shorter patterns due to the nature of the experiment, where they had to remember the two patterns they composed for the subsequent improvisation task. Upon composing and recording the patterns, the synthetic training dataset was created on-the-fly, and the RNN model was trained to be used in the improvisation as elaborated in Section 4.2.3.

Improvisation Task

Upon training the model, each musician was asked to improvise over the backing tracks, using the composed patterns in their improvisation. Similar to the patterns composition phase, each

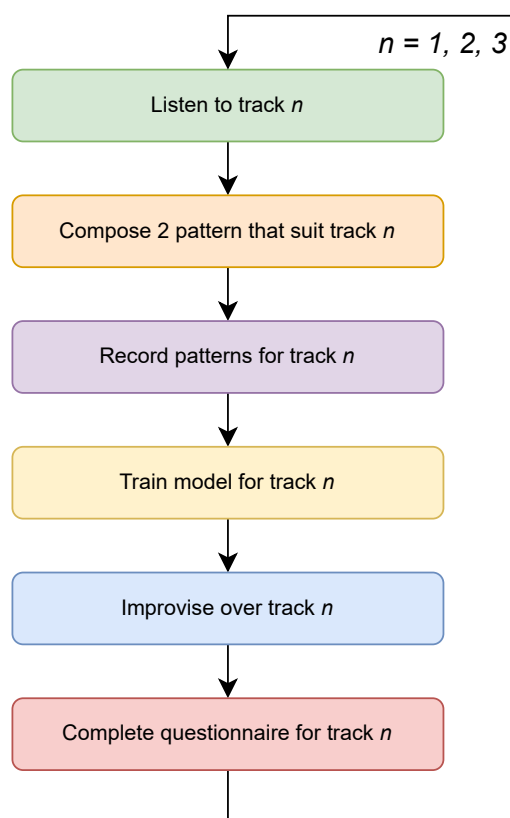


Figure 6.1: A block diagram of the user study procedure.

musician's personal style and musical background was reflected in their improvisation. During each improvisation, the input is routed to the pattern detection algorithm containing VST. The VST plugin was initialized to use the RNN model corresponding to the patterns associated with each track on setup. The user study was set up to display the pattern detection using a graphical application running on a laptop, which changed the screen background color upon the reception of a given pattern - to simulate a stage light or a smoke machine triggered. The decision to simulate the output digitally rather than involving peripherals was taken due to the ease of setting up the system at different locations. The two patterns were assigned to separate colors for easier observation (blue and red) . While the musician was improvising, the number of detections, as shown by the desktop application, was recorded. This data was used to assess the accuracy of the system.

Questionnaire

Participants were asked to fill a questionnaire upon completing each improvisation, which served to gain a qualitative insight on the user experience and the system potential applications. Musicians' willingness to use Artificial Intelligence (AI)-based solutions was also one of the key observations of the questionnaire. Specifically, musicians were asked to rate the following statements on a 11-point Likert scale [not at all, very much]:

Q1: I am satisfied with the behaviour of the system;

Q2: I enjoyed using the system;

Q3: I would use AI tools in my performances;

Q4: I was confident when using the system.

Upon completion of all 3 improvisations, the musicians were asked to rate the following questions on the same scale as before:

Q5: To what extent did you enjoy using the system?

Q6: To what extent would you consider using this system into your performance?

Q7: Would you recommend this to a fellow musician?

Q8: To what extent was the system easy to learn?

Moreover, musicians were asked to answer the following open-ended questions having the purpose of understanding musicians' perception about the system, acquiring feedback on how they would use the system, and obtaining suggestions for improvements:

- In what musical activities would you use this system?
- Please explain any other way you would use this system?

	mean	standard deviation
Detection latency (ms)	14.6	3.8
Transmission latency (ms)	45.0	12.1
Average CPU usage (%)	31.3	1.5
Average RAM usage (MB)	442.6	1.0

Table 6.1: The average latency values for the pattern detection and to emit a control message upon detecting a pattern, along with the average CPU and RAM usage.

- If you had access to this system (where another external output can supplement the music) when composing a piece, will there be any changes in the way you would approach the composition?
- Would you associate specific musical phrases with specific output devices?
- How would you improve the system?
- In your opinion, what is the added value of the system?
- Do you have any other comments?

Recognition accuracy

The accuracy of the system was measured by calculating the number of correct detections. The experimenter kept track of the correct detections while the musician was improvising. Moreover, the improvisations were recorded as audio for later analysis. It is possible to notice that the system showed a high level of average accuracy across musicians with various backgrounds and playing styles. Figure 6.6 illustrates the average precision, recall, and F1 score for the 3 improvisations per musician, and Appendix A.4 shows all the results of the experiment. These results show that some musicians experienced high rates of accuracy, while some musicians had a low accuracy. This may be attributed to the fact that the improvisations done by some musicians were very similar to the patterns they composed, hence rendering false detections. Figure 6.6 also shows the average accuracy, precision and f1 score for the guitar players and the keyboard players through the 3 improvisations separately. It is evident that there is no significant changes in accuracy between the two instruments.

Latency and computational load

Latency is a crucial aspect in any real-time system. For the pattern detection to work in real-time, the pattern must be identified as soon as the musician plays the last note, and the control signal must be transmitted instantaneously. As the pattern detection system runs within an embedded computer, it must have a low computational load. Table 6.1 shows the average latency in detecting a pattern after the last note has been played, the latency in emitting a control message upon discovering a pattern, and the average CPU consumption of the algorithm.

	mean	standard deviation
Precision	0.723	0.169
Recall	0.670	0.222
F1 Score	0.663	0.143

Table 6.2: Average Precision, Recall, and F1 score across all improvisations.

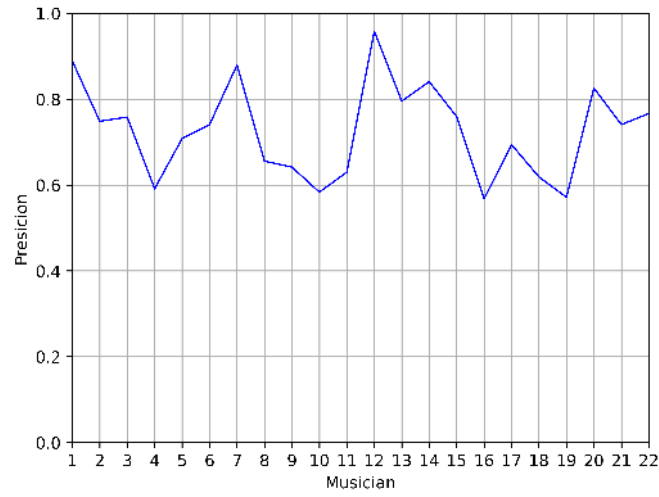


Figure 6.2: Average Precision across the 3 improvisations for each musician.

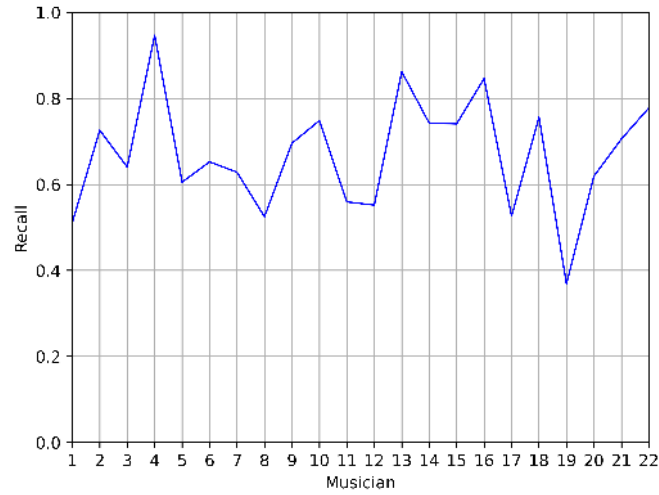


Figure 6.3: Average Recall across the 3 improvisations for each musician.

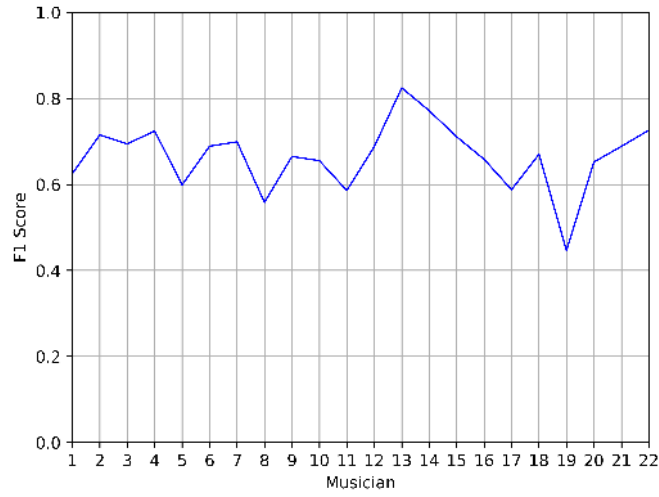


Figure 6.4: Average F1 Score across the 3 improvisations for each musician.

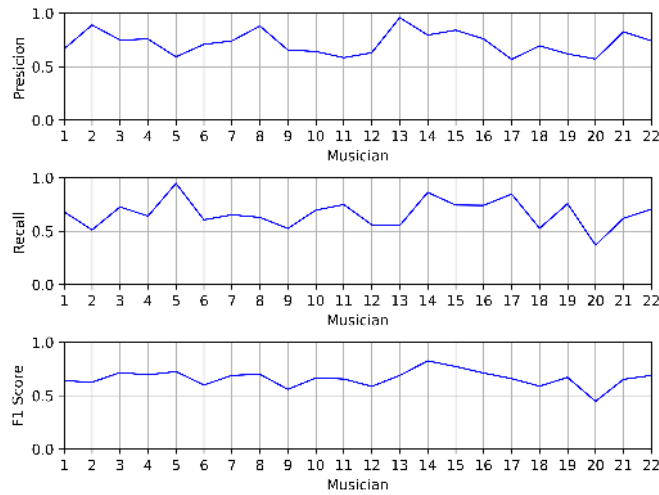


Figure 6.5: Average accuracy, precision, and f1 score across the 3 improvisations for each musician.

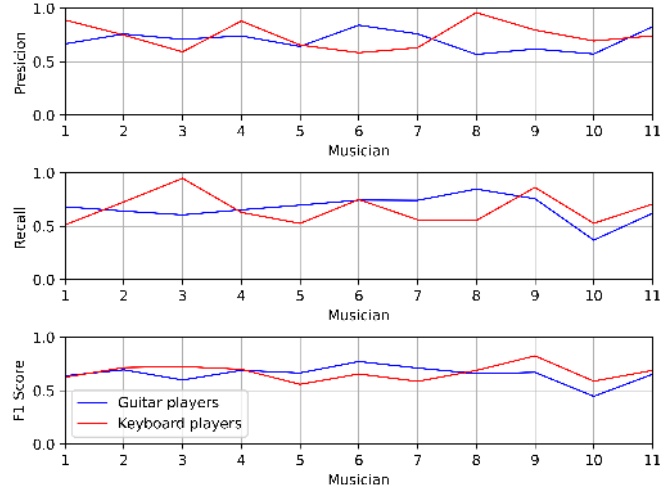


Figure 6.6: Average accuracy, precision, and f1 score across the 3 improvisations, presented separately for the guitarists and the keyboard players.

6.1.2 Questionnaire quantitative Analysis

Figure 6.11 shows the results of the questionnaire after each improvisation. The ratings show that on average musicians enjoyed using the pattern detection system and they were satisfied with its performance. Moreover, the results show that musicians who used this system were likely to use AI-based tools during a musical performance, and that they found it relatively easy to use.

The musicians were asked to complete the same questions after the second and third improvisation. Figure 6.11 also shows the responses after the second and third improvisation respectively. These results show that the initial perception of the system was unchanged.

Figure 6.12 shows the results of questionnaire given to the musicians after completing all 3 improvisations. The results show that musicians were likely to integrate AI-based solutions into their performances, they would recommend AI-based solutions to other musicians, and that the system proposed was easy to learn.

6.1.3 Questionnaire qualitative analysis

The open-ended questions were analyzed with a reflexive thematic analysis [120]. The following themes were identified.

Performance Applications

codes: Solo performances, band performances, DJ sets, improvisation

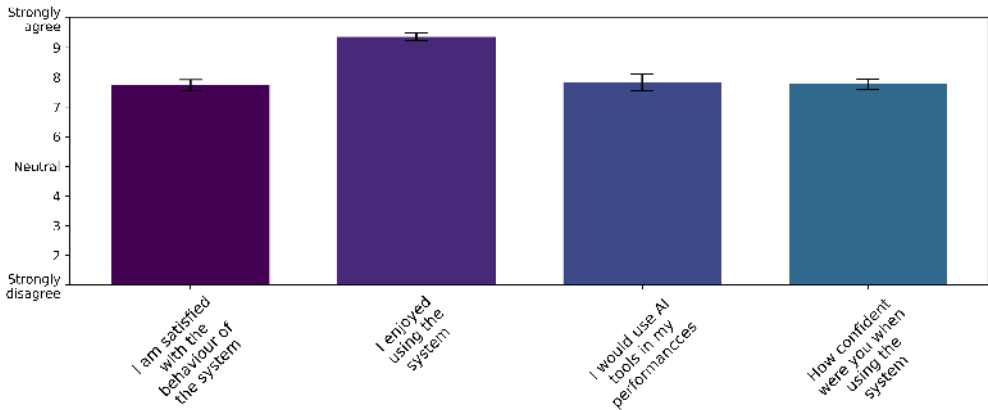


Figure 6.7: Results of the questionnaire after improvisation 1.

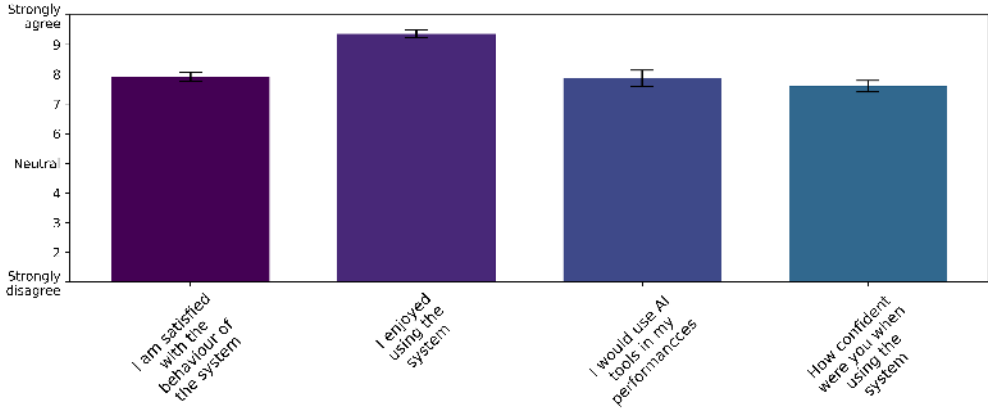


Figure 6.8: Results of the questionnaire after improvisation 2.

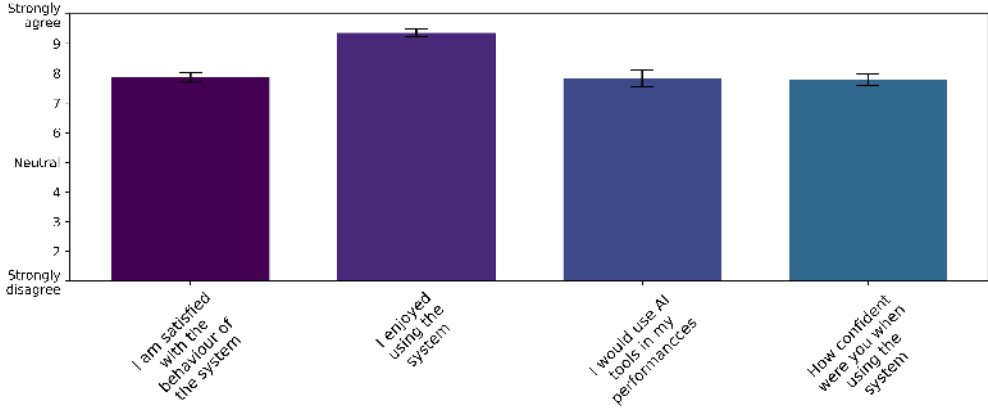


Figure 6.9: Results of the questionnaire after improvisation 3.

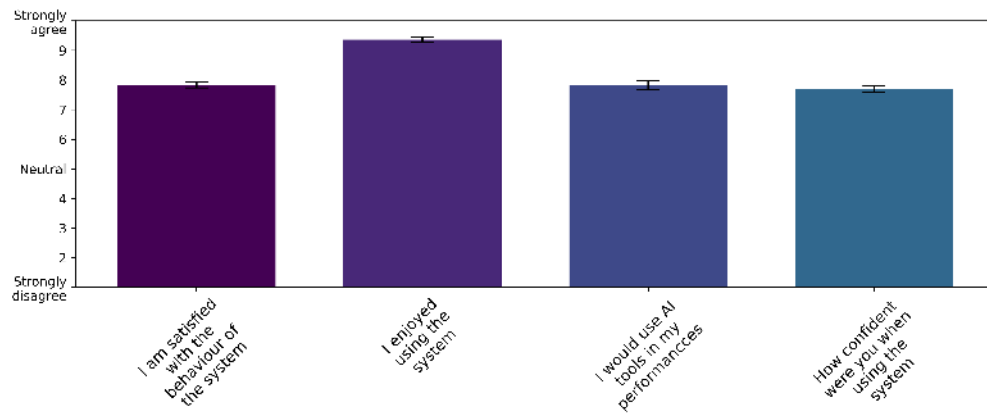


Figure 6.10: Results of the questionnaire after all improvisations.

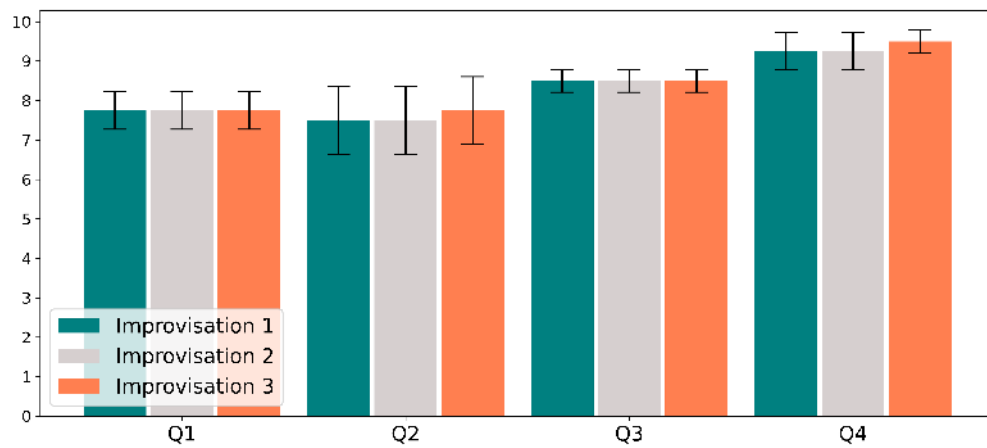


Figure 6.11: Results of the questionnaire after each improvisation.

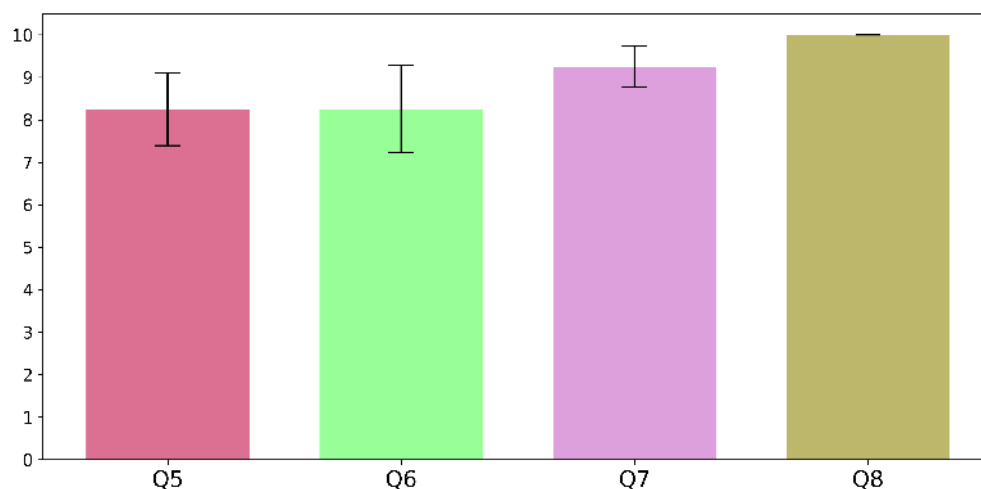


Figure 6.12: Results of the final questionnaire after completing all 3 improvisations.

16 musicians expressed a strong interest in using the system across various performance contexts. Solo performances, and improvisational performances were frequently mentioned, highlighting the potential to supplement their artistic output. The ability a musician would gain to maintain consistency across different performance setups was particularly valued.

“I would use this system in a live improvisation. I would also use this in a solo performance and during teaching activities.”

“I would use this during solo concerts. I can use some lights at lobby performances to improve my solo sets.”

“I would use this during live performances with my band and also while I’m doing session work. It would be great to always have the same output so that I can maintain consistency when I’m playing with different bands or with different sound and light setups.”

“I would use this system when I’m working as a DJ. The playlist is not always consistent and sometimes I change the orders of the songs depending on the crowd interaction at the time. Having something like this will help me to always trigger the lights and smoke machine I associate with certain sections of songs”

Educational Uses

codes: teaching aid, technique recognition, feedback for students, disabled musicians support

The potential of this system as an educational aid was a recurring theme for 9 participants. Those musicians saw it as a valuable tool for teaching, to provide a feedback and enhance

6.1. *Musicians and Machines: The Human Factor in Pattern Detection*

student's learning. The ability to recognize whether the student has played a phrase correctly, and the ability to identify the extent of the correctness.

“It would be beneficial to see how similar the phrase played is to the original pattern, this would assist in teaching activities.”

“Can you use the system to track finger positions? It would be great if I get an indication if the student uses the correct hand position to play scales - this would be really useful as teaching aid.”

“I would love to use this as a teaching aid for disabled musicians. It would be great if blind students can get a haptic feedback based on how accurate their playing is to what actually should have been played.”

Creative Impact

codes: changes in composition style, more motivated to use repeating themes

Musicians believed that access to the system could positively influence their creative processes, particularly in composition. They felt that the system would motivate them to use more repeating themes and iconic phrases. The ability to associate specific musical patterns with visual or equipment effects was seen as a creative opportunity that could enhance their performances and compositions.

“Yes there would be changes in the way I compose if I have access to this. I would do my compositions with the phrases that are specifically associated with lights”

“If I had access to this when I am composing, I would be more motivated to use repeating themes in my compositions”

Chapter 7

Symphony of the Senses: Crafting the multi-sensory Experience

One of the central tenets of the IoMusT paradigm is that of enabling interconnections between Musical Things and, as a consequence, multidirectional interactions between their users. This spurs the emergence of novel performance ecosystems of humans and machines [121, 122]. In recent years, various systems have been proposed to interconnect heterogeneous musical devices over co-located or remote networks [123–129]. The interconnection capabilities offered by the IoMusT paradigm have opened up a whole new realm of creative possibilities for composers and performers, allowing them to create connected, interactive and multi-sensory performances [51, 130–134]. However, to date only a handful of studies are available on such a topic and the potential of IoMusT-based performance ecosystems is still largely unexplored. In particular, the ability of IoMusT technologies to support multi-sensory experiences for audiences has been scarcely addressed thus far.

An example of this system would be a guitarist performing on a smart guitar equipped with pattern detection capabilities. The smart guitar analyzes the presence of the predefined patterns as the musician plays different chords and melodies, and triggers external devices such as lighting systems, smoke machines, visual projections, haptic devices or even other musical instruments. In the context of a band, the different SMIs utilized by the performers would wirelessly control different devices. Such performance ecosystems of humans and machines aims at enhancing the overall performance by synchronizing different elements and creating a multi-sensory experience for the audience.

To validate our vision of novel performance formats enabled by multi-sensory IoMusT technologies, we created a performance ecosystem involving different SMIs as well as devices generating visual, haptic and olfactory stimuli. We then validated our concept and the underlying technology by means of an actual live concert which featured a custom composition.



Figure 7.1: The musicians playing the SMIs. The lights and the smoke machine were triggered by the SMIs.

7.1 Ecosystem description

In the following, we describe the main components of the developed IoMusT ecosystem. A picture of the ecosystem during an actual performance is illustrated by Figure 7.1, which shows the three SMIs, along with musical things that are controlled by the instruments.

7.1.1 Smart Musical Instruments

We developed three SMIs:

- a smart electric guitar;
- a smart keyboard;
- a smart drumkit.

Such smart instruments were developed by integrating an electric guitar, a keyboard and an electronic drum with embedded computing devices running the RNN-based algorithm described in Section 4.2.3. Figure 7.1 shows the three musicians with the SMIs. Each algorithm was trained on a dataset corresponding to the musical instrument. The keyboard and the drum provided MIDI by default. For the electric guitar, we used a pitch to MIDI converter (Fishman TriplePlay¹, see Figure 7.3).

The chosen embedded computing device is a Raspberry Pi 4 with the Hi-Fi Berry audio shield. The RNN-based algorithm is hosted within a VST plugin which runs on Elk Audio OS [118]. The input to the VST plugin can either be audio or MIDI directly from the instrument. Upon

¹<https://www.fishman.com/tripleplay/>

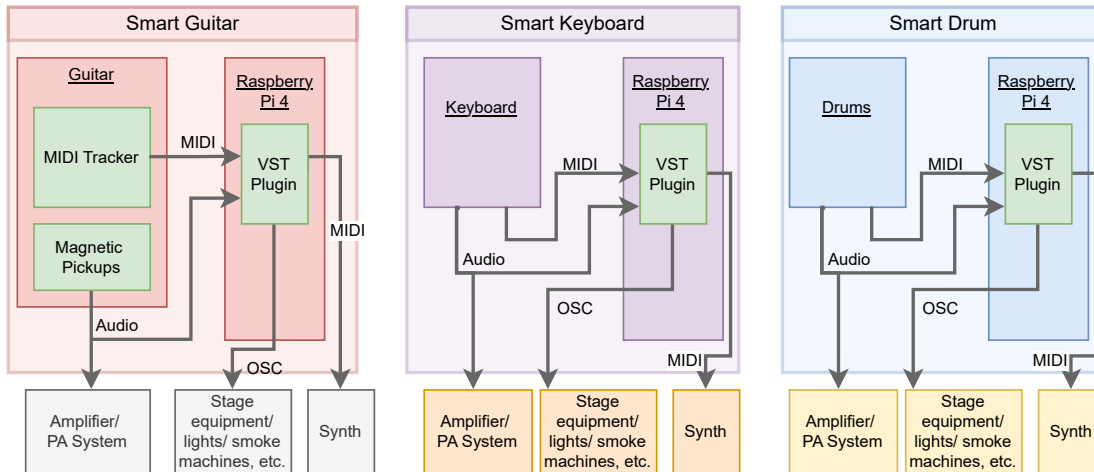


Figure 7.2: Block diagram of the smart electric guitar, the smart keyboard, and the smart drum.



Figure 7.3: The developed smart electric guitar, to which the Raspberry Pi with the HiFi Berry shield, the battery and the MIDI converter is attached.

detecting a pattern, an OSC command is wirelessly sent to the central mapping system, which can be assigned to control any peripheral device. A block diagram of the SMIs' operation is shown by Figure 7.2. Figure 7.3 shows a picture of one of the developed instruments - the smart guitar with the attached embedded system running the pattern detection algorithm².

7.1.2 Musical Things for the audience

We used a heterogeneous set of musical things for the audience which were controlled by the SMIs. As illustrated in Figure 7.4 and Figure 7.1, these were:

²<https://hotlicksvst.github.io/>



Figure 7.4: Musical things controlled by the SMIs. The lights, smoke machine, MRs headset and haptic vest are shown in the image.

- Smoke machine (Sagitter Color Smoke 900 FC); the smoke machine was connected through a DMX interface, which controlled the smoke and the RGB lights;
- Stage lights (Stairville Stage TRI LED and Varytec Hero Wash 715): the stage lights were connected via DMX. Each pattern was assigned a different color;
- Two haptic vests (Skinetic by Actronika): each of these was connected to a raspberry PI 3B, which run a Pure Data application implementing a set of vibration patterns;
- Two MR headsets (Oculus Quest 3 by Meta); for this purpose we created a standalone Unity 3D application that superimposed different virtual objects on the real scene seen via the headset pass-through feature.

7.1.3 Router, central server and tablet

We created a local area network via a WiFi router (TL-WR902AC by TP Link), which interconnected all the musical devices in the ecosystem. The WiFi was configured according to the guidelines reported in [135] to optimize the latency. A central server (an Alienware M15 laptop) was used to run the Hot Licks Mapper described in Section 7.1.4. This was dynamically controlled by a web app which can be accessed remotely via a tablet connected to the same network. The web app enables composers to associate a pattern of a given SMI to a stimulus provided by the musical things for the audience. Figure 7.5 shows a block diagram of the ecosystem, illustrating its main components and the data flow.

7.1.4 Hot Licks Mapper

The Hot Licks Mapper is an application designed to facilitate the dynamic mapping between SMIs with RTPD capability and external peripherals, to allow an interactive, multi-sensory experience

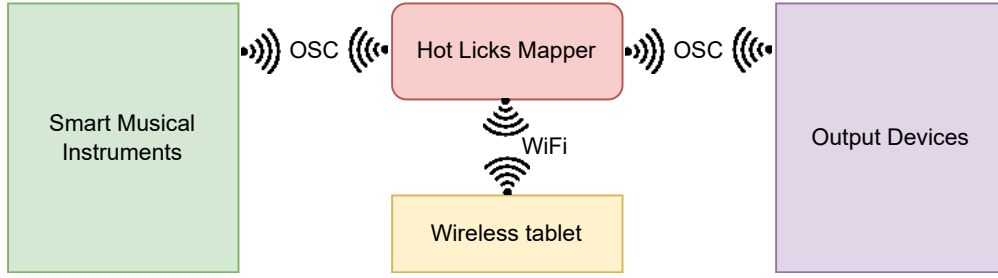


Figure 7.5: A block diagram of the operation of the mapping system.

for the audience. Hot Licks Mapper serves as a visual and interactive tool that allows a composer or a performer to connect multiple patterns detected through different SMIs to multiple stimuli created by different output devices. Specifically, the application enables a composer to define 4 types of mappings:

1. **one-to-one:** one pattern of a smart instrument to one stimulus on a single type musical thing for the audience;
2. **one-to-many:** one pattern of a smart instrument to many stimuli on a single or multiple types of musical things for the audience;
3. **many-to-one:** a single pattern for each of multiple smart instruments to one stimulus on a single type of musical things for the audience);
4. **many-to-many:** multiple patterns played on multiple smart instruments to many stimuli on a single or multiple types of musical things for the audience.

As elaborated in Chapter 5, the developed SMIs are able to output OSC commands upon detection of various patterns. Consider N SMIs, each equipped with its own model trained to identify n patterns. Thus, the inputs to the Hot Licks Mapper will be $pattern_1^1, pattern_1^2, \dots, pattern_N^{n-1}, pattern_N^n$, where $pattern_N^n$ is the n^{th} pattern produced by the N^{th} SMI.

Similarly, the outputs of the hot licks mapper can be specified as $stimulus_1^1, stimulus_1^2, \dots, stimulus_M^{m-1}, stimulus_M^m$, where $stimulus_M^m$ is the m^{th} stimulus produced by the M^{th} output device. Output devices may be stage lights, smoke machines, VR headsets, projectors, mobile phones, wearable haptic devices and so forth. Different stimuli may be colors and strobe patterns of lights, smoke patterns, odors and colors, projections on screens, different vibration patterns on wearable haptic devices, etc.

To enable a truly immersive multi-sensory experience, all communications are wireless as illustrated by Figure 7.5, where each SMI emits OSC commands via WiFi upon detecting a pattern. The User can create a one-to-one, or one-to-many routing between patterns and stimuli. Moreover, the user can dynamically create or remove connections in real-time with a handheld device, which communicates wireless via WiFi. The OSC commands are routed to each output device wirelessly via WiFi.

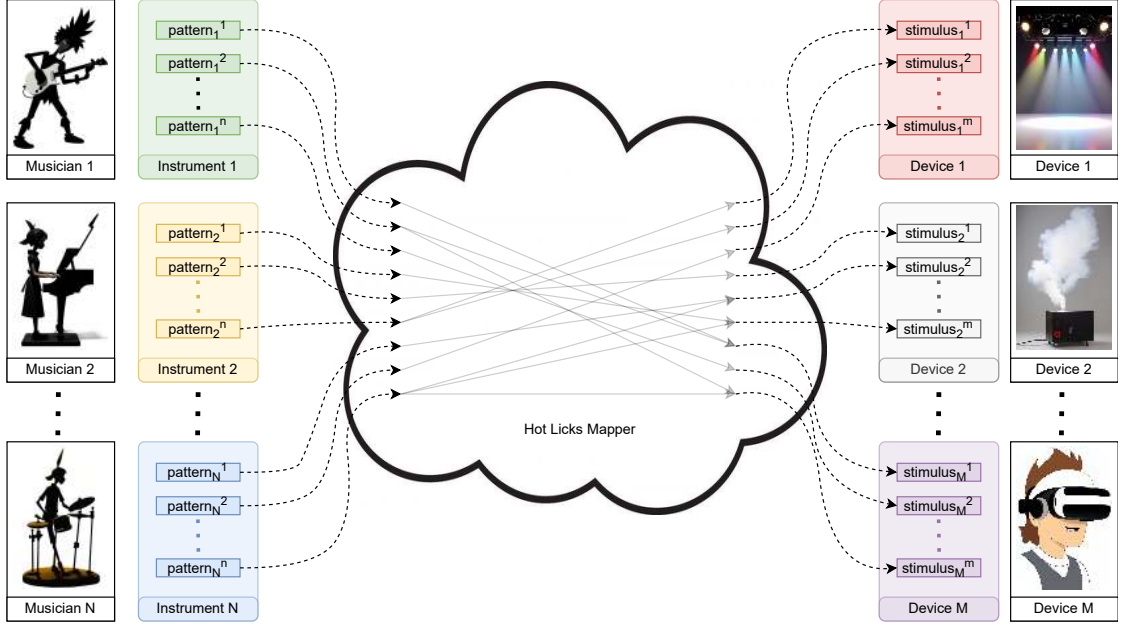


Figure 7.6: A diagram of the functionality of the proposed mapping system.

Figure 7.6 shows a schematic of the Hot Licks Mapper, where n patterns from N SMIs are routed to m stimuli across M output devices. It also shows some arbitrary connections between the patterns and stimuli.

The user interacts with the hot licks mapper through a web interface where inputs and outputs are represented as blocks. These blocks can be visually connected with lines (arrows), symbolizing the flow data from inputs to outputs. Each input block can be associated with a SMI, and each output block can be associated with an output device, allowing for complex audio routing and manipulation scenarios. Figure 7.7 shows the initial routing we used during the evaluation presented by Section 7.2 routing, where the user has routed several inputs with one or multiple output devices. The user is also able to create new mappings or adjust existing ones by drawing arrows from patterns to stimuli, facilitating a customizable routing environment in real-time.

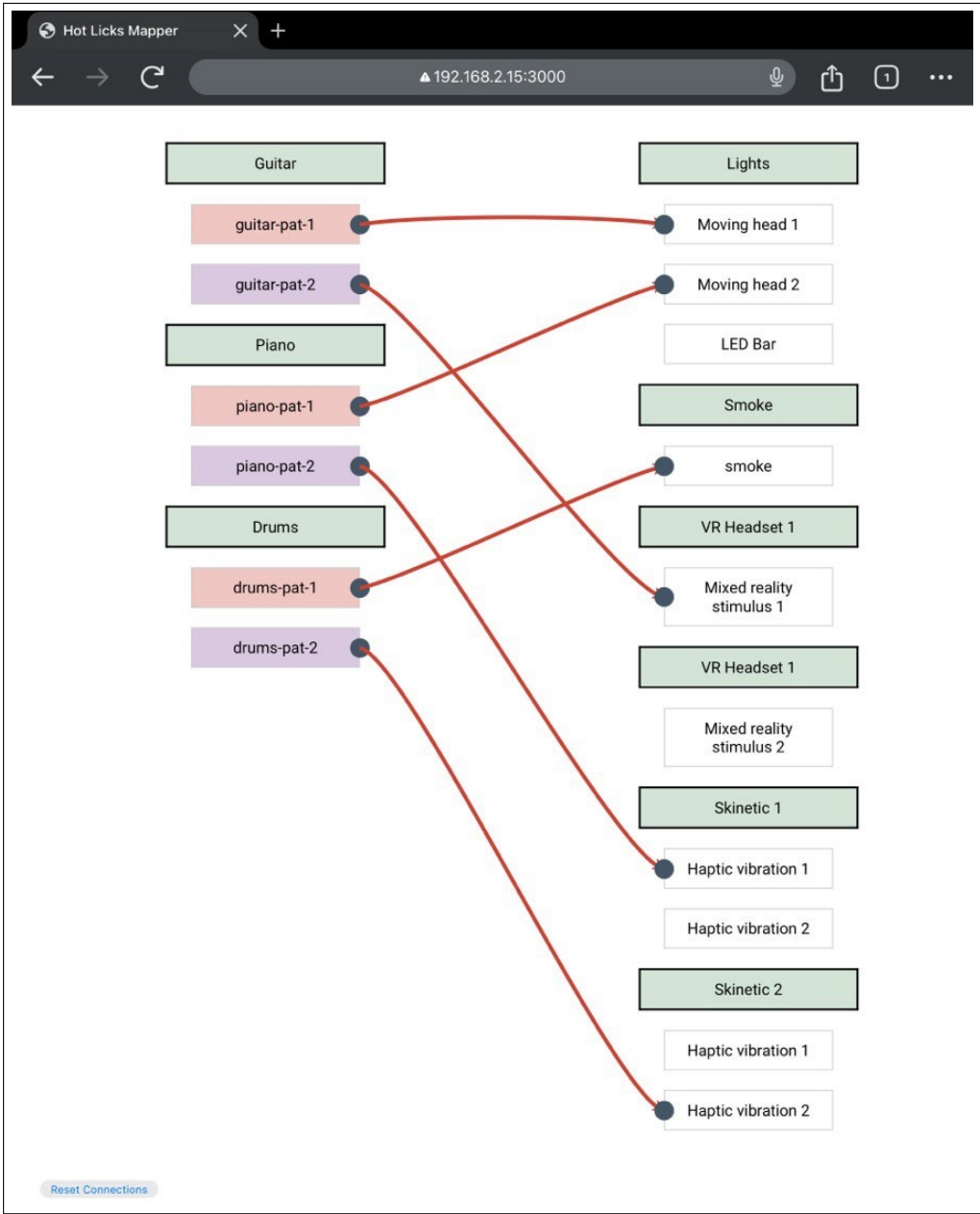


Figure 7.7: A screenshot of the initial mapping between the patterns and musical things, as described in Table 7.1.

7.2 Evaluation

The main aim of the evaluation was the assessment of the composition process and the technical validation of the ecosystem components, rather than the proper assessment of the experience of the audience members. We tested our concept of multi-sensory performers-audience interactions enabled by SMIs and their pattern recognition system by performing an actual concert. The participants involved in the evaluation (composer, performers, audience members) were asked to provide informed consent. The procedure, approved by the local ethics committee, was in accordance with the 1964 Helsinki Declaration.

The concert featured a composition of a composer who was recruited for the test. The composer (male, 33 years old, 15 years of professional musical experience) holds a degree from the Conservatory of Trento in Electronic Music Composition and extensive expertise with classical, electric and electronic music performances. The composition was conceived for a trio (drum, electric guitar and keyboard). Accordingly, three performers were recruited from the local musicians community. Upon the request of the composer, the performers were asked to define a given number of patterns with some given characteristics.

The composer was provided with a set of pre-defined set of stimuli developed by the authors, which could be generated by each of the Musical Things involved in the ecosystem. He was then asked to use the Hot Licks Mapper to create the mappings between musical patterns and multi-sensory stimuli. A total of 6 musical patterns (2 for the electric guitar, 2 for the keyboard, and 2 for the drums), along with 8 different stimuli (2 moving heads, 1 LED bar, 1 smoke machine, 2 haptic vests, and 2 MR headsets) were made available to the composer. The created composition comprised the one-to-one mappings reported in Table 7.1. These were just the starting point, as during the performance the composer could dynamically change the mappings through the Hot Licks Mapper running on the tablet.

The performance was attended by 9 audience members (3 females, 6 males) recruited from the University of Trento. Nevertheless, only two of them wore the haptic vest and the MR headset due to the availability of the equipment. The other participants could enjoy only the performance visual stimuli coming from the lights, but not the haptic sensations provided by the haptic vests nor the virtual objects delivered by the headsets.

Table 7.1: The initial mappings between musical patterns and multi-sensory stimuli used during the first part of the performance.

Smart Instrument	Pattern	Musical Thing	Stimulus
Smart Guitar	pat 1	Moving head 1	The light rotates along the x and y axes to illuminate the guitarist and turns blue for a duration of 3000 ms, then switches off and rotates to the original position;
Smart Guitar	pat 2	MR headset	Virtual objects spawning on the headset. The users can use their hands to move the objects around the space;
Smart Keyboard	pat 1	Moving Head 2	The light rotates along the x and y axes to illuminate the keyboard player and turns red for a duration of 3000 ms, then switches off and rotates to the original position;
Smart Keyboard	pat 2	Haptic vests	intermittent vibration pulses of a sawtooth lasting 250 ms and separated by 250 ms, for a duration of 5 seconds;
Smart Drum	pat 1	Smoke machine	The smoke machine turns on for 50 ms, along with the red, green and blue LED lights in it to add a colored effect to the smoke;
Smart Drum	pat 2	Haptic vests	intermittent vibration pulses of an oscillator lasting 500 ms and separated by 500 ms, for a duration of 5 seconds.

7.3 Discussion

On the one hand, the conducted evaluation aimed at assessing the technical ability of the ecosystem to support the performers-audience interaction from the technological standpoint. By performing a real live concert, we showed that the developed technological ecosystem was effective in detecting the patterns in real-time on each SMI and routing them to the different stimuli of the various musical things involved, based on the defined mappings.

On the other hand, the evaluation focused on the assessment of the Hot Licks Mapper in supporting the composition process. Overall the feedback of the composer was very positive about the novelty and the concept. Based to the composer's comments, the Hot Licks Mapper was proven to be intuitive to use and, ultimately, successful in allowing the user to create a new artistic format based on the created performance ecosystem. The test showed that the composed ecosystem can support novel aesthetic narratives driven by performers' expressions over the course of live performances. In particular, the real-time control by SMIs of MR headsets, haptic vests and scented smoke machines represents a radically new avenue for interactive live performances, where multi-sensory virtual content can blend with the musical and embodied expressions performed on stage.

The composer initially assigned each musical pattern to different stimuli as elaborated in Table 7.1. During the performance, the composer was able to remove existing mappings, as well as create new ones. The discussion with the composer upon concluding the performance revealed that there is potential in such a system for live performances, where different stimuli may be assigned to musical phrases, based on the interactions of the performers and the audience, as well as the emotions associated with the music at any given time.

Notably, by working with the composer, we noticed how the complexity of the proposed IoMusT-based multi-sensory performances imposes a constant and tight interaction between the artist providing the creative ideas and the technologists accomplishing technical implementations for their realization. For instance, in the case of the proposed ecosystem there was the need of implementing visualizations for the MR headsets or vibration patterns for the haptic vests, beyond the knowledge of the wireless network configurations. These aspects necessitate ad hoc expertise with a variety of dedicated software that rarely is known in detail by a single person, being him/her a composer or a creative technologist.

Moreover, from the composition process the need of conducting fundamental research on how to best associate the musical information with the sensory content clearly emerged. There is the need of defining guidelines that can support composers in their creative processes. As advocated in [136], a potential avenue is represented by crossmodal correspondences [137], i.e., people's tendency of matching information presented in one sensory modality to that presented in another.

Overall, the performance was well received by participants who attended the concert. All of them provided enthusiastic comments about the novelty of the experience and its potentialities for transforming the live music world.

We conducted an in-depth interview with the audience members who wore the MR headsets and the haptic vests to understand the multi-sensory stimuli they experienced during the concert. These two audience members reported to have enjoyed the experience of the haptic sensations and virtual elements that were triggered by the SMIs. They reported the ability to distinguish that different stimuli were triggered by different musical phrases. Specifically, the objects spawned on the MR headset could be maneuvered by the user by using their hands. We could observe the users moving around the performance venue, trying to move the virtual objects that spawned upon a musician playing the pattern associated with it. Moreover, the two audience members also experienced haptic vibrations that were produced when the musician would play the pattern assigned to it. Importantly, they reported feeling more engaged with the music upon the inclusion of multiple sensory stimuli into the performance.

The audience members that did not wear the haptic vests and the MR headset reported the engagement with the visual stimuli provided through the stage lights and the smoke machine. Also the relationship between the musical content and the triggering of the controlled devices was clear to them. Consistently, all participants, whether wearing MR headsets and haptic vests or not, agreed that the multi-sensory stimuli associated with musical phrases created an engaging experience.

In a different vein, our study focused only on the unidirectional communication from performers' to audience members' devices. Nevertheless, the proposed IoMusT communication infrastructure can easily support a bidirectional communication where also the audience can send messages to the performers. The latter could be triggered by some gestural patterns, tracked by wearable motion sensors. This avenue falls in the remit of the field of technology-mediated audience participation [138].

Our study presents the notable limitation that we tested the developed technology and the ecosystem with only one composer and with only one composition. A more thorough study with more composers and multiple compositions, as well as the adoption of a longitudinal study as a methodological approach, are in need to reveal more details about the strengths and weaknesses of our concept, and let the actual needs and desires of composers, performers, and audience members emerge. However, we believe that despite being preliminary, our results concretely showed the potential of the IoMusT paradigm in enabling unprecedented forms of musical expression. This calls for further interdisciplinary research at the confluence of the technological and artistic domains.

Chapter 8

Grand Finale: The Second Performance and Audience Impact

8.1 Introduction

The intersection of technology and music has been a fertile ground for innovation, enabling for the emergence of paradigms like the IoMusT. These new technologies allow musicians and researchers to explore novel forms of performances and artistic interactions. Multi-sensory performances, as discussed in Chapter 7 are an ideal candidate to explore the fusion of SMIs and immersive performances. We explore a novel form a multi-sensory performance that integrate auditory, visual, and tactile stimuli to create immersive experiences where sound becomes a gateway to broader sensory experiences.

This chapter presents an in-depth account of a two-day long multi-sensory performance designed as the final step of the research presented by this thesis. The experiment involved SMIs equipped with advanced RTPDs and peripheral devices - termed *Musical Things* - to create a performance integrating sound, light, vibration, and MR.

The motivation for this experiment was the research question: *How does the coherence of multi-sensory elements influence audience and musicians engagement and perception during a live musical performance?*. This question was explored through two performances, each featuring a live band playing 3 SMIs:

- Smart electric guitar with embedded audio RTPDs.
- Smart keyboard guitar with embedded symbolic RTPDs.
- Smart drum kit with embedded symbolic RTPDs.

Audience members experienced two versions of each song. The first version with Musical Things being triggered by the pattern detection system, and the second version where Musical Things

were triggered randomly. This allowed for a controlled comparison of structured and unstructured sensory experiences.

8.2 Experiment Design

The experiment was designed to explore the relationship between music and different sensory elements. The experiment was carried out as two performances over two days. Each performance followed the same structure to ensure consistency. Moreover, each performance involved different musicians and different audiences to ensure a proper comparative analysis.

8.2.1 Musical Repertoire and Performance Structure

The performance featured two original songs created to complement the technical setup. Several factors had to be met when creating the compositions to ensure that they are suited to evaluate multi-sensory performances and RTPDs:

- The compositions must be simple and easy to understand. As the performance includes multiple types of sensory stimuli, careful attention must be paid to not subject the audience to a *cognitive overload*;
- The compositions must have easily recognizable patterns to evaluate the pattern detection system;
- The composition must allow all musicians to contribute equally with their instrument. All solos, improvisations and melodic segments must be distributed equally among all instruments;
- Careful attention must be paid to the locations of musical patterns within the composition, so as they are spread equally.

Keeping this in mind, we created the two compositions in collaboration with a composer (male, age: 33 years, professional music experience) holding a diploma from Trinity College London in Rock and Pop guitar, and having extensive experience with live performances of various musical styles. The two songs can be described as follows:

1. Song 1 is inspired by grunge and electronic music with several melodic elements are played over a 4 chord loop. The song employed 1 pattern each for the guitar, piano, and drums.
2. Song 2 is an upbeat track, with inspiration from 80's pop music. This song was a high energy piece with an upbeat tempo and vibrant instrumentation. Similar to song 1, this too had 1 pattern each for the guitar, piano, and drums.

The patterns were strategically distributed within each song to minimize confusion (i.e., if the guitar pattern and they keyboard pattern were immediately adjacent to each other, it would cause some confusion to the listener). Each pattern repeated multiple times within the song

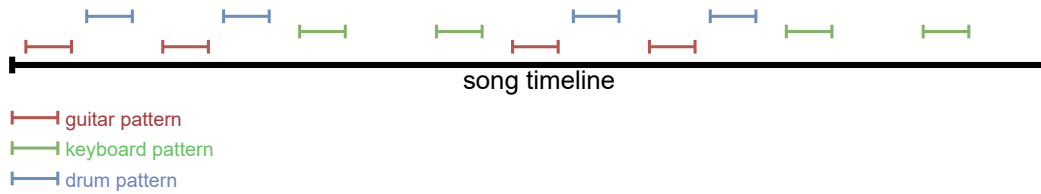


Figure 8.1: How each instrument's patterns can be spread within the timeline of the song.

to allow the system to trigger sensory effects. Figure 8.1 shows an example where each instrument's patterns are spread within the timeline of the song.

8.2.2 Smart Musical Instruments and Pattern Detection

The core of the experiment revolved around the SMIs outfitted with embedded computing devices with the integrated pattern detection algorithm. Each SMI had an RNN model trained to detect the pattern corresponding to the instrument. Figure 7.3 shows the smart electric guitar with the embedded system attached, and Figure 8.2 shows the smart keyboard and the smart drum kit, where the embedded computing devices are highlighted in red.

Each instrument was connected to a local area network wirelessly as shown by Figure 8.3. Upon detection of a pattern, the corresponding OSC message was then transmitted to a central server, which decoded the OSC message and triggered the relevant musical thing. The audio output of each instrument was sent to a central audio mixer and then to stereo loudspeakers.

8.2.3 Musical Things

Musical things refer to the peripheral devices synchronized to the performance. We use Musical Things to create multi-sensory stimuli. Beyond the sound from the SMIs that create an auditory stimulus, the lights create visual stimuli, the smoke creates a visual and somewhat of an olfactory stimuli, the MR headsets create a visual stimulus, and the mobile phone vibrations create a haptic stimuli. Below are the Musical Things used in the performance.

- **Moving Head Lights:** Two *Varytec Hero Wash 715 HEX LED* lights capable of dynamic color changes, rotations, and beam adjustments;
- **LED Bar:** A *Stairville Stage TRI LED*, with 4 individually controllable RGB lights;
- **Smoke Machine:** *AF-180 LED Fogger* to enhance dramatic moments;
- **VR Headsets:** Ten *Meta Quest 3* headsets were used, displaying interactive 3D graphics that was triggered by the patterns;
- **Android Smartphones:** Each audience member held a smartphone that provided tactile feedback through vibrations triggered by the pattern detection system.

8.2. Experiment Design

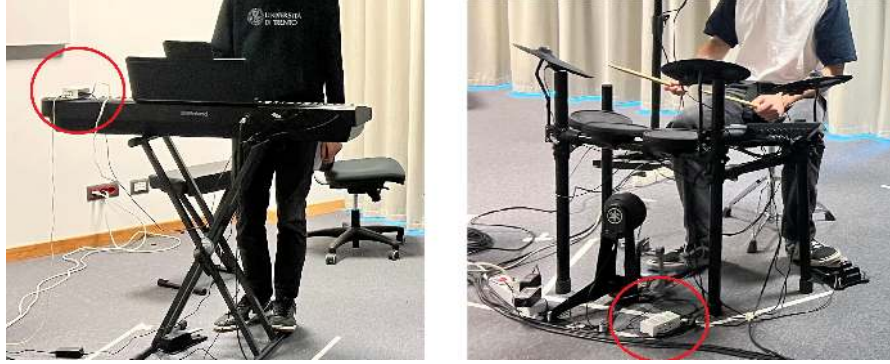


Figure 8.2: The smart keyboard and the smart drum kit, with the indication of the respective embedded systems.

Figure 8.3 illustrate the connections between the SMIs and Musical Things. Solid lines indicate wired connections, and dotted lines indicate wireless connections. The hot licks mapper was responsible for routing all messages to Musical Things. The mappings between the patterns and Musical Things are shown in Table 8.1. The integration of visual, auditory, and haptic elements created a dynamic interplay between sound and sensory effects.

Table 8.1: The mappings between musical patterns and multi-sensory stimuli used during the performance.

Smart Instru- ment	Musical Thing	Stimulus
Smart guitar	Moving head 1	The light rotates along the x and y axes to illuminate the guitarist and turns blue for a duration of 3000 ms, then switches off and rotates to the original position.
	Moving head 2	The light rotates along the x and y axes to illuminate the guitarist and turns blue for a duration of 3000 ms, then switches off and rotates to the original position.
	LED bar	The light illuminates blue for a duration of 3000 ms, then switches off.
	MR headsets	Virtual objects spawning on the headset. The users can use their hands to move the objects around the space.
	vibrations	no vibrations
	smoke	no smoke

Smart keyboard	Moving head 1	The light rotates along the x and y axes to illuminate the keyboard player and turns green for a duration of 3000 ms, then switches off and rotates to the original position.
	Moving head 2	The light rotates along the x and y axes to illuminate the guitarist and turns green for a duration of 3000 ms, then switches off and rotates to the original position.
	LED bar	The light illuminates green for a duration of 3000 ms, then switches off.
	MR headsets	Virtual objects spawning on the headset. The users can use their hands to move the objects around the space.
	vibrations	no vibrations
	smoke	no smoke
Smart drum kit	Moving head 1	The light rotates along the x and y axes to illuminate the drummer and turns red for a duration of 3000 ms, then switches off and rotates to the original position.
	Moving head 2	The light rotates along the x and y axes to illuminate the drummer and turns red for a duration of 3000 ms, then switches off and rotates to the original position.
	LED bar	The light illuminates red for a duration of 3000 ms, then switches off.
	MR headsets	no 3d graphics
	vibrations	The android phones emit a series of pulses of 200ms, with an interval of 100ms.
	smoke	The smoke machine produces smoke for 1500ms.

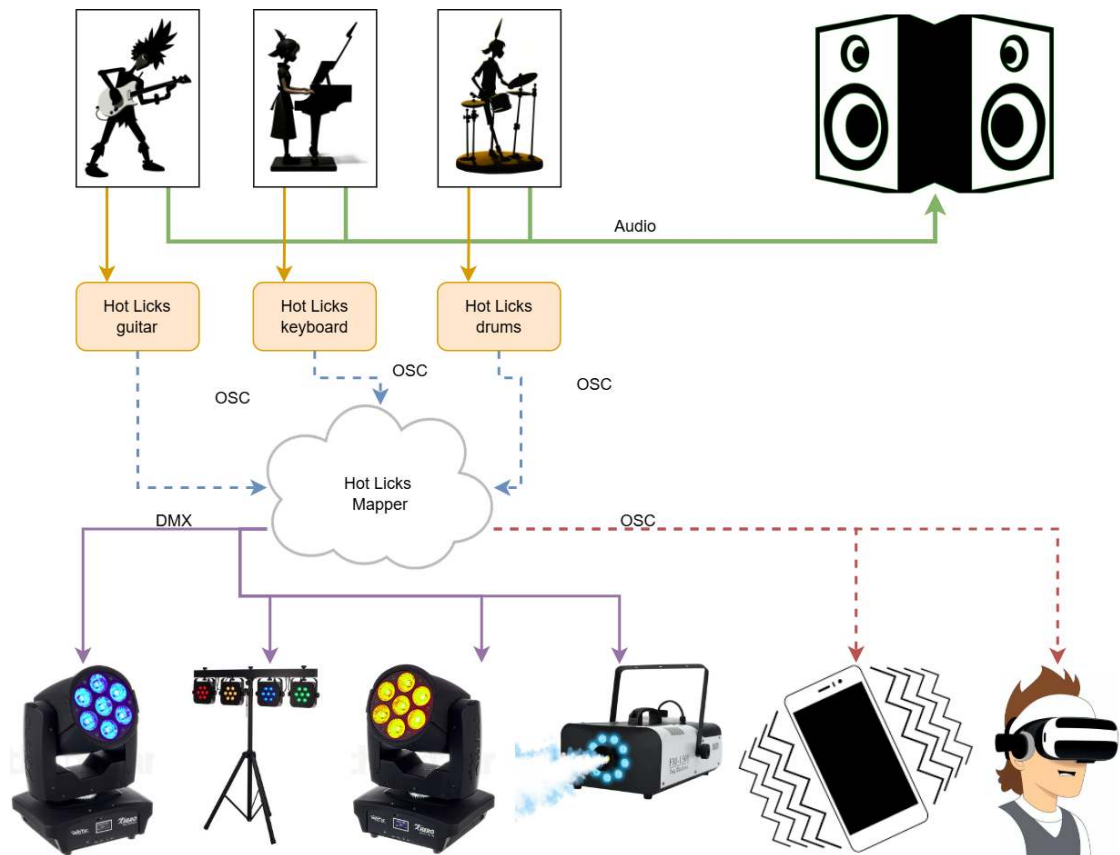


Figure 8.3: The connections between SMIs and Musical Things.

8.3 Audience Setup

Each performance consisted of 10 audience members. Each member was equipped with a Meta Quest 3 MR headset, and an android phone connected to the local network. Figure 8.4 shows an audience member wearing the MR headset and holding an android phone. The position of the audience, performers, and Musical Things in the room shown by Figure 8.5. The MR headset displayed a pass-through view with multi colored spheres spawning on the view upon receiving a OSC trigger message. Figure 8.6 shows a screen capture from the MR headset where the spheres are present.

The audience were standing, facing the band. They could see the musicians, lights, as well as the other audience members through the pass-through view of the MR headset. The two performances had a total of 20 audience members (3 female, 17 male), 10 in each performance. They were aged between 22 and 31 years (mean = 27.65, standard deviation = 3.23). The audience had varying musical expertise, with 5 audience members being professional musicians. They also had a diverse experience with VR headsets (the audience reported a mean familiarity with VR of 4.72 with a standard deviation of 3.46, on a 0-to-10 point Likert scale). This diversity yielded a accurate and generalized set of answers to the questionnaires provided.



Figure 8.4: The audience of the performance, wearing the MR headsets and holding the mobile phones.

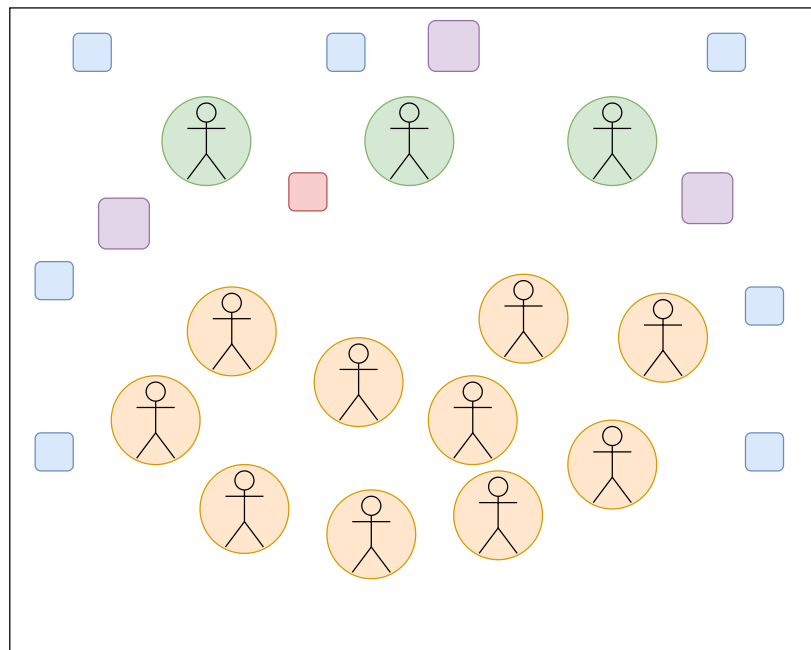


Figure 8.5: A diagram of the experiment setting. The performers are shown by the green circles, the audience members are shown by the orange circles. The purple boxes indicate stage lights, the red box indicates the smoke machine, and the blue boxes indicate the position of the loudspeakers.



Figure 8.6: Screenshot of the MR headset view.

8.4 Multi-sensory Performances

The two experimental sessions were held in two different days, involving two different trios and diverse audience members. Each of the two experimental sessions began with a brief introduction to the nature of the performance. We instructed the audience members on how to connect to the local network and open the web-based application to receive OSC commands that triggered the vibrations. They were then instructed to wear the VR headset.

Each experiment involved two music pieces with two versions of each piece being performed, for a total of 4 performances:

- **Version A:** In this version, the Musical Things were triggered by the RTPD system. This created a coherent and synchronized relationship between the music and sensory effects.
- **Version B:** In this version, the Musical Things were manually triggered at random times during the performance. This provided a contrasting experience, which allowed the audience to have a clear distinction between the two versions.

After each performance, the audience members were asked to complete a questionnaire. The questionnaire consisted of a series of the following statements to be rated on a 11-point Likert scale [0 = not at all, 10 = very much]:

Q1: To what extent did you enjoy the performance experience?

Q2: How much did the Lights make you feel involved in the performance?

Q3: How much did the Lights make you feel distracted from the performance?

Q4: How coherent were the Lights with the music during the performance?

Q5: How much did the Vibrations make you feel involved in the performance?

Q6: How much did the Vibrations make you feel distracted from the performance?

Q7: How coherent were the Vibrations with the music during the performance?

Q8: How much did the 3D Graphics in the headset make you feel involved in the performance?

Q9: How much did the 3D Graphics in the headset make you feel distracted from the performance?

Q10: How consistent was the 3D Graphics in the headset from the performance?

Moreover, they were asked to express their preference via the following question:

Q11: Did you prefer the first performance or the second performance?

The questionnaire was designed to obtain an understanding of the audience's experience of each performance. Upon completing all performances, the audience members were asked to fill a post-experiment questionnaire involving the following open-ended questions:

Q1: What was the most memorable experience for you during the performance? And why?

Q2: How clear was the relationship between what you heard and what you saw or felt?

The musicians involved in the performance were given a similar questionnaire used for our earlier user study presented in Section 6.1.1. They were asked to rate the following statements on a 11-point Likert scale [0 = not at all, 10 = very much]:

Q1: I am satisfied with the behavior of the system;

Q2: I enjoyed using the system;

Q3: I would use AI tools in my performances;

Q4: I was confident when using the system.

Q5: To what extent would you consider using this system into your performance?

Q6: Would you recommend this to a fellow musician?

The musicians were also asked to answer the following open-ended questions to understand their perception about the system, acquiring feedback on how they would use the system, and obtaining suggestions for improvements:

- In what musical activities would you use this system?
- Please explain any other way you would use this system?
- If you had access to this system (where another external output can supplement the music) when composing a piece, will there be any changes in the way you would approach the composition?
- Would you associate specific musical phrases with specific output devices?
- How would you improve the system?
- In your opinion, what is the added value of the system?
- Do you have any other comments?

Upon completing the experiment, we conducted informal interviews with 3 composers to understand the potential of such a system, along with multi-sensory performances from the perspective of a music composer. We interviewed 3 composers (2 male, 1 female; aged 29, 33, and 34 years; with at least 8 years of professional music experience). The musicians were asked to respond to the question - *“How would a RTPD system and multi-sensory performances change the way you approach music composition”*.

The framework for the evaluation of DMIs presented by [139] identifies multiple stakeholders - performers, audience members, composers, and designers. Our evaluation considers 3 of these stakeholders. Qualitative and quantitative feedback was obtained from the audience and the musicians involved in the multi-sensory performance, and qualitative feedback was obtained from composers. Instrument designers were excluded as they are not applicable in our use case.

8.5 Results and Discussion

8.5.1 Quantitative Analysis - Audience

Overall, the concert performance was well-received. Participants expressed enthusiasm about the novelty of the experience. From the questionnaire responses, we calculated the Involvement, Distraction, and Coherence for Lighting effects, Haptic feedback, and VR Visual elements. Some of the key findings include:

- Lighting Effects: Participants found the lighting effects to be highly involving (mean rating 7.29) and coherent with the music (mean 7.0), with minimal distraction (mean 2.41).
- Haptic Feedback: The vibrations were less recieved, with moderate involvement (mean 3.76) and coherence (mean 3.76)
- VR Visual Elements: The VR graphics provided a mixed experience, with moderate to high involvement (mean 4.88) but notable distraction (mean 5.35)

Figure 8.7 shows the average involvement, distraction, and coherence for Musical Things triggered with pattern detection vs. Musical Things triggered randomly. We can observe that lighting effects were the most engaging and least distracting for both performances. Vibrations showed a promising engagement score. 3D graphics in the MR headset showed mixed reactions, with higher distraction scores compared to lights and vibrations. These findings are confirmed through a Wilcoxon Signed-Rank Test, which revealed that, between the performance with Musical Things being triggered with RTPD and Musical Things being triggered randomly:

- There is no significant difference in the *Enjoyment* ($p > .05$);
- There is a slight, but not significant difference in the *Lights Involvement* ($p > .05$);
- There is a slight, but not significant difference in the *Lights Distraction* ($p > .05$);
- There is a significant difference in the in *Lights Coherence* ($p < .05$);
- There is no significant difference in the in *Vibrations Involvement* ($p > .05$);
- There is no significant difference in the in *Vibrations Distraction* ($p > .05$);
- There is s significant difference in the in *Vibrations Coherence* ($p < .05$);
- There is no significant difference in the in *3D Graphics Involvement* ($p > .05$);
- There is no significant difference in the *3D Graphics Distraction* ($p > .05$);
- There is a slight, but not significant difference in the *3D Graphics Coherence* ($p > .05$).

Figure 8.8 shows the participants preference between the performance with RTPD-based triggering and random triggering. These results show that that the audience has no significant preference over the RTPDs over the random version. However, the statistical analysis revealed that when the RTPD system was used, the Musical Things were perceived to be more coherent and less distracting. This statement is supported by Figure 8.9, which shows that the participants with a high experience in MR headsets reported a higher coherence for the 3D Graphics during performance with RTPD-based triggering as opposed to that with random triggering.

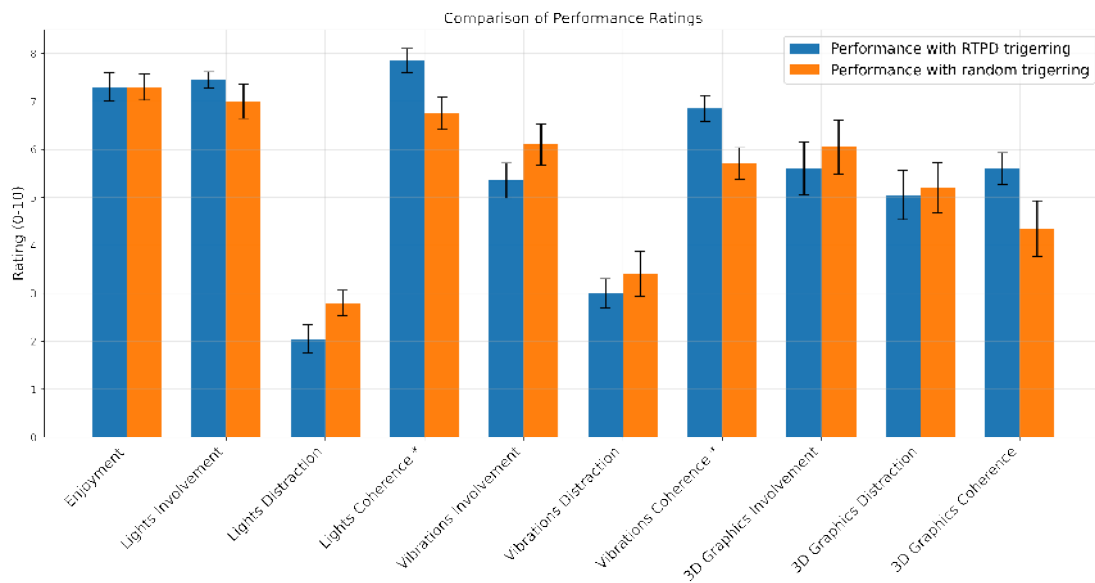


Figure 8.7: Mean and standard error for the two performances.

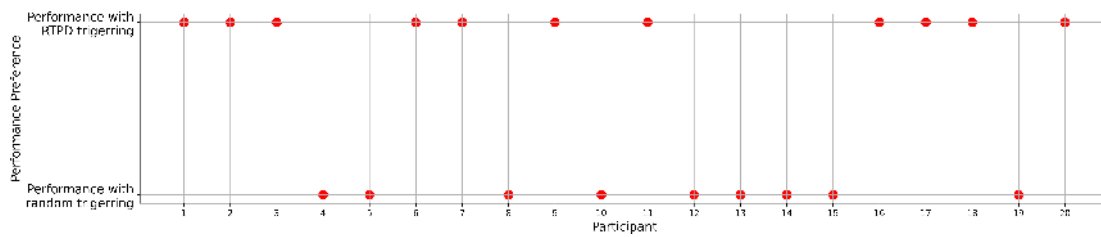


Figure 8.8: Participants' preference between the RTPD and random triggered performances.

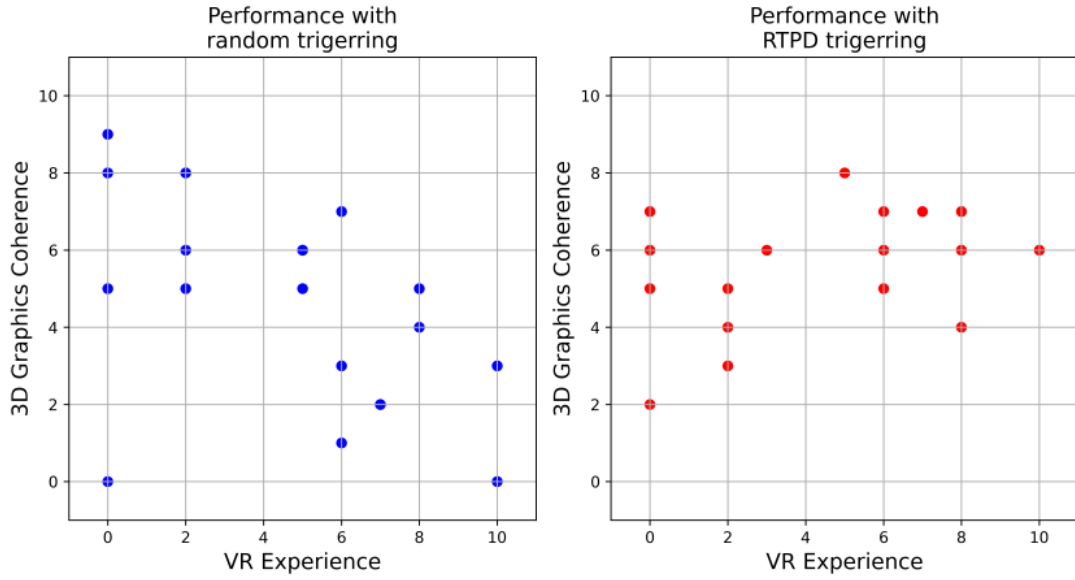


Figure 8.9: 3D Graphics coherence measured against participants experience with VR headsets.

8.5.2 Quantitative Analysis - Musicians

Upon completing the performance, each musician was asked to complete a questionnaire to assess their perception of using the system, and also identify their interest in involving AI-based solutions into their performances. Figure 8.10 show the results of quantitative questionnaire given to the musicians.

The quantitative results show that musicians expressed a high satisfaction in the behavior of the system. Moreover, they expressed enjoyment in using the system and a strong interest in using AI-based tools in their performances. The musicians also expressed a high interest in incorporating the RTPD system into their existing setups, and they expressed willingness to recommend this system to fellow musicians.

8.5.3 Qualitative Analysis - Audience

The results of the open-ended questions were analyzed using a reflexive thematic analysis [120], and two main themes were identified.

Multi-sensory Integration

Multi-sensory integration emerged as a critical theme to highlight how the participants experienced the coherence between the music and various sensory modalities involved. The quantitative data shows varying levels of coherence across different modalities. Lights showed the

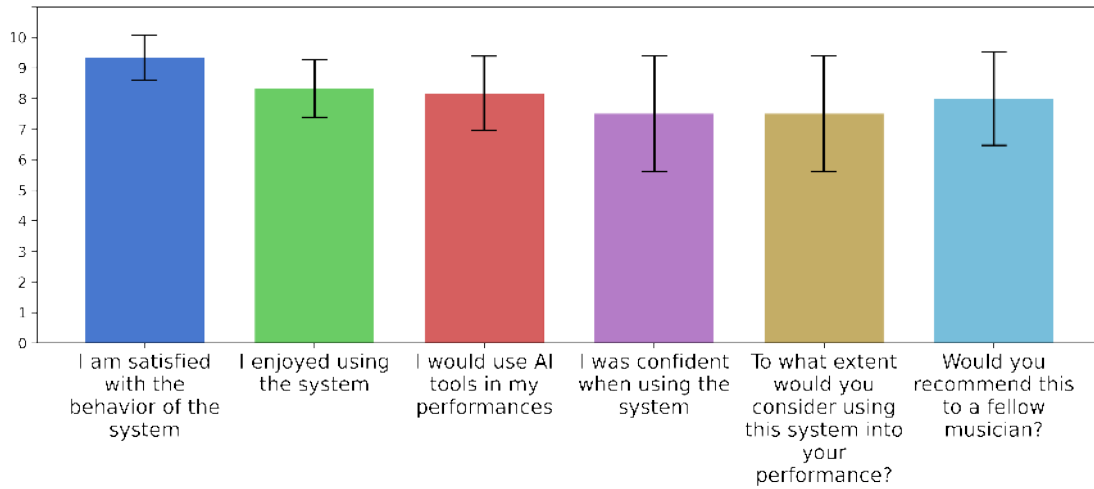


Figure 8.10: Results of the questionnaire given to the musicians.

highest coherence in comparison with VR elements. This disparity is highlighted in participants' experiences with some participants expressing perfect synchronization while others experienced a disconnect. Examples of quotes from participants are:

"There was a very clear relationship between the music and the vibrations in the first performance."

"in the last performance there were several moment with good lights, smoke and the virtual ball actually came in sync with the music."

"In some music, there was a clear relationship between what I heard and what I saw and felt."

"The haptic feedback and the lights were clear between the performance and what I saw and felt."

"Feeling the vibrations with the kick drum was a very enjoyable experience."

Social Interaction

The social interaction was identified as an unexpected theme that emerged from the shared experience of using this novel system in group setting. The presence of other participants and their interactions with the virtual objects proved to be an additional level of enjoyment.

Moreover, several statements reflected shared experiences suggesting that the social aspect of the performance enhanced the experience. Examples of participants' quotes are:

"Watching others interact with things that are not there is very funny"

“Most of the audience were trying to touch the balls. It was very engaging to see the others play with the balls in their headsets.”

“It was interesting to see the combination of the real atmosphere and virtual reality. Seeing everyone with a headset and playing with nonexistent things, while I am playing with the balls in my headset was very cool.”

8.5.4 Qualitative Analysis - Musicians

The musicians’ responses to the open ended questions were analyzed using a reflexive thematic analysis [120]. The responses were fitting with the themes identified in our earlier user study conducted at the University of Visual and Performing Arts, Sri Lanka (Section 6.1.3).

Performance Applications

The musicians expressed an interest in using the system in live performances. Solo performances and improvisations were some of the potential use cases mentioned. Some quotes from the musicians are below.

“I would use this system during a live performance, specifically during a section where I would improvise. I can use some licks to involve lights as a part of my playing.”

“I would use this during live performances with my band. it would be great to see how every instrument can be used to trigger different light and smoke effects.”

Educational Uses

Some of the musicians involved in the experiment were also involved in teaching activities. They expressed interest in adapting the RTPD system as a teaching tool. Specifically, the potential of using the classification result as a measure to detect the accuracy in playing.

“If I can use the output of this system to trigger a red light and a green light, I could use this as a teaching tool. When the student plays a phrase with a specified accuracy, I can use the green light to signal it. If the playing is not accurate enough, the red light will be lit. Students can use this tool by themselves to practice correct technique and phrasing.”

Creative Impact

Musicians mentioned the possibility of using this system in their composition and creative process. They expressed an interest in integrating the RTPD system with commercial software for music production.

“Do you think I can use this with a DAW software? It would be a great tool when producing music. I can use this to spur my creativity in the studio, and also while improvising.”

8.5.5 Qualitative Analysis - Composers

Upon conducting a reflexive thematic analysis [120] of the interviews done with composers, we identified the following themes in their responses.

Compositional Transformation

RTPD can allow composers to expand their toolkit beyond those of traditional music. They can incorporate visual, tactile, and 3D components into their musical compositions. This allows for the creation of performances focused on multidimensionality. Below are some of the quotes from composers pertaining to this theme:

“This pattern detection system lets me compose not just with notes, but also lights, smoke, vibrations, and 3D elements. Imagine when the guitarist plays a pattern and a set of lights are triggered - its like a visual echo of the music!”

“The use of VR headsets can open up completely new compositions. I can have 3D objects on the headset that respond to the detected patterns. I can move the music to a 3-dimensional space. Access to technologies like this will completely transform how I produce music.”

“The pattern detection and the multi-sensory technologies will allow me to think of a composition as designing an environment. The music, lights, haptics, smoke and anything else can be a part in that environment.”

“With this technology, I can integrate audio and visual elements, which will enhance the immersive experience of the audience. This can radically change the way I approach music making.”

Interactive control of peripherals

The technology allows performers will novel forms of control and expressiveness. Musicians can trigger various stimuli in real-time, based on their playing. This will allow for the creation of dynamic and responsive performances.

“The real time pattern detection will let me create compositions where lights, smoke, vibrations, and anything else can respond to the performance. It’s like having an intelligent collaborator on stage.”

“I can create compositions where I leave space for the musician to play a pattern. The musician can decide, at the moment, what output device should be triggered, and then use the corresponding pattern in the performance.”

Multi-sensory Orchestration

RTPD will facilitate for multiple sensory elements to be synchronized, which will allow composers to create complex, interlaced experiences across different sensory modalities. This orchestration of visual, auditory, and haptic sensations can create a rich and immersive environment for the audience. Some quotes from musicians supporting this theme is:

“This system would be amazing to use! Imagine that the drummer is playing a pattern on a loop. I can create amazing rhythms with vibrations that the audience can feel, while they are listening to the music.”

“Imagine this scenario: the pianist plays a pattern, I can use it to trigger a light, then a vibration, then something on the headset, and then smoke. It will be like a multi-sensory delay effect!”

8.6 Conclusion

This chapter presented the design, execution and evaluation of multi-sensory performances involving SMIs to trigger connected *Musical Things* for the audience. We explored the interplay between auditory, haptic, and visual stimuli and we discussed how the coherence of these elements impact audience and musicians engagement. We provided a controlled framework by a comparison between a pattern detection enabled operation and a random operation to evaluate the effects of coherence.

The results showed that lighting effects were deemed as the most coherent with a high engagement and minimal distraction. Haptic feedback showed a moderate level of engagement. The virtual elements had mixed results, where some participants experiencing a high involvement and some experiencing a high distraction. These findings show the importance of balancing sensory stimuli to avoid a cognitive overload while maximizing the audience engagement.

Finally, the qualitative analysis revealed several themes: Multi-sensory Integration and Social Interaction for the audience; Performance Applications, Educational Uses, and Creative Impact for the musicians; and Compositional Transformation, Interactive control of peripherals, and Multi-sensory Orchestration for the composers. While the former highlighted the critical role of coherence in creating engaging multi-sensory experiences, the latter showcased the influence of shared social dynamics in enhancing enjoyment. These insights not only validate the potential of SMIs coupled with multi-sensory technologies for creating novel art formats, but also provide a roadmap for future designs that consider both technological integration and audience interaction in performance settings.

Chapter 9

Echoes of Progress: Reflections and Future Directions

9.1 Research Contributions

The work presented by this doctoral research contributed to the fields of RTPD and SMIs. This research addressed crucial gaps in both the understanding and the practical implementation of RTPD systems based on SMIs. This work successfully developed and validated a novel approach for embedding pattern detection capabilities within SMIs, which in turn can create new possibilities for interactive musical performance and multi-sensory experiences.

Through the research presented by this thesis, we addressed the research questions initially identified in Section 1.2 successfully. We presented an RNN-based RTPD system that is able to identify musical patterns despite expressive variations done to them. This was accomplished through the rule-based model to generate a synthetic training dataset, which included synthetic variations of patterns based on the actual variations present in the datasets created. The RNN-based method was packaged into a VST plugin that can run within a resource constrained embedded system with minimal latency. We demonstrated the positive perception of musicians through a user study with 20 musicians. We also demonstrated a total of three multi-sensory concerts where auditory, visual, and haptic stimuli were invoked in tandem to create a novel type of musical performances. The audience's, musicians, and composers feedback was obtained through questionnaires and interviews, which revealed a positive outlook on RTPD-based SMIs.

The development of Hot Licks - a RTPD system bundled within a VST plugin is a major technical achievement of this research. The system successfully integrated a RNN based algorithm into a VST plugin, which runs on an embedded system. The algorithm is able to run in real-time, with a minimal latency and a high accuracy. The time capability is also due to the integration with Elk Audio OS and optimization of data processing pipelines. The successful implementation of MIDI processing, audio analysis, and RNN inference within an embedded environment

demonstrates the feasibility of music technology tasks on resource constrained portable devices.

The evaluation results revealed good accuracy. The system achieves a good precision and recall in pattern detection. A precision of 0.723, recall of 0.67, and a F1 score of 0.663 was observed through the evaluation with musicians (Chapter 7). This research has thus successfully addressed the real-time detection of MIDI and audio patterns, in monophony as well as polyphony. However, the algorithms must be improved to achieve a much higher accuracy needed for deployment in professional performances.

This research also contributed to musical interfaces design, particularly SMIs. The integration of pattern detection into the instrument itself has the potential to create new possibilities for musical expression and interaction. The Hot Licks Mapper (Section 7.1.4) enables composers and musicians to map the output of SMIs to Musical Things, and change mappings in real-time. Moreover, development of the Hot Licks Controller (Section 5.5) gives the ability to musicians with little technical background to use the system. The edge device inference and cloud-based training (Section 5.4) is able to isolate labor intense tasks to be computed offline.

9.2 Performance Evaluation and User Experience

This research has provided insights on how musicians would interact with RTPDs systems in real world performance contexts. The comprehensive user studies revealed important findings on the balance between technical capabilities and musical usability. The user studies conducted with musicians revealed that they enjoyed using the system, and they were satisfied with its performance. The user studies revealed that musicians were likely to integrate AI-based solutions into their performances, they would be happy to recommend AI-based solutions to other musicians, and the our system was easy to use.

The investigation into how musicians interacted with the system revealed that they can add pattern detection capabilities into their performances effectively. This fact is supported by qualitative and quantitative data showing positive user experiences while interacting with the system. The integration of RTPDs capabilities into live musical performances showed that the proposed technical innovations have the potential to enhance musical performance.

This research also explored the integration of musical pattern detection into multi-sensory performances. The developed system links detected patterns with various sensory stimuli to create novel forms of audience engagement. The Hot Licks Mapper (Section 7.1.4) - which maps the SMIs to Musical Things is also a novel contribution made by this research.

Moreover, the evaluation of the audience responses upon attending the multisensory concert presented in Chapter 8 revealed that the coherence between the music and multi-sensory Musical Things was higher in the performance with RTPD triggering as opposed to the performance with random triggering. The musicians involved in the multi-sensory experiment too expressed their high satisfaction and confidence in using the system. Upon conducting interviews with composers, they revealed that RTPD-based multi-sensory performances would be a novel addition

to traditional performances, and it would add value to their artistic compositions.

9.3 Future Implications and Research Directions

This research delivered a framework to implement pattern detection into embedded systems, with implications for the field of music technology. The work done led to the development of efficient algorithms and data processing pipelines. The open-source software, datasets, and frameworks has the potential to be valuable resources for future development in SMI design.

The research presented by this thesis has implications for the IoMusT field by enabling performer-audience communications. The technology also has the potential to create novel forms of audience participation, where the audience feedback can be integrated into the performance in real-time. Moreover, this research facilitated the creation of three SMIs that are equipped with embedded RTPD. Moreover, this research demonstrated the successful execution of multi-sensory performances, an application envisioned in the IoMusT paradigm.

The future technical development of this work could focus on expanding the pattern detection to handle more complex musical structures. Research into the inclusion of timbral characteristics is another possible technical advancement. Another potential development is an increased accuracy. Despite the evaluation results showing a good accuracy, further optimization is needed to achieve an exceptional accuracy suited for demanding live performances. Research suggests numerous possibilities for expanding the applications of pattern detection systems. These include novel ways of mapping patterns to musical and sensory outputs and creating more sophisticated interactive performance systems.

The broader implications of this research extend beyond the immediate field of music technology, and contributes to the understanding of human-computer interaction. The work presented by this thesis emphasizes the role of technology in facilitating collaboration between composers, performers, and audience members through real-time coordination between multiple stakeholders (i.e., performers triggering Musical Things for the audience). The multi-sensory experiments revealed how multimodal systems can enhance the user experience and engagement in complex interactive systems. Moreover, the user studies identified possible applications to this system that is beyond music (i.e., the potential to use the RTPD system in teaching contexts).

The work presented by this thesis demonstrates the potential for technology to enhance creative expression while maintaining the essential qualities of musical performance, providing a strong foundation for future research and development in this rapidly evolving field.

Bibliography

- [1] *Logic Pro 9: User Manual*, California, 2011. [Online]. Available: [https://help.apple.com/logicpro/mac/9.1.6/en/logicpro/usermanual/Logic%20Pro%209%20User%20Manual%20\(en\).pdf](https://help.apple.com/logicpro/mac/9.1.6/en/logicpro/usermanual/Logic%20Pro%209%20User%20Manual%20(en).pdf)
- [2] L. Turchet, A. McPherson, and C. Fischione, “Smart Instruments: Towards an Ecosystem of Interoperable Devices Connecting Performers and Audiences,” in *Proceedings of the Sound and Music Computing Conference*, 2016, pp. 498–505.
- [3] L. Turchet, T. West, and M. M. Wanderley, “Smart Mandolin and Musical Haptic Gilet: effects of vibro-tactile stimuli during live music performance,” in *Proceedings of Audio Mostly Conference*, 2019, pp. 168–175, place: Nottingham, United Kingdom.
- [4] L. Turchet, “Smart Mandolin: autobiographical design, implementation, use cases, and lessons learned,” in *Proceedings of Audio Mostly Conference*, 2018, pp. 13:1–13:7, place: Wrexham, United Kingdom. [Online]. Available: <http://doi.acm.org/10.1145/3243274.3243280>
- [5] L. Turchet, C. Fischione, G. Essl, D. Keller, and M. Barthet, “Internet of Musical Things: Vision and Challenges,” *IEEE Access*, vol. 6, pp. 61 994–62 017, 2018, publisher: IEEE.
- [6] D. Temperley, “Melodic pattern repetition and efficient encoding: A corpus study,” *Empirical Musicology Review*, vol. 18, no. 2, pp. 97–116, 2023.
- [7] P. van Kranenburg, A. Volk, and F. Wiering, “A Comparison between Global and Local Features for Computational Classification of Folk Song Melodies,” *Journal of New Music Research*, vol. 42, no. 1, pp. 1–18, 2013, publisher: Routledge.

- [8] R. Dannenberg and N. Hu, “Linear time for discovering non-trivial repeating patterns in music databases,” in *ISMIR 2002 Conference Proceedings: Third International Conference on Music Information Retrieval*, 2002, pp. 63–70.
- [9] I. Y. Ren, H. V. Kooops, A. Volk, and W. Swierstra, “In Search of the Consensus Among Musical Pattern Discovery Algorithms,” in *Proceedings of the 18th International Society for Music Information Retrieval Conference*, 2017, pp. 23,27.
- [10] M. R. Jones, “Dynamic pattern structure in music: Recent theory and research,” *Perception & Psychophysics*, vol. 41, no. 6, pp. 621–634, Nov. 1987. [Online]. Available: <https://doi.org/10.3758/BF03210494>
- [11] L. Doris, “Musical Pattern Perception,” *College Music Symposium*, vol. 19, no. 1, pp. 166–182, 1979, publisher: The College Music Society. [Online]. Available: https://journals.scholarsportal.info/details/00695696/v19i0001/166_mpp.xml
- [12] D. Conklin, “Pattern in music,” *Journal of Mathematics and Music*, vol. 15, no. 2, pp. 95–98, May 2021. [Online]. Available: <https://doi.org/10.1080/17459737.2021.1947404>
- [13] T. Collins, J. Thurlow, R. Laney, A. Willis, and P. Garthwaite, “A comparative evaluation of algorithms for discovering translational patterns in Baroque keyboard works,” in *Proceedings of the International Symposium on Music Information Retrieval*, 2010.
- [14] D. Conklin and C. Anagnostopoulou, “Representation and Discovery of Multiple Viewpoint Patterns,” in *Proceedings of the 2001 International Computer Music Conference*, 2001, pp. 479–485.
- [15] J.-L. Hsu, C. Liu, and A. Chen, “Discovering nontrivial repeating patterns in music data,” *IEEE Trans. Multim.*, vol. 3, pp. 311–325, 2001.
- [16] I. Knopke and F. Jürgensen, “A System for Identifying Common Melodic Phrases in the Masses of Palestrina,” *Journal of New Music Research*, vol. 38, no. 2, pp. 171–181, 2009, publisher: Routledge.
- [17] C. Meek, “Automatic Thematic Extractor,” *Journal of Intelligent Information Systems*, vol. 21, pp. 9–33, 2003.
- [18] D. Stefani and L. Turchet, “On the challenges of embedded real-time music information retrieval,” in *Proceedings of the 25th International Conference on Digital Audio Effects (DAFx20in22)*, 09 2022.

- [19] N. Silva, C. Fischione, and L. Turchet, “Towards real-time detection of symbolic musical patterns: Probabilistic vs. deterministic methods,” in *2020 27th Conference of Open Innovations Association (FRUCT)*, 2020, pp. 238–246.
- [20] P. Meier, C.-Y. Chiu, and M. Müller, “A real-time beat tracking system with zero latency and enhanced controllability,” *Transactions of the International Society for Music Information Retrieval*, vol. 7, no. 1, pp. 213–227, 2024.
- [21] T.-P. Chen and L. Su, “Discovery of repeated themes and sections with pattern clustering,” in *Presented at the 18th International Society for Music Information Retrieval Conference*, 2017.
- [22] O. Nieto and M. Farbood, “Identifying polyphonic patterns from audio recordings using music segmentation techniques,” *Journal of New Music Research*, vol. 43, no. 1, pp. 1–20, 2014.
- [23] C. Finkensiep, M. Neuwirth, and M. Rohrmeier., “Generalized skipgrams for pattern discovery in polyphonic streams,” in *Proceedings of the 19th International Society for Music Information Retrieval Conference*, 2018, pp. 23–27.
- [24] M. Pesek, A. Leonardis, and M. Marolt, “Symchmmerge: an extension to the compositional hierarchical model for pattern discovery in symbolic music representations,” in *Presented at the 18th International Society for Music Information Retrieval Conference*, 2017.
- [25] D. Meredith, “Using siateccompress to discover repeated themes and sections in polyphonic music,” in *Presented at the 18th International Society for Music Information Retrieval Conference*, 2017.
- [26] O. Lartillot and P. Toivainen, “Motivic matching strategies for automated pattern extraction,” *Musicae Scientiae*, vol. 11, no. 1_suppl, pp. 281–314, 2007.
- [27] F. Jamshidi, G. Pike, A. Das, and R. Chapman, “Machine learning techniques in automatic music transcription: A systematic survey,” 06 2024.
- [28] B. Janssen, W. de Haas, A. Volk, and P. Van Kranenburg, “Discovering repeated patterns in music: state of knowledge, challenges, perspectives,” 10 2013.
- [29] A. Mesaros, T. Heittola, and T. Virtanen, “Metrics for polyphonic sound event detection,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 9, pp. 1500–1512, 2016.

- [30] O. Nieto and J. P. Bello, “Bridging the audio-symbolic gap: The discovery of repeated note content directly from polyphonic music audio,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 12, pp. 1760–1773, 2014.
- [31] J. Forth and G. Wiggins, “An approach for identifying salient repetition in multidimensional representations of polyphonic music,” *Journal of New Music Research*, vol. 38, no. 3, pp. 267–292, 2009.
- [32] A. Laaksonen and K. Lemström, “Discovering distorted repeating patterns in polyphonic music,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 29, no. 5, pp. 1234–1245, 2021.
- [33] —, “Discovering Distorted Repeating Patterns in Polyphonic Music,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 29, no. 5, pp. 1234–1245, 2021.
- [34] R. Paiva, T. Mendes, and A. Cardoso, “A methodology for detection of melody in polyphonic musical signals,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 18, no. 3, pp. 564–574, 2010.
- [35] M. Witek, E. Clarke, M. Kringelbach, and P. Vuust, “Polyphonic context and instrumentation in syncopation,” *Music Perception*, vol. 31, no. 3, pp. 234–250, 2014.
- [36] S. Edirisooriya, H. W. Dong, J. McAuley, and T. Berg-Kirkpatrick, “An empirical evaluation of end-to-end polyphonic optical music recognition,” *Pattern Recognition Letters*, vol. 145, pp. 1–8, 2021.
- [37] A. Volk and P. van Kranenburg, “Melodic similarity among folk songs: An annotation study on similarity-based categorization in music,” *Musicae Scientiae*, vol. 16, no. 3, pp. 317–339, 2012.
- [38] P. van Kranenburg, A. Volk, and F. Wiering, “A comparison between global and local features for computational classification of folk song melodies,” *Journal of New Music Research*, vol. 42, no. 1, pp. 1–18, 2013.
- [39] M. Thomas, Y. V. S. Murthy, and S. G. Koolagudi, “Detection of largest possible repeated patterns in indian audio songs using spectral features,” in *2016 IEEE Canadian Conference on Electrical and Computer Engineering*, 2016, pp. 1–5.
- [40] C.-i. Wang, J. Hsu, and S. Dubnov, “Music pattern discovery with variable markov oracle: A unified approach to symbolic and audio representations,” in *Proceedings of the 16th International Society for Music Information Retrieval Conference*, 2015.

-
- [41] S. Chakrabarty, R. Islam, E. Pricop, and H. Sarma, “An approach to discover similar musical patterns,” *IEEE Access*, vol. 10, pp. 47 322–47 339, 2022.
 - [42] E. Cakir, T. Heittola, H. Huttunen, and T. Virtanen, “Polyphonic sound event detection using multi label deep neural networks,” in *International Joint Conference on Neural Networks*, 2015, pp. 1–7.
 - [43] O. Dikmen and A. Mesaros, “Sound event detection using non-negative dictionaries learned from annotated overlapping events,” in *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2013, pp. 1–4.
 - [44] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “Panns: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.
 - [45] T. de Reuse and I. Fujinaga, “Pattern clustering in monophonic music by learning a non-linear embedding from human annotations,” in *Proceedings of the 20th International Society for Music Information Retrieval Conference*, 2019, pp. 761–768.
 - [46] I. Y. Ren, H. V. Koops, A. Volk, and W. Swierstra, “In search of the consensus among musical pattern discovery algorithms,” in *Proceedings of the 18th International Society for Music Information Retrieval Conference*, 2017, pp. 23,27.
 - [47] D. Stefani, S. Peroni, and L. Turchet, “A comparison of deep learning inference engines for embedded real-time audio classification,” in *Proceedings of the Digital Audio Effects Conference*, 2022.
 - [48] T. Pelinski, R. Diaz, A. L. B. Temprano, and A. McPherson, “Pipeline for recording datasets and running neural networks on the bela embedded hardware platform,” in *Proceedings of the International Conference on New Interfaces for Musical Expression*, 2023.
 - [49] L. Vignati, S. Zambon, and L. Turchet, “A comparison of real-time linux-based architectures for embedded musical applications,” *Journal of the Audio Engineering Society*, vol. 70, no. 1/2, pp. 83–93, 2022.
 - [50] A. McPherson and V. Zappi, “An environment for submillisecond-latency audio and sensor processing on beaglebone black,” in *Audio Engineering Society Convention 138*. Audio Engineering Society, 2015.
 - [51] R. Vieira, D. C. Muchaluat-Saade, and P. César, “Towards an internet of multisensory, multimedia and musical things (io3mt) environment,” in *2023 4th International Symposium on the Internet of Sounds*. IEEE, 2023, pp. 1–10.

- [52] C. Spence, “Musical scents: On the surprising absence of scented musical/auditory events, entertainments, and experiences,” *i-Perception*, vol. 12, no. 5, p. 20416695211038747, 2021.
- [53] R. Vieira, S. Wei, T. Rögglä, D. C. Muchaluat-Saade, and P. César, “Immersive io3mt environments: Design guidelines, use cases and future directions,” in *2024 IEEE 5th International Symposium on the Internet of Sounds (IS2)*. IEEE, September 2024, pp. 1–10.
- [54] R. Vieira, D. Muchaluat-Saade, and P. Cesar, “Exploring artificial intelligence for advancing performance processes and events in io3mt,” in *International Conference on Multimedia Modeling*. Cham: Springer Nature Switzerland, January 2024, pp. 234–248.
- [55] J. Park, Y. Choi, and K. M. Lee, “Research trends in virtual reality music concert technology: A systematic literature review,” *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [56] H. Yakura and M. Goto, “Enhancing participation experience in vr live concerts by improving motions of virtual audience avatars,” in *2020 IEEE international symposium on mixed and augmented reality (ISMAR)*. IEEE, 2020, pp. 555–565.
- [57] T. Venkatesan and Q. J. Wang, “Feeling connected: the role of haptic feedback in vr concerts and the impact of haptic music players on the music listening experience,” in *Arts*, vol. 12, no. 4. MDPI, 2023, p. 148.
- [58] G. W. Young, N. O’Dwyer, M. F. Vargas, R. M. Donnell, and A. Smolic, “Feel the music!—audience experiences of audio–tactile feedback in a novel virtual reality volumetric music video,” in *Arts*, vol. 12, no. 4. MDPI, 2023, p. 156.
- [59] K. Jung, S. Kim, S. Oh, and S. H. Yoon, “Hapmotion: motion-to-tactile framework with wearable haptic devices for immersive vr performance experience,” *Virtual Reality*, vol. 28, no. 1, p. 13, 2024.
- [60] L. Turchet, T. West, and M. M. Wanderley, “Touching the audience: Musical Haptic Wearables for augmented and participatory live music performances,” *Journal of Personal and Ubiquitous Computing*, pp. 1–21, 2020. [Online]. Available: <https://doi.org/10.1007/s00779-020-01395-2>
- [61] Z. Zhou, A. D. Cheok, W. Liu, X. Chen, F. Farbiz, X. Yang, and M. Haller, “Multisensory musical entertainment systems,” *IEEE MultiMedia*, vol. 11, no. 3, p. 88–101, Jul. 2004.

-
- [62] E. Lindström, A. Camurri, A. Friberg, G. Volpe, and M.-L. Rinman, “Affect, attitude and evaluation of multisensory performances,” *Journal of New Music Research*, vol. 34, no. 1, pp. 69–86, 2005.
- [63] L. Turchet, M. Benincaso, and C. Fischione, “Examples of use cases with smart instruments,” in *Proceedings of Audio Mostly Conference*, 2017, pp. 47:1–47:5. [Online]. Available: <https://doi.org/10.1145/3123514.3123553>
- [64] O. Kilic Afsar, Y. Luft, K. Cotton, E. R. Stepanova, C. Núñez Pacheco, R. Kleinberger, F. Ben Abdesslem, H. Ishii, and K. Höök, “Corsetto: A kinesthetic garment for designing, composing for, and experiencing an intersubjective haptic voice,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’23. New York, NY, USA: Association for Computing Machinery, 2023.
- [65] L. Turchet, F. Antoniazzi, F. Viola, F. Giunchiglia, and G. Fazekas, “The internet of musical things ontology,” *Journal of Web Semantics*, vol. 60, p. 100548, 2020. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S1570826820300019>
- [66] R. Masu, N. N. Correia, and T. Romão, “Technology-mediated musical connections: The ecology of a screen-score performance,” in *Proceedings of the 16th International Audio Mostly Conference*, ser. AM ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 109–116. [Online]. Available: <https://doi.org/10.1145/3478384.3478395>
- [67] D. Martin, S. Malpica, D. Gutierrez, B. Masia, and A. Serrano, “Multimodality in vr: A survey,” *ACM Comput. Surv.*, vol. 54, no. 10s, sep 2022. [Online]. Available: <https://doi.org/10.1145/3508361>
- [68] R. Dannenberg and N. Hu, “Pattern discovery techniques for music audio,” *Journal of New Music Research*, vol. 32, no. 2, pp. 153–163, 2003, publisher: Taylor & Francis. [Online]. Available: <https://doi.org/10.1076/jnmr.32.2.153.16738>
- [69] E. Chew, “COSMOS: Computational Shaping and Modeling of Musical Structures,” *Frontiers in Psychology*, vol. 13, May 2022, publisher: Frontiers. [Online]. Available: <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2022.527539/full>
- [70] L. A. Lanzendörfer, F. Grötschla, E. Funke, and R. Wattenhofer, “Disco-10m: A large-scale music dataset,” in *Advances in Neural Information Processing Systems*, ser. NeurIPS 2023 Datasets and Benchmarks, 2023.

- [71] J. Ens and P. Pasquier, “Building the metamidi dataset: Linking symbolic and audio musical data,” in *Proceedings of the International Society for Music Information Retrieval Conference*, ser. ISMIR 2021, 2021.
- [72] Y.-W. Hsiao, T.-Y. Hung, T.-P. Chen, and L. Su, “BPS-MOTIF: A DATASET FOR REPEATED PATTERN DISCOVERY OF POLYPHONIC SYMBOLIC MUSIC,” 2023.
- [73] T. Collins, “Discovery of repeated themes and sections,” in *Music Information Retrieval Evaluation eXchange (MIREX)*, ser. MIREX 2013, 2013.
- [74] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, “Enabling factorized piano music modeling and generation with the MAESTRO dataset,” in *International Conference on Learning Representations*, 2019.
- [75] T. Cherrier, T. Langlois, F. Piccoli, and A. Déchelotte, “Dadagp: A dataset of tokenized guitarpro songs for sequence models,” *arXiv preprint arXiv:2107.14653*, 2021.
- [76] C. Dittmar and M. Müller, “Real-time transcription and separation of drum recordings based on nmf decomposition,” in *Proceedings of the International Conference on Digital Audio Effects*, ser. DAFx-14, 2014.
- [77] O. Gillet and G. Richard, “Enst-drums: an extensive audio-visual database for drum signals processing,” in *Proceedings of the 7th International Conference on Music Information Retrieval*, ser. ISMIR 2006, 2006.
- [78] R. A. Moog, “Midi: Musical instrument digital interface,” *Journal of The Audio Engineering Society*, vol. 34, pp. 394–404, 1986. [Online]. Available: <https://api.semanticscholar.org/CorpusID:109225046>
- [79] D. M. Huber, *The MIDI manual*. USA: Sams, 1991.
- [80] 2017:discovery of repeated themes & sections. [Online]. Available: https://www.music-ir.org/mirex/wiki/2017:Discovery_of_Repeated_Themes_%26_Sections#What_is_a_Pattern.3F
- [81] A. Arronte Alvarez and F. Gómez, “Motivic pattern classification of music audio signals combining residual and lstm networks,” *International Journal of Interactive Multimedia and Artificial Intelligence*, 05 2021.
- [82] O. Melkonian, I. Ren, W. Swierstra, and A. Volk, “What constitutes a musical pattern?” in *Proceedings of the 7th ACM SIGPLAN International Workshop on Functional Art, Music,*

- Modeling, and Design*. New York, NY, USA: Association for Computing Machinery, 2019, p. 95–105.
- [83] F. Gómez, M. Tizón, A. Arronte Alvarez, and V. Padilla, “Rhetorical pattern finding,” *International Journal of Interactive Multimedia and Artificial Intelligence*, 11 2022.
 - [84] I. Shmulevich, O. Yli-Harja, E. Coyle, D. Povel, and K. Lemström, “Perceptual issues in music pattern recognition: Complexity of rhythm and key finding,” *Computers and the Humanities*, vol. 35, no. 1, pp. 23–35, 2001.
 - [85] R. Dannenberg, “Music Understanding by Computer,” *IAKTA/LIST International Workshop on Knowledge Technology in the Arts Proceedings*, 1993. [Online]. Available: https://www.academia.edu/51428655/Music_Understanding_by_Computer
 - [86] C. J. Plack, A. J. Oxenham, and R. R. Fay, *Pitch - Neural Coding and Perception*, 1st ed. New York: Springer Verlag, 2005.
 - [87] A. Klapuri and M. Davy, *Signal Processing Methods for Music Transcription*, 1st ed. New York: Springer Verlag, 2006.
 - [88] Official midi specifications: General midi 1. [Online]. Available: <https://www.midi.org/specifications/midi1-specifications/general-midi-specifications/general-midi-1>
 - [89] R. Dannenberg, “The interpretation of midi velocity,” in *Proceedings of the 2006 International Computer Music Conference, San Francisco, CA: The International Computer Music Association*, 2006, pp. 193–196.
 - [90] G. Read, *Music Notation: A Manual of Modern Practice*. Boston, Massachusetts: Allyn and Bacon, 1969.
 - [91] Official midi specifications: Midi tuning (updated specification). [Online]. Available: <https://www.midi.org/specifications/midi1-specifications/midi-1-addenda/midi-tuning-updated>
 - [92] B. Moore, *An introduction to the psychology of hearing*. Brill, 2012.
 - [93] J. Rothstein, *MIDI: A Comprehensive Introduction*. A-R Editions, 1995, vol. 3.
 - [94] A. V. Oppenheim, *Discrete-Time Signal Processing*. Pearson Education, 1999, google-Books-ID: geTn5W47KEsC.

- [95] J. Allen, "Short term spectral analysis, synthesis, and modification by discrete Fourier transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 25, no. 3, pp. 235–238, Jun. 1977, conference Name: IEEE Transactions on Acoustics, Speech, and Signal Processing. [Online]. Available: <https://ieeexplore.ieee.org/document/1162950>
- [96] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, Jul. 2002, conference Name: IEEE Transactions on Speech and Audio Processing. [Online]. Available: <https://ieeexplore.ieee.org/document/1021072>
- [97] Q. Liu, M. Xu, D. Zhang, and H. Zhang, "Cnn-based pitch detection and tracking for real-time music transcription," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 384–391.
- [98] R. Bittner, B. McFee, J. Salamon, P. Li, and J. P. Bello, "Deep salience representations for f0 estimation in polyphonic music," in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017, pp. 63–70.
- [99] J. Volkmann, S. Stevens, and E. Newman, "A scale for the measurement of the psychological magnitude pitch," *The Journal of the Acoustical Society of America*, vol. 8, pp. 208–208, 06 2005.
- [100] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004, conference Name: IEEE Transactions on Image Processing.
- [101] A. Prajapati, S. Naik, and S. Mehta, "Evaluation of Different Image Interpolation Algorithms," *International Journal of Computer Applications*, vol. 58, pp. 6–12, 10 2012.
- [102] V. Patel and K. Mistree, "A Review on Different Image Interpolation Techniques for Image Enhancement," *International Journal of Emerging Technology and Advanced Engineering*, vol. 3, pp. 129–133, 12 2013.
- [103] M. Patel and J. Wang, "Applications, challenges, and prospective in emerging body area networking technologies," *IEEE Wireless communications*, vol. 17, no. 1, pp. 80–88, 2010, publisher: IEEE.
- [104] N. Silva and L. Turchet, "A structural similarity index based method to detect symbolic monophonic patterns in real-time," in *Proceedings of the 25th International Conference on Digital Audio Effects*, 2022, pp. 161–168.

-
- [105] M. Wöllmer, M. Al-Hames, F. Eyben, B. Schuller, and G. Rigoll, “A multidimensional dynamic time warping algorithm for efficient multimodal fusion of asynchronous data streams,” *Neurocomputing*, vol. 73, no. 1, pp. 366–380, 2009.
 - [106] M. Shokoohi-Yekta, B. Hu, H. Jin, J. Wang, and E. Keogh, “Generalizing dtw to the multi-dimensional case requires an adaptive approach,” *Data Mining and Knowledge Discovery*, vol. 31, 01 2017.
 - [107] I. D. Mienye, T. G. Swart, and G. Obaïdo, “Recurrent Neural Networks: A Comprehensive Review of Architectures, Variants, and Applications,” *Information*, vol. 15, no. 9, p. 517, Sep. 2024, number: 9 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: <https://www.mdpi.com/2078-2489/15/9/517>
 - [108] M. Amerotti, S. Benford, B. Sturm, , and C. Vear, “A live performance rule system informed by irish traditional dance music,” in *Proceedings of the International Symposium on Computer Music Multidisciplinary Research*, 2023, pp. 1–12.
 - [109] NVIDIA, “Operationalizing pytorch models using onnx and onnx runtime,” 2024, accessed: 2024-12-11. [Online]. Available: <https://developer.nvidia.com/gtc/2020/video/s21368-vid>
 - [110] M. Openja, A. Nikanjam, A. H. Yahmed, F. Khomh, and Z. M. J. Jiang, “An Empirical Study of Challenges in Converting Deep Learning Models ,” in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. Los Alamitos, CA, USA: IEEE Computer Society, Oct. 2022, pp. 13–23. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICSME55016.2022.00010>
 - [111] L. Turchet, “Smart Mandolin: autobiographical design, implementation, use cases, and lessons learned,” in *Proceedings of Audio Mostly Conference*, 2018, pp. 13:1–13:7. [Online]. Available: <http://doi.acm.org/10.1145/3243274.3243280>
 - [112] L. Turchet, A. McPherson, and M. Barthet, “Co-design of a Smart Cajón,” *Journal of the Audio Engineering Society*, vol. 66, no. 4, pp. 220–230, 2018. [Online]. Available: <https://doi.org/10.17743/jaes.2018.0007>
 - [113] L. Turchet and C. Fischione, “Elk Audio OS: an open source operating system for the Internet of Musical Things,” *ACM Transactions on the Internet of Things*, vol. 2, no. 2, pp. 1–18, 2021, publisher: ACM.
 - [114] L. Turchet and S. Krstulović, “DSP as a Service: Foundations and Directions,” *IEEE Open Journal of the Communications Society*, vol. 5, pp. 6212–6226, 2024, conference Name: IEEE Open Journal of the Communications Society. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10684798>

- [115] What is Elk Audio OS? [Online]. Available: https://elk-audio.github.io/elk-docs/html/intro/how_elk_works.html. [Accessed:02.12.2024].
- [116] B. Moore, *An introduction to the psychology of hearing*. Brill, 2012.
- [117] vignati luca, zambon stefano, and turchet luca, “a comparison of real-time linux-based architectures for embedded musical applications,” *journal of the audio engineering society*, vol. 70, pp. 83–93, january 2022.
- [118] L. Turchet and C. Fischione, “Elk Audio OS: an open source operating system for the Internet of Musical Things,” *ACM Transactions on the Internet of Things*, vol. 2, no. 2, pp. 1–18, 2021.
- [119] Sushi The DAW Inside Elk. [Online]. Available: https://elk-audio.github.io/elk-docs/html/documents/sushi_overview.html. [Accessed:02.12.2024].
- [120] V. Braun and V. Clarke, “Reflecting on reflexive thematic analysis,” *Qualitative research in sport, exercise and health*, vol. 11, no. 4, pp. 589–597, 2019, publisher: Taylor & Francis.
- [121] S. Waters, “Performance Ecosystems: Ecological approaches to musical interaction,” *EMS: Electroacoustic Music Studies Network*, pp. 1–20, 2007.
- [122] R. Masu, M. Bettega, N. N. Correia, T. Romão, and F. Morreale, “ARCAA: a framework to analyse the artefact ecology in computer music performance,” in *Proceedings of the 9th International Conference on Digital and Interactive Arts*, 2019, pp. 1–9.
- [123] J. Malloch, S. Sinclair, and M. Wanderley, “Libmapper: (a Library for Connecting Things),” in *Extended Abstracts on Human Factors in Computing Systems*. ACM, 2013, pp. 3087–3090.
- [124] A. Fraietta, O. Bown, S. Ferguson, S. Gillespie, and L. Bray, “Rapid composition for networked devices: HappyBrackets,” *Computer Music Journal*, vol. 43, no. 2, pp. 89–108, 2019.
- [125] A. Fraietta, O. Bown, and S. Ferguson, “Transparent communication within multiplicities,” in *2020 27th Conference of Open Innovations Association (FRUCT)*. IEEE, 2020, pp. 61–72.
- [126] R. Vieira, D. C. Muchaluat-Saade, and F. L. Schiavoni, “Sunflower: An interactive artistic environment based on iomust concepts,” in *ACM International Conference on Interactive Media Experiences*, 2022, pp. 245–248.

- [127] R. Dannenberg, “O2: A network protocol for music systems,” *Wireless Communications and Mobile Computing*, vol. 2019, 2019.
- [128] L. Turchet and F. Antoniazzi, “Semantic web of musical things: Achieving interoperability in the internet of musical things,” *Journal of Web Semantics*, vol. 75, p. 100758, 09 2022.
- [129] B. Matuszewski, “A web-based framework for distributed music system research and creation,” *Journal of the Audio Engineering Society*, vol. 68, no. 10, pp. 717–726, 2020.
- [130] I. Clester and J. Freeman, “Composing the network with streams,” in *Proceedings of the Audio Mostly Conference*, 2021, pp. 196–199.
- [131] A. Yaseen and J. Timoney, “Possibilities emerging on the trajectory from iot to iomust: Enabling ubiquitous musical interactions for wellbeing,” in *EMPATHY: 3rd International Workshop on Empowering People in Dealing with Internet of Things Ecosystems. Workshop co-located with AVI 2022*, 2022.
- [132] K. Mikolajczyk, S. Ferguson, L. Candy, A. Dias Periera dos Santos, and O. Bown, “Space shaping in the design process for creative coding: a case study in media multiplicities,” *Digital Creativity*, pp. 1–21, 2024.
- [133] F. Visi, T. Basso, B. Greinke, E. Wood, P. Gschwendtner, C. Hope, and S. Östersjö, “Networking concert halls, musicians, and interactive textiles: Interwoven sound spaces,” *Digital Creativity*, pp. 1–22, 2024.
- [134] L. Turchet, T. West, and M. Wanderley, “Touching the audience: musical haptic wearables for augmented and participatory live music performances,” *Personal and Ubiquitous Computing*, vol. 25, 08 2021.
- [135] T. Mitchell, S. Madgwick, S. Rankine, G. Hilton, A. Freed, and A. Nix, “Making the most of wi-fi: Optimisations for robust wireless live music performance,” in *Proceedings of the Conference on New Interfaces for Musical Expression*, 2014, pp. 251–256.
- [136] P. Walker, “Cross-sensory correspondences: A theoretical framework and their relevance to music,” *Psychomusicology: Music, Mind, and Brain*, vol. 26, no. 2, p. 103, 2016.
- [137] O. Deroy and C. Spence, “Crossmodal correspondences: four challenges,” *Multisensory research*, vol. 29, no. 1-3, pp. 29–48, 2016.
- [138] O. Hödl, G. Fitzpatrick, and F. Kayali, “Design implications for technology-mediated audience participation in live music,” in *Proceedings of the Sound and Music Computing Conference*, 2017, pp. 28–34.

- [139] S. O'modhrain, "A framework for the evaluation of digital musical instruments," *Computer Music Journal*, vol. 35, no. 1, pp. 28–42, 2011, publisher: MIT Press. [Online]. Available: https://doi.org/10.1162/COMJ_a_00038

Appendix A

A.1 MIDI velocity and Musical dynamics

Table A.1: *MIDI velocity and dynamics mapping in Logic Pro 9 (retrieved from [\[1\]](#)).*

Name	Level	Marker	Velocity
fortississimo	very very loud	<i>fff</i>	127
fortissimo	very loud	<i>ff</i>	112
forte	loud	<i>f</i>	96
mezzo-forte	average	<i>mf</i>	80
mezzo-piano	average	<i>mp</i>	64
piano	quiet	<i>p</i>	48
pianissimo	very quiet	<i>pp</i>	32
pianississimo	very very quiet	<i>ppp</i>	16

A.2 General MIDI Percussion Key Map

For MIDI Channel 10, each MIDI KEY number ("NOTE#") corresponds to a different drum sound, as shown below. While many current instruments also have additional sounds above or below the range show here, and may even have additional "kits" with variations of these sounds, only these sounds are supported by General MIDI Level 1 devices.

Key#	Note	Drum Sound	Key#	Note	Drum Sound
35	B0	Acoustic Bass Drum	59	B2	Ride Cymbal 2
36	C1	Bass Drum 1	60	C3	Hi Bongo
37	C#1	Side Stick	61	C#3	Low Bongo
38	D1	Acoustic Snare	62	D3	Mute Hi Conga
39	E♭1	Hand Clap	63	E♭3	Open Hi Conga
40	E1	Electric Snare	64	E3	Low Conga
41	F1	Low Floor Tom	65	F3	High Timbale
42	F#1	Closed Hi Hat	66	F#3	Low Timbale
43	G1	High Floor Tom	67	G3	High Agogo
44	A♭1	Pedal Hi-Hat	68	A♭3	Low Agogo
45	A1	Low Tom	69	A3	Cabasa
46	B♭1	Open Hi-Hat	70	B♭3	Maracas
47	B1	Low-Mid Tom	71	B3	Short Whistle
48	C2	Hi Mid Tom	72	C4	Long Whistle
49	C#2	Crash Cymbal 1	73	C#4	Short Guiro
50	D2	High Tom	74	D4	Long Guiro
51	E♭2	Ride Cymbal 1	75	E♭4	Claves
52	E2	Chinese Cymbal	76	E4	Hi Wood Block
53	F2	Ride Bell	77	F4	Low Wood Block
54	F#2	Tambourine	78	F#4	Mute Cuica
55	G2	Splash Cymbal	79	G4	Open Cuica
56	A♭2	Cowbell	80	A♭4	Mute Triangle
57	A2	Crash Cymbal 2	81	A4	Open Triangle
58	B♭2	Vibraslap			

Table A.2: MIDI Drum Mapping

A.3 List of musicians Involved in User Study I

Table A.3: Details of the musicians who participated in the user study.

	Instrument	Age	Sex	Musical Background
Musician 1	Guitar	33	Male	Contemporary rock and pop guitar, guitar teacher, session and recording musician, composer
Musician 2	Piano	21	Female	Western classical music student, composer
Musician 3	Piano	23	Female	Western classical music student, composer
Musician 4	Guitar	36	Male	Classical guitar teacher, solo musician, member of the symphony orchestra, composer
Musician 5	Piano	28	Male	Contemporary rock and pop and jazz, solo pianist, session and recording musician, composer
Musician 6	Guitar	24	Male	Flamenco guitarist, classical guitar student, solo performer, composer
Musician 7	Guitar	19	Male	Eastern classical music student, member of a disabled musical group, composer
Musician 8	Piano	19	Male	Eastern classical music student, member of a disabled musical group, composer
Musician 9	Piano	29	Male	Session and recording guitarist, composer, music producer, DJ
Musician 10	Guitar	31	Female	Session and recording guitarist, composer, music producer, DJ
Musician 11	Piano	34	Male	Contemporary rock and pop and jazz, solo pianist, session and recording musician, composer, producer

A.3. List of musicians Involved in User Study I

Musician 12	Piano	34	Male	Contemporary rock and pop and jazz, solo pianist, session and recording musician, composer, producer
Musician 13	Piano	33	Male	Contemporary rock and pop and jazz, solo pianist, session and recording musician, composer, producer
Musician 14	Piano	32	Male	Contemporary rock and pop and jazz, solo pianist, session and recording musician, composer, producer
Musician 15	Guitar	29	Male	Classical and jazz guitarist, composer, session musician, producer
Musician 16	Guitar	29	Male	Contemporary rock and pop guitar, guitar teacher, session and recording musician, composer
Musician 17	Guitar	31	Male	Contemporary rock and pop guitar, guitar teacher, session and recording musician, composer
Musician 18	Piano	26	Female	Western classical music student, composer
Musician 19	Guitar	25	Male	Western classical music student, composer
Musician 20	Guitar	25	Male	Western classical music student, composer
Musician 21	Guitar	25	Male	Western classical music student, composer
Musician 22	Piano	24	Female	Western classical music student, composer

A.4 User Study I - results

Table A.4: Results of the user study I

	Improvisation no	Pattern 1 repetitions	Pattern 2 repetitions	True Positives	True Negatives	False Positives	False Negatives
Musician 1	0	9	4	12	95	5	1
	1	5	9	4	98	2	10
	2	7	11	15	91	9	3
Musician 2	0	6	9	6	100	0	9
	1	4	5	6	97	3	3
	2	9	6	7	100	0	8
Musician 3	0	9	4	11	92	8	2
	1	10	9	12	100	0	7
	2	10	10	14	93	7	6
Musician 4	0	2	5	5	99	1	2
	1	3	6	4	96	4	5
	2	18	3	16	99	1	5
Musician 5	0	6	6	12	96	4	0
	1	6	7	12	88	12	1
	2	3	9	11	90	10	1
Musician 6	0	6	6	10	94	6	2
	1	4	4	5	95	5	3
	2	7	7	5	100	0	9
Musician 7	0	3	3	6	100	0	0
	1	5	5	6	97	3	4
	2	6	8	5	96	4	9
Musician 8	0	3	3	3	99	1	3
	1	4	4	8	99	1	0
	2	6	7	5	100	0	8
Musician 9	0	5	9	12	94	6	2
	1	3	6	4	99	1	5

A.4. User Study I - results

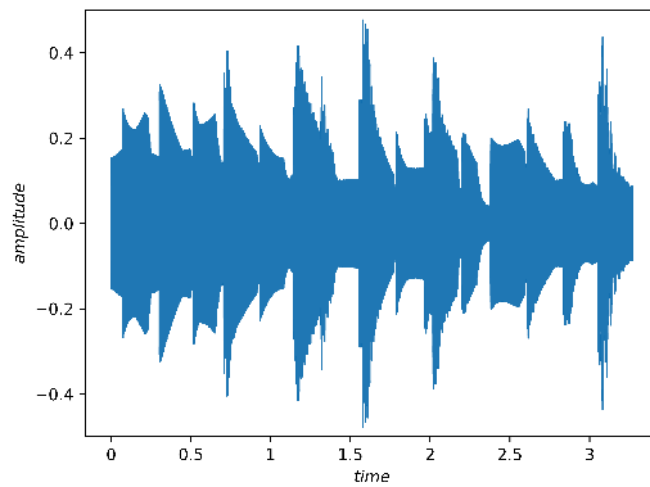
	2	6	5	3	97	3	8
Musician 10	0	7	8	10	94	6	5
	1	4	7	6	96	4	5
	2	3	5	7	97	3	1
Musician 11	0	5	4	6	94	6	3
	1	5	4	7	95	5	2
	2	6	4	8	96	4	2
Musician 12	0	4	5	4	97	3	5
	1	6	6	4	96	4	8
	2	7	3	9	98	2	1
Musician 13	0	3	2	2	100	0	3
	1	4	6	7	99	1	3
	2	5	4	5	100	0	4
Musician 14	0	8	6	11	98	2	3
	1	5	8	13	97	3	0
	2	4	6	8	97	3	2
Musician 15	0	4	4	8	98	2	0
	1	6	5	8	99	1	3
	2	5	5	5	99	1	5
Musician 16	0	4	5	8	92	8	1
	1	3	6	7	98	2	2
	2	4	5	5	100	0	4
Musician 17	0	6	6	12	89	11	0
	1	7	6	7	96	4	6
	2	6	6	12	90	10	0
Musician 18	0	2	7	5	97	3	4
	1	4	4	6	99	1	2
	2	6	5	3	98	2	8
Musician 19	0	8	12	12	95	5	8
	1	8	6	11	91	9	3
	2	9	8	15	90	10	2
Musician 20	0	5	3	2	98	2	6
	1	6	4	5	98	2	5
	2	7	7	5	95	5	9
Musician 21	0	5	6	3	100	0	8
	1	4	3	6	98	2	1
	2	3	8	8	97	3	3

Appendix B

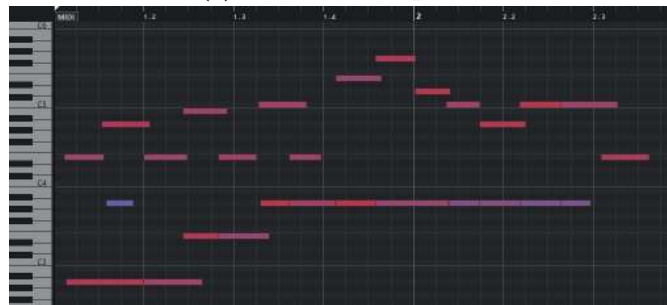
B.1 User Study I - Images and Plots

This section illustrates some of the sample recordings from the user study presented by Chapter 6. Figure B.1a shows one of the patterns a played by a keyboard player as a waveform. Figure B.1b shows the piano roll representation, and Figure B.1c shows the MFCC spectrogram of the same pattern.

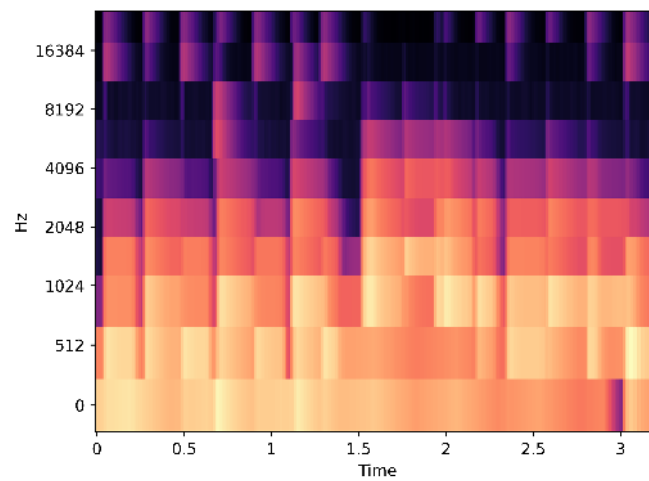
Similarly, Figure B.2a shows one of the patterns a played by a keyboard player as a waveform. Figure B.2b shows the piano roll representation, and Figure B.2 shows the MFCC spectrogram of the same pattern.



(a) Waveform of the pattern

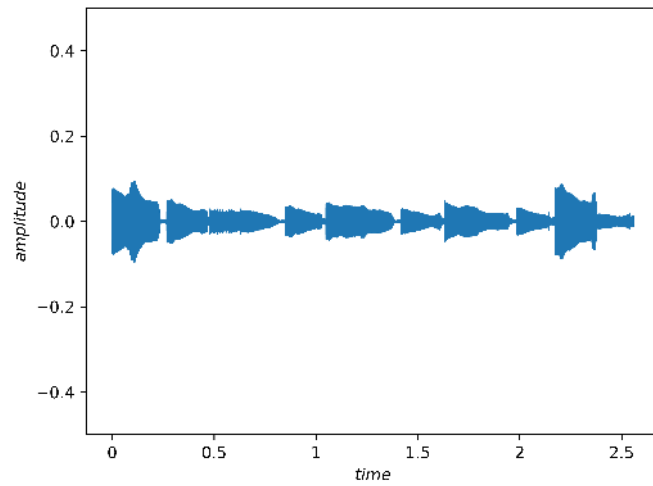


(b) MIDI piano roll display of the pattern

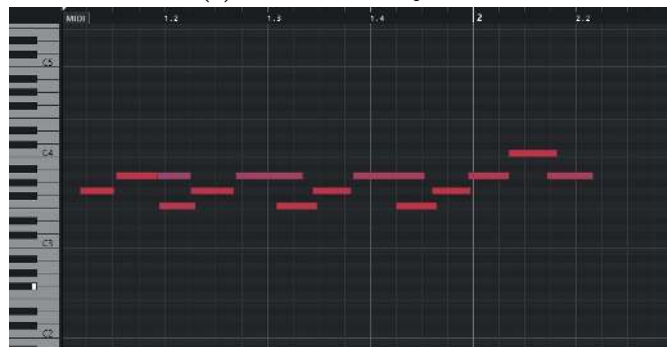


(c) Spectrogram of the pattern

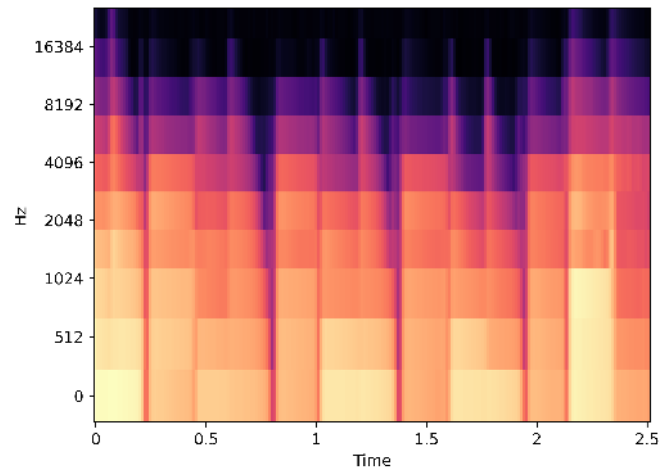
Figure B.1: A pattern played by one of the keyboard players during the study.



(a) Waveform of the pattern

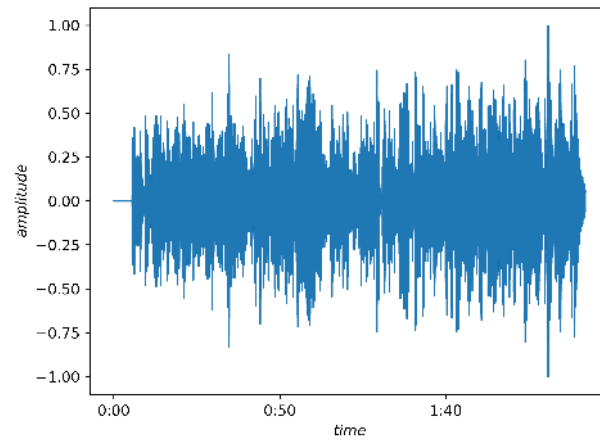


(b) MIDI piano roll display of the pattern

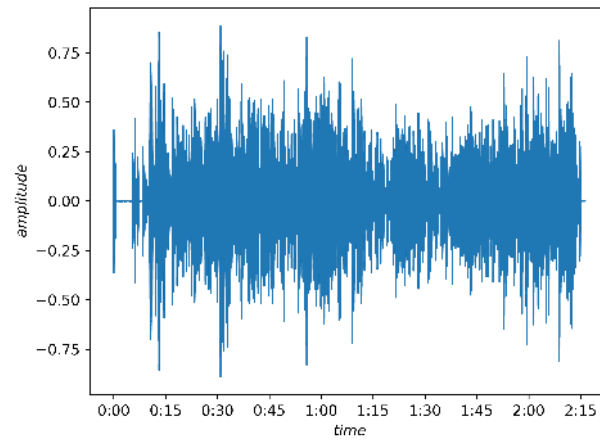


(c) Spectrogram of the pattern

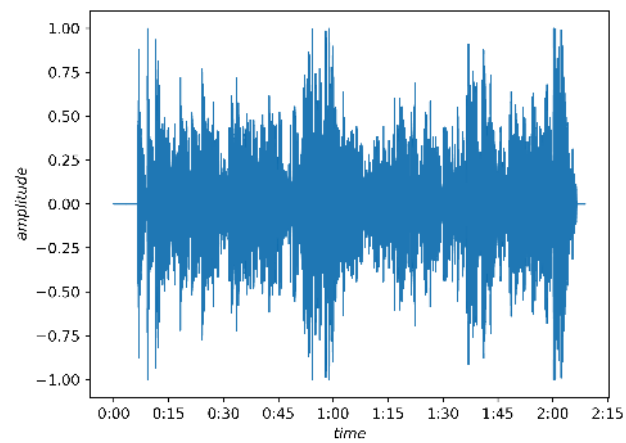
Figure B.2: A pattern played by one of the guitar players during the study.



(a) Track 1 improvisation

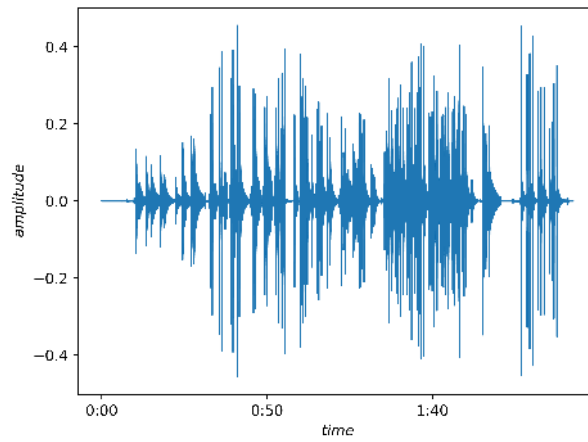


(b) Track 2 improvisation

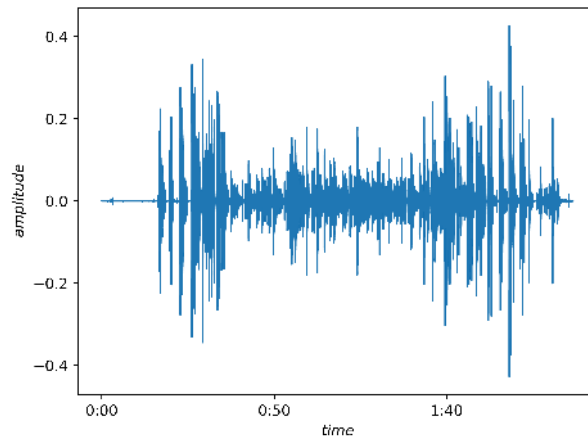


(c) Track 3 improvisation

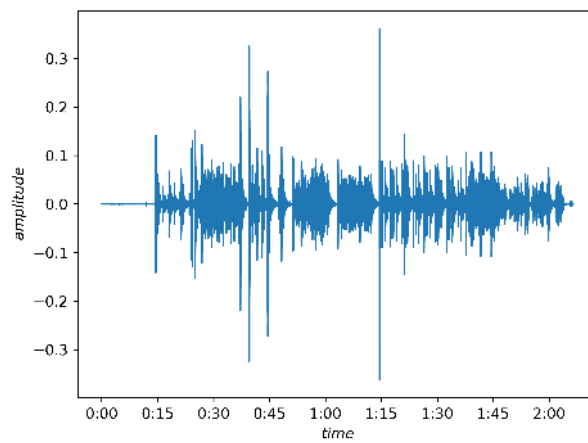
Figure B.3: The improvisations by one of the keyboard players



(a) Track 1 improvisation



(b) Track 2 improvisation



(c) Track 3 improvisation

Figure B.4: The improvisations by one of the guitar players

B.2 MM computation

To illustrate on the algorithm let us consider an example from the JKUPDD dataset, specifically Beethoven's Piano Sonata in F minor, and consider one repeating pattern, 29 notes in length. Let us name the pattern pitch vector as P_p , the amplitude vector as P_a , the pitch-bend vector as P_{pb} , and the duration vector as P_d :

$$\begin{aligned} P_p = [69, 69, 67, 65, 62, 64, 65, 67, 69, 62, 64, 65, \\ 67, 69, 65, 67, 67, 64, 65, 67, 67, 69, 65, 67, \\ 65, 64, 65, 64, 65] \end{aligned} \quad (\text{B.1})$$

$$\begin{aligned} P_a = [90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, \\ 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, \\ 90, 90, 90, 90, 90] \end{aligned} \quad (\text{B.2})$$

$$\begin{aligned} P_{pb} = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, \\ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] \end{aligned} \quad (\text{B.3})$$

$$\begin{aligned} P_d = [0.72, 0.24, 0.48, 0.48, 0.24, 0.24, 0.24, 0.24, \\ 0.48, 0.72, 0.24, 0.24, 0.24, 0.24, 0.24, 1.44, \\ 0.48, 0.48, 0.48, 0.72, 0.24, 0.48, 0.48, 0.72, \\ 0.24, 0.48, 0.96, 0.48, 0.48] \end{aligned} \quad (\text{B.4})$$

Within the piece, there is a repetition of the pattern where two doubled notes are present, making the sequence 31 notes long. Let us illustrate this sequence pitch vector as S_p , the amplitude vector as S_a , the pitch-bend vector as S_{pb} , and the duration vector as S_d :

$$\begin{aligned} S_p = [65, 69, 69, 67, 65, 62, 64, 65, 67, 69, 62, 64, \\ 65, 67, 69, 65, 67, 67, 64, 65, 67, 67, 69, 65, \\ 67, 65, 64, 65, 64, 65, 69] \end{aligned} \quad (\text{B.5})$$

$$\begin{aligned} S_a = [90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, \\ 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, 90, \\ 90, 90, 90, 90, 90, 90, 90] \end{aligned} \quad (\text{B.6})$$

B.3 Logic Flow of Hot Licks Controller

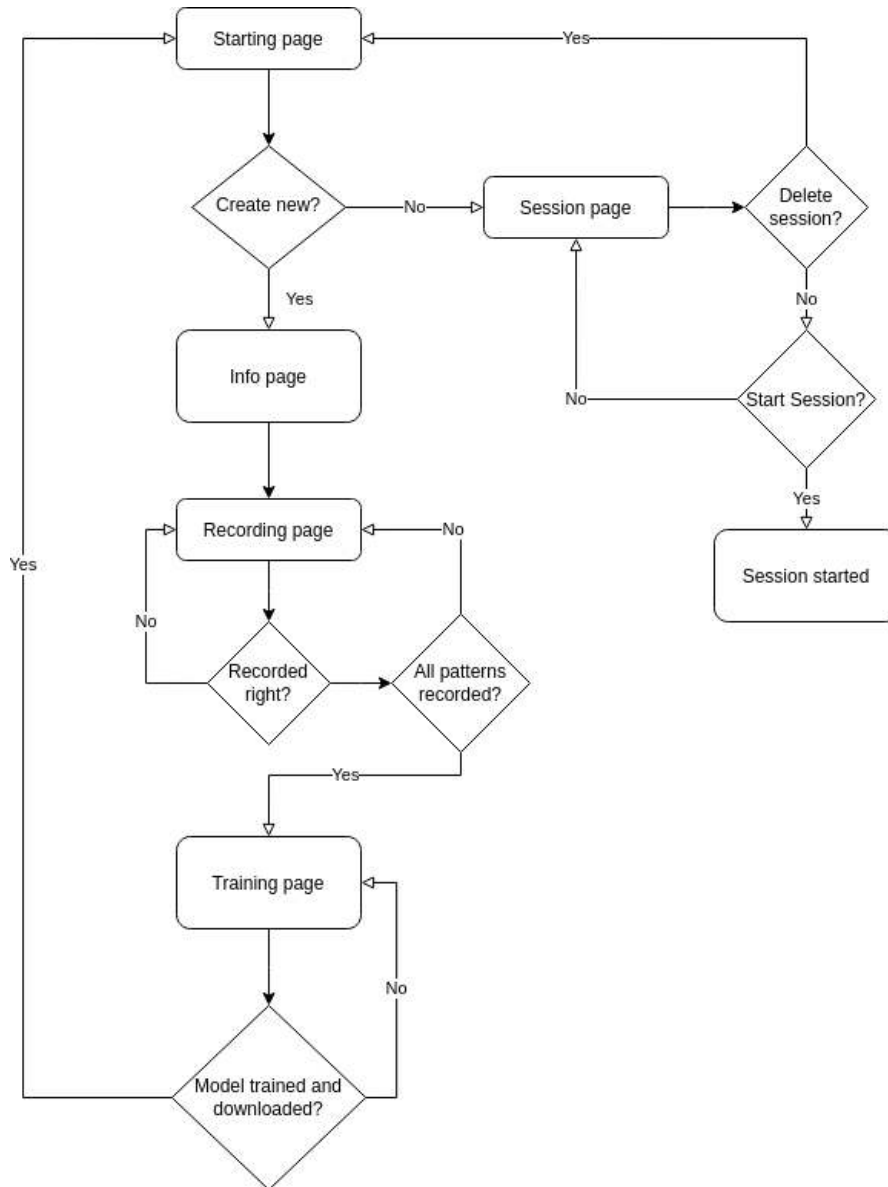


Figure B.5: Logic Flow of Hot Licks Controller

VALAR
MORGHULIS